



ACI Utility Manual

© 2006 by ACI Worldwide Inc. All rights reserved.

All information contained in this documentation is confidential and proprietary to ACI Worldwide Inc., and has been delivered for use pursuant to license. No part of this documentation may be photocopied, electronically transferred, modified, or reproduced in any manner that is contrary to license without the prior written consent of ACI Worldwide Inc.

Access Control Server, ACI Automated Case Management System, ACI Card Management System, ACI Claims Manager, ACI Commerce Gateway, ACI Debit Card System, ACI e-Courier, ACI Host Link, ACI Merchant Accounting System, ACI Payments Manager, ACI Proactive Risk Manager, ACI Smart Chip Manager, BASE24, BASE24-atm, BASE24-card, BASE24-check auth, BASE24 configuration manager, BASE24-es, BASE24-frequent shopper, BASE24-infobase, BASE24-pos, BASE24-refunds, BASE24-remote banking, BASE24-teller, NET24, and WINPAY24 are trademarks or registered trademarks of ACI Worldwide Inc., Transaction Systems Architects, Inc., or their subsidiaries.

Other companies' trademarks, service marks, or registered trademarks and service marks are trademarks, service marks, or registered trademarks and service marks of their respective companies.

Version Record

Nov-2006, Version 10.1

Table Of Contents

Section 1	1-1
Introduction	1-1
ACI Utilities Overview	1-1
Utility Procs – File Description	1-2
Procedures for Using Utilities	1-3
Syntax Conventions	1-5
Section 2	2-1
Arithmetic Overflow Prevention	2-1
ADD^DOUBLE	2-1
ADD^DOUBLEX (extended address version)	2-2
ADD^FIXED	2-3
ADD^FIXEDX (extended address version)	2-4
ADD^INT	2-5
ADD^INTX (extended address version)	2-6
MULT^DOUBLE	2-7
MULT^DOUBLEX (extended address version)	2-8
MULT^FIXED	2-9
MULT^FIXEDX (extended address version)	2-10
MULT^INT	2-11
MULT^INTX (extended address version)	2-12
Section 3	3-1
Bit Manipulation	3-1
CLEARBIT	3-1
CLEAR^BITX (extended address version)	3-2
SETBIT	3-3
SET^BITX (extended address version)	3-4
TESTBIT	3-5
TEST^BITX (extended address version)	3-6
Section 4	4-1
Circular Linked List Procs	4-1
Overview	4-1
CL^DELETE	4-2
CL^INSERT	4-3
Section 5	5-1
COBOL Processing	5-1
COBOL^ALTERPRIORITY	5-1
COBOL^CREATE	5-2
COBOL^DELAY	5-4
COBOL^GETCRTPID	5-5

COBOL^GETFILENUM.....	5-6
COBOL^LOOKUPPPD	5-7
COBOL^PURGE	5-8
COBOL^RENAME	5-9
Section 6.....	6-1
Date and Time Conversions	6-1
ASCII^JULIAN^TIMESTAMP	6-1
ASCII^TIMESTAMP	6-2
BUMP^DAY	6-3
BUMP^YYMMDD	6-4
CLOCK^TIME	6-5
COMPARE^ASCII^YYMMDD	6-6
COMPARE^BINARY^YYMMDD	6-8
CONTIMESTAMP	6-10
CONVERT^JULIAN^TO^GREG.....	6-11
CONVERT^TIME^TO^YYMMDD	6-12
CONVERT^TIMESTAMP^FORMAT	6-13
CONVERT^YYMMDD^TO^TIME	6-14
CURRENT^TIME^AS^STRING.....	6-15
DAY^OF^WEEK.....	6-16
GET^DAY^OF^WEEK.....	6-17
GET^NEXT^DAY	6-18
JULIAN	6-19
JULIAN^DATE	6-20
JULIAN^TIMESTAMP^ASCII	6-21
JULIAN^TO^YYMMDD	6-22
LEAP^YEAR	6-23
LONG^DATE^AND^TIME	6-24
TEST^HOLIDAY	6-25
TEST^WEEKEND	6-26
TIMESTAMP^ADJUST^NUM^MINUTES	6-27
TIMESTAMP^ADJUST^NUM^MINUTES^DBL	6-28
TIMESTAMP^ASCII	6-29
TIME^ASCII	6-30
TIME^TIMESTAMP.....	6-32
TIME^ZONE^CHANGE.....	6-33
TIME^ZONE^CHANGEX (extended address version)	6-35
TODAYS^DATE.....	6-37
VALID^DAT^TIM^STRING.....	6-38
VALID^DATE^YYMMDD	6-40
VALID^TIM^STRING	6-41
YYMMDD^TO^JULIAN	6-43
YY^DDD^TO^TIME.....	6-44

Section 7	7-1
Debugging Procs	7-1
HEX^DUMP	7-1
PEP^OFFSET	7-2
TRACER	7-3
TRACER printout example:.....	7-5
TRACK.....	7-6
Section 8	8-1
Encryption/Decryption Procs.....	8-1
DECODE	8-1
DECODEX (extended address version).....	8-3
DECRYPT.....	8-5
DECRYPT^PIN	8-6
DECRYPT^PIN^1	8-7
ENCODE	8-8
ENCODEX (extended address version).....	8-10
ENCRYPT.....	8-12
PROCESS^DES^PIN.....	8-13
Section 9	9-1
Format Conversion Procs	9-1
ALL^ASCII^DOUBLE	9-1
ALL^ASCII^FIXED	9-2
ALL^ASCII^INTEGER.....	9-3
ALL^HEX	9-4
ALL^NUMERIC.....	9-5
ALL^NUMERICX (extended address version)	9-6
ASCII^DOUBLE	9-7
ASCII^DOUBLEX (extended address version)	9-8
ASCII^FIXED	9-9
ASCII^FIXEDX (extended address version)	9-10
ASCII^INTEGER.....	9-11
ASCII^INTEGERX (extended address version)	9-12
ASCII^TO^D2230	9-13
ASCII^TO^EBCDIC.....	9-14
ASCII^TO^EBCDICX (extended address version).....	9-15
ASTERISK^FILL	9-16
BASE64^DECODE	9-17
BASE64^ENCODE	9-18
BASE94^DECODE	9-19
BASE94^ENCODE	9-20
BINARY^HEXCHAR	9-21
BINARY^HEXCHARX (extended address version).....	9-22
CRNCY^DECIMAL^PLACES.....	9-23
GET^CNTRY^DATA.....	9-24
GET^CRNCY^CDE^ALPHA.....	9-25
CRNCY^DECIMAL^PLACES^ALPHA.....	9-26

DOLLAR^SUPPRESS	9-27
DOUBLE^ASCII	9-28
DOUBLE^ASCIIX (extended address version)	9-29
DOUBLE^DOLLAR	9-30
DOUBLE^DOLLAR^ASTERISK	9-32
EBCDIC^TO^ASCII	9-34
EBCDIC^TO^ASCIIX (extended address version)	9-35
FIXED^ASCII	9-36
FIXED^ASCIIX (extended address version)	9-37
FIXED^DOLLAR	9-38
FIXED^DOLLAR^ASTERISK	9-39
FNUMIN	9-40
FNUMOUT	9-41
FORMATTED^FNUMOUT	9-43
GRAPHIC^HEX	9-45
HEXCHAR^BINARY	9-46
HEXCHAR^BINARYX (extended address version)	9-48
HEXCHAR^INTEGER	9-50
INTEGER^ASCII	9-51
INTEGER^ASCIIX (extended address version)	9-52
INTEGER^HEXCHAR	9-53
SET^PARITY	9-54
TRANSLATE	9-55
TRANSLATEX (extended address version)	9-56
ZDNUMIN	9-57
ZDNUMOUT	9-58
ZNUMOUT	9-59
Section 10.....	10-1
Manipulation.....	10-1
String Manipulation	10-1
CONVERT^FIELD^JUSTIFICATION	10-1
CONVERT^STATE^CODE	10-2
DELETE^EXTRA^BLANKS	10-3
FIELD^FINDER	10-4
FIND^STRING	10-6
FORMAT^DECIMAL	10-8
LEFT^JUSTIFY	10-10
MOD10^ADD	10-11
MOD10^ADDX (extended address version)	10-12
MOD10^SUBTRACT	10-13
MOD10^SUBTRACTX (extended address version)	10-14
PACK	10-15
REMOVE^BLANKS	10-17
REMOVE^NON^NUMERIC	10-18
RIGHT^JUSTIFY	10-19
STRLEN	10-20

STRLENX (extended address version)	10-21
SYM^NAME^LEN	10-22
UNPACK	10-23
VALID^DECIMAL^NUMBER	10-25
ZERO^SUPPRESS	10-26
Data Manipulation	10-27
DEFINES	10-28
UPSHIFT^FIELD	10-30
Section 11	11-1
Timer Routines	11-1
Overview	11-1
User Data Segment Timer Routines	11-2
DELETE^THIS^TIMER	11-5
FIND^SPECIFIC^TIMER	11-6
NEXT^EXPIRE^TIME	11-7
TIMER^DELETE	11-8
TIMER^DELETE^1	11-9
TIMER^EXPIRED	11-11
TIMER^INIT	11-12
TIMER^INSERT	11-13
Extended Memory Segment Timer Routines	11-14
DELETE^THIS^TIMERX	11-17
FIND^SPECIFIC^TIMERX	11-18
NEXT^EXPIRE^TIMEX	11-19
TIMER^ALLOCATEX	11-20
TIMER^DELETEx	11-21
TIMER^EXPIREDX	11-23
TIMER^INITX	11-24
TIMER^INSERTX	11-26
Section 12	12-1
Miscellaneous Procs	12-1
BINARY^SEARCH	12-1
CIRC^LEFT^SHIFT	12-3
CIRC^RIGHT^SHIFT	12-4
COMPLEMENT	12-5
CONVERT^REC^ADDR^TO^REF^NO	12-6
CONVERT^REF^NO^TO^REC^ADDR	12-7
CURRENT^ADDR	12-8
DBL^OCTAL^WIDTH	12-9
DBL^WIDTH	12-10
EDITLINENO	12-11
FUNCTION^KEY	12-12
GET^NEXT^ARG	12-13
MOTHER	12-15
MSG^AGE	12-16
OCTAL^WIDTH	12-17

Table Of Contents

RANDOM	12-18
SHA1^HASH	12-19
TRACK^LEN	12-20
UT^FNAMECOLLAPSE	12-21
WIDTH	12-22

Section 1

Introduction

ACI Utilities Overview

The ACI Utilities are a collection of “utility” PROCs distributed on the ACIUTILS subvolume. These utilities are considered “release independent” of ACI products. The utilities are contained in a BINDable object file, and are distributed only in object file form. Contained on the same subvolume are ACI Utilities external files and C language prototype files, documentation files, as well as the Supercrypt Utility files. The utility PROCs are described in this document. For information on the Supercrypt Utility files see the *BASE24 Transaction Security Manual*. The contents of this subvolume are briefly described below:

CLUTILE	C Language externals – Large model.
CWUTILE	C Language externals – Wide model.
D30UTILB	NSK D30 version of UTILB.
KEYUTIL	Supercrypt Library.
KEYUTILE	External file for KEYUTIL
SKCRYPT	Supercrypt Tool.
SYMSGS	TAL source code with STRUCTs for system messages.
UTILB	ACI Utility Library, NSK C30 version by default.
UTILDEFS	ACI Utility Library DEFINES and STRUCTs.
UTILEXTS	External declarations for ACI Utility Library.
UTILHIST	History of changes to ACI Utility Library Source File.
UTILUPDT	Documentation detailing changes to ACIUTILS subvolume contents.

This manual contains a section giving examples of how to use Compaq’s NSK BIND program to BIND in the utilities you require.

It is important to note that many of the procedures in this manual remain for backward compatibility. Before using procedures in ACI Utilities for new development, it is important to do some research. It may be that better alternatives have been developed in an ACI Product specific utility library (i.e. BASE Utilities or ATM Utilities), or that Compaq has introduced a comparable procedure into the NSK run time library.

Utility Procs – File Description

The utility PROCs have been grouped into files on \$<vol>.ACIUTILS. For the purposes of this document, \$<vol> represents the volume on a given system where ACIUTILS is installed. Since most ACI products refer to ACIUTILS using file defines, the location isn't critical. There are twelve files that make up the utility subvol. Following is a description of their contents.

- | | |
|--------------|---|
| CLUTILE | C Language externals – Large model. This file contains C programming language prototypes for ACI Utilities. |
| 1. CWUTILE | C Language externals – Wide model. This file contains C programming language prototypes for ACI Utilities. |
| 2. D30UTILB | NSK D30 version of UTILB. This is a BINDable object file compiled with Compaq's NSK D30 operating system compilers. |
| 3. KEYUTIL | Supercrypt Library. The Supercrypt Library and Tools are described in the <i>BASE24 Transaction Security Manual</i> . |
| 4. KEYUTILE | External file for KEYUTIL |
| 5. SKCRYPT | Supercrypt Tool. |
| 6. SYSMSGs | TAL source code with STRUCTs for system messages. |
| 7. UTILB | ACI Utility Library, currently Compaq's NSK C30 version. |
| 8. UTILDEFS | ACI Utility Library DEFINEs and STRUCTs. |
| 9. UTILEXTS | TAL External declarations for ACI Utility Library. |
| 10. UTILHIST | History of changes to ACI Utility Library Source File. Since the utilities are not distributed in source file form, the history sections are distributed to customers in this file. |
| 11. UTILUPDT | Documentation detailing changes to ACIUTILS subvolume contents. As files on ACIUTILS are added and removed, these changes should be documented here. |

Procedures for Using Utilities

The utility library configuration has been designed to facilitate the use of the NSK BIND program.

For the compilation of your source program, you may need \$<vol>.ACIUTILS.UTILEXTS, the externals file and depending on usage \$<vol>.ACIUTILS.UTILDEFS, if TAL structures and defines are required, and optionally, \$<vol>.ACIUTILS.CWUTILE if a C language source file (wide model) is being used. The files needed to run BIND are \$<vol>.ACIUTILS.UTILB and the object code file. In these examples, the following NSK TACL File Defines will be used to point to these source files.

```
TACL> add define =aciutils_utilexts, file $<vol>.ACIUTILS.UTILEXTS
TACL> add define =aciutils_cwutile,  file $<vol>.ACIUTILS.CWUTILE
TACL> add define =aciutils_utilb,    file $<vol>.ACIUTILS.UTILB
TACL> add define =aciutils_utildefs, file $<vol>.ACIUTILS.UTILDEFS
```

Following is an example of how to use NSK BIND. For more information on the different commands and their parameters, see Compaq's *Binder Manual*.

Example:

Following is a description of how to BIND in the library utilities ASCII^INTEGER, ADD^DOUBLE, and WIDTH. These utilities were only chosen as an example. These procedures will work for any of the utilities.

To compile the source program that refers to any of the library utilities, the external declaration for each utility needed must be SOURCED in. For this example you would need:

```
?source =ACIUTILS_UTILEXTS (
?   ASCII^INTEGER,
?   ADD^DOUBLE,
?   WIDTH )
```

The C Language equivalent is:

```
#include "=aciutils_cwutile ( \
    ASCII_INTEGER,           \
    ADD_DOUBLE,              \
    WIDTH )" NOLIST
```

Note: The carrot character (^) is changed to an underscore character (_) in C. The prototypes are specified in both all upper and all lower case. Since C is case sensitive both options are provided. The NOLIST directive is optional.

The next step is to create an edit file (BIND can be run interactively, but an edit file allows you to reuse the commands without having to type them in each time). This file should contain the following information:

```
ADD * FROM <yourfile>           ①
SELECT SEARCH =ACIUTILS_UTILB   ②
BUILD newfile!                  ③
```

- ① Selects all PROCs, global code and global data from your object file.
- ② Instructs BIND to resolve unresolved externals with code sections from =ACIUTILS_UTILB.
- ③ Creates a new object program file "newfile" containing the object code from "yourfile" and the utilities.

To execute these commands, enter:

```
TACL> BIND /IN editfile/
```

This will produce the new bound object code.

Several of the procedures require STRUCT definitions or DEFINES. These require the use of the definitions file on =ACIUTILS_UTILDEFS. Following is an example of the how to use the TIMER procs, which require STRUCT templates from the definitions file.

To compile your source program, you need to SOURCE the following statement into the section of your program where you define STRUCT templates:

```
?source =ACIUTILS_UTILDEFS( timer^structs )
```

Note: C Language equivalents are not available for data structures and defines in the ACI Utilities Sources.

You will also need to SOURCE in the following EXTERNAL declarations:

```
?source =ACIUTILS_UTILEXTS( TIMER^INIT,
?     NEXT^EXPIRE^TIME,
?     TIMER^DELETE,
?     TIMER^INSERT,
?     TIMER^EXPIRED,
?     FIND^SPECIFIC^TIMER )
```

To create the edit file containing the BIND commands, the same procedures can be used as described in the preceding example.

Syntax Conventions

The following is a summary of the characters and symbols used in the syntax notation for the utility procedures in this manual. The syntax standards follow those used by Tandem.

NOTATION	MEANING
UPPER-CASE CHARACTERS	All keywords appear in capital letters. A keyword is defined as one that must be present for the procedure to work properly.
<lower-case characters>	All variable entries supplied by the user are shown in lower-case characters and enclosed in angle brackets. If the entry is required, it is underlined; otherwise, it is optional.
Punctuation	All punctuation and symbols must be entered precisely as shown. The only exception is the repeat symbol (. . .) that appears in some procedures.
System Procedure Calls	Calls to the utility procedures are shown in the following form: <pre>CALL <procedure name> (<parameters>)</pre> or <pre><stat> := <procedure name> (<parameters>)</pre> CALL is a TAL CALL statement. "<stat> :=" indicates that procedure is a function procedure. (I.e., it returns a value of the indicated <type> when referenced in an expression.) <procedure name> is the name of the utility procedure. Required parts of the calling sequence are underlined. Optional parameters may be omitted, but placeholder commas (',') must be present except for right-side omitted parameters. A function procedure's return value is described as follows: <stat>, <type>

NOTATION	MEANING
	<type> is INT, INT(32), STRING, or FIXED [(<fpoint>)] A function procedure can be called with a CALL statement, but the return value will be lost. <parameters> are described as follows: <parameter>,<type>:ref:<num elements> or value <type> is INT, INT(32), STRING or FIXED [(<fpoint>)] 'ref' indicates a reference parameter. <num elements> indicates that the procedure returns a value of <type> to <parameter> for <num elements>. 'value' indicates a value parameter.

Section 2

Arithmetic Overflow Prevention

These routines are designed to prevent arithmetic overflow traps. There are routines for both addition and multiplication, on INT, DOUBLE, and FIXED variables.

ADD^DOUBLE

This routine was designed to prevent arithmetic overflow traps when adding several double-word variables. The routine returns the result of the operation in <result> if an overflow does not occur. If over-flow occurs, the PROC returns FALSE in <stat>.

Syntax:

<stat> := ADD^DOUBLE(<result> , <val1> , <val2> , <val3> , . . . <val8>)

where:

<stat>, INT,	returns FALSE (0) if an overflow occurs or if <result> is not passed as a parameter.
<result>, INT(32):ref,	if present, contains the result of the addition if no overflow occurs.
<val1>...<val8>, INT(32):ref,	are the possible double-words to be added. Any subset of these variables will be accepted. If none are present, <result> will be 0.

example:

stat := ADD^DOUBLE(result, val1, val2, val3, val4)

ADD^DOUBLEX (extended address version)

This routine was designed to prevent arithmetic overflow traps when adding several double-word variables. The routine returns the result of the operation in <resultx> if an overflow does not occur. If overflow occurs, the PROC returns FALSE in <stat>.

Syntax:

<stat> := ADD^DOUBLEX(<resultx> , <val1> , <val2> , <val3> , . . . <val8>)

where:

<stat>, INT,	returns FALSE (0) if an overflow occurs or if <resultx> is not passed as a parameter.
<resultx>, INT(32):ref,	if present, contains the result of the addition if no overflow occurs.
<val1>...<val8>, INT(32):ref,	are the possible double-words to be added. Any subset of these variables will be accepted. If none are present, <resultx> will be 0.

example:

stat := ADD^DOUBLEX(resultx, val1, val2, val3, val4)

ADD^FIXED

This routine was designed to prevent arithmetic overflow traps when adding several FIXED (64 bit) variables. The routine returns the result of the operation in <result> if an overflow does not occur. If overflow occurs, the PROC returns FALSE in <stat>. The user is responsible for decimal alignment.

Syntax:

<stat> := ADD^FIXED(<result>, <val1>, <val2> , <val3>, . . .<val8>)

where:

<stat>, INT,	returns FALSE (0) if an overflow occurs or if <result> is not passed as a parameter.
<result>, FIXED:ref,	if present, contains the result of the addition if no overflow occurs.
<val1>...<val8>, FIXED:ref,	are the possible FIXED variables to be added. Any subset of these variables will be accepted. If none are present, <result> will be 0.

example:

stat := ADD^FIXED(result, val1, val2, val3, val4)

ADD^FIXEDX (extended address version)

This routine was designed to prevent arithmetic overflow traps when adding several FIXED (64 bit) variables. The routine returns the result of the operation in <resultx> if an overflow does not occur. If overflow occurs, the PROC returns FALSE in <stat>. The user is responsible for decimal alignment.

Syntax:

<stat> := ADD^FIXEDX(<resultx>, <val1>, <val2> , <val3>,...<val6>)

where:

<stat>, INT,	returns FALSE (0) if an overflow occurs or if <resultx> is not passed as a parameter.
<resultx>, FIXED:ref,	if present, contains the result of the addition if no overflow occurs.
<val1>...<val6>, FIXED:ref,	are the possible FIXED variables to be added. Any subset of these variables will be accepted. If none are present, <result> will be 0.

example:

stat := ADD^FIXEDX(resultx, val1, val2, val3, val4)

ADD^INT

This routine was designed to prevent arithmetic overflow traps when adding several INT variables. The routine returns the result of the operation in <result> if an overflow does not occur. If overflow occurs, the PROC returns FALSE.

Syntax:

<stat> := ADD^INT(<result> , <val1> , <val2> , <val3> , <val4> , ... <val8>)

where:

<stat>, INT,	returns FALSE (0) if an overflow occurs or if <result> is not passed as a parameter.
<result>, INT:ref,	if present, contains the result of the addition if no overflow occurs.
<val1>...<val8>, INT:ref,	are the possible INT variables to be added. Any subset of these variables will be accepted. If no values are supplied, <result> is 0.

example:

stat := ADD^INT(result, val1, val2, val3, val4)

ADD^INTX (extended address version)

This routine was designed to prevent arithmetic overflow traps when adding several INT variables. The routine returns the result of the operation in <resultx> if an overflow does not occur. If overflow occurs, the PROC returns FALSE.

Syntax:

<stat> := ADD^INTX(<resultx> , <val1> , <val2> , <val3> , <val4> , ... <val8>)

where:

<stat>, INT,	returns FALSE (0) if an overflow occurs or if <resultx> is not passed as a parameter.
<resultx>, INT:ref,	if present, contains the result of the addition if no overflow occurs.
<val1>...<val8>, INT:ref,	are the possible INT variables to be added. Any subset of these variables will be accepted. If no values are supplied, <resultx> is 0.

example:

stat := ADD^INTX(resultx, val1, val2, val3, val4)

MULT^DOUBLE

This routine was designed to prevent arithmetic overflow traps when multiplying several double-word variables. The routine returns the result of the operation in <result> if an overflow does not occur. If overflow occurs, the PROC returns FALSE.

Syntax:

<stat> := MULT^DOUBLE(<result> , <val1> , <val2>, <val3>, . . . <val8>)

where:

<stat>, INT,	returns FALSE (0) if an overflow occurs or if <result> is not passed as a parameter.
<result>, INT(32):ref,	if present, contains the result of the multiplication if no overflow occurs.
<val1>...<val8>, INT(32):ref,	are the possible double-words to be multiplied. Any subset of these variables will be accepted. If none are present, <result> will be 1.

example:

stat := MULT^DOUBLE(result, val1, val2, val3)

MULT^DOUBLEX (extended address version)

This routine was designed to prevent arithmetic overflow traps when multiplying several double-word variables. The routine returns the result of the operation in <resultx> if an overflow does not occur. If overflow occurs, the PROC returns FALSE.

Syntax:

<stat> := MULT^DOUBLEX(<resultx> , <val1> , <val2>, <val3>, . . . <val8>)

where:

<stat>, INT,	returns FALSE (0) if an overflow occurs or if <resultx> is not passed as a parameter.
<resultx>, INT(32):ref,	if present, contains the result of the multiplication if no overflow occurs.
<val1>...<val8>, INT(32):ref,	are the possible double-words to be multiplied. Any subset of these variables will be accepted. If none are present, <resultx> will be 1.

example:

stat := MULT^DOUBLEX(resultx, val1, val2, val3)

MULT^FIXED

This routine was designed to prevent arithmetic overflow traps when multiplying several FIXED (64 bit) variables. The routine returns the result of the operation in <result> if an overflow does not occur. If overflow occurs, the PROC returns FALSE. The user is responsible for decimal alignment.

Syntax:

<stat> := MULT^FIXED(<result>, <val1>, <val2>, <val3>, . . . <val8>)

where:

<stat>, INT,	returns FALSE (0) if an overflow occurs or if <result> is not passed as a parameter.
<result>, FIXED(*):ref,	if present, contains the result of the multiplication if no overflow occurs.
<val1> . . . <val8>, FIXED(*):ref,	are the possible FIXED variables to be multiplied. Any subset of these variables will be accepted. If none are present, <result> will be 1.

example:

stat := MULT^FIXED(result, val1, val2, val3)

MULT^FIXEDX (extended address version)

This routine was designed to prevent arithmetic overflow traps when multiplying several FIXED (64 bit) variables. The routine returns the result of the operation in <resultx> if an overflow does not occur. If overflow occurs, the PROC returns FALSE. The user is responsible for decimal alignment.

Syntax:

<stat> := MULT^FIXEDX(<resultx>, <val1>, <val2>, <val3>, . . . <val6>)

where:

<stat>, INT,	returns FALSE (0) if an overflow occurs or if <resultx> is not passed as a parameter.
<resultx>, FIXED(*):ref,	if present, contains the result of the multiplication if no overflow occurs.
<val1> . . . <val6>, FIXED(*):ref,	are the possible FIXED variables to be multiplied. Any subset of these variables will be accepted. If none are present, <resultx> will be 1.

example:

stat := MULT^FIXEDX(resultx, val1, val2, val3)

MULT^INT

This routine was designed to prevent arithmetic overflow traps when multiplying several INT variables. The routine returns the result of the operation in <result> if an overflow does not occur. If overflow occurs, the PROC returns FALSE.

Syntax:

<stat> := MULT^INT(<result>, <val1>, <val2>, <val3> . . . <val8>)

where:

<stat>, INT,	returns FALSE (0) if an overflow occurs or if <result> is not passed as a parameter.
<result>, INT:ref,	if present, contains the result of the multiplication if no overflow occurs.
<val1>...<val8>, INT:ref,	are the possible INT variables to be multiplied. Any subset of these variables will be accepted. If no values are supplied, <result> is 1.

example:

stat := MULT^INT(result, val1, val2, val3, val4)

MULT^INTX (extended address version)

This routine was designed to prevent arithmetic overflow traps when multiplying several INT variables. The routine returns the result of the operation in <resultx> if an overflow does not occur. If overflow occurs, the PROC returns FALSE.

Syntax:

<stat> := MULT^INTX(<resultx>, <val1>, <val2>, <val3> . . . <val8>)

where:

<stat>, INT,	returns FALSE (0) if an overflow occurs or if <resultx> is not passed as a parameter.
<resultx>, INT:ref,	if present, contains the result of the multiplication if no overflow occurs.
<val1>...<val8>, INT:ref,	are the possible INT variables to be multiplied. Any subset of these variables will be accepted. If no values are supplied, <resultx> is 1.

example:

stat := MULT^INTX(resultx, val1, val2, val3, val4)

Section 3

Bit Manipulation

CLEARBIT

Clears a bit of a vector.

Syntax:

CALL CLEARBIT(<array> , <bit>)

where:

<array>, INT:ref,

is the bit vector to be modified.

<bit>, INT,

is the bit to be cleared; the value of <bit> must be in the range:

[0:(# words in <array> * 16) - 1].

example:

```
INT    bit^array[ 0:19 ],  
      bit;
```

```
bit := 25;  
CALL CLEARBIT( bit^array, bit );
```

NOTE: It is up to the user to insure that the value of <bit> lies in the defined range; this PROC has no knowledge of the declared length of <array>.

CLEAR^BITX (extended address version)

Clears a bit of a vector.

Syntax:

CALL CLEAR^BITX(<arrayx> , <bit>)

where:

<arrayx>, INT:ref, is the bit vector to be modified.

<bit>, INT, is the bit to be cleared; the value of <bit>
must be in the range:
[0:(# words in <array> * 16) - 1].

example:

```
INT    .ext bit^arrayx[ 0:19 ],  
        bit;
```

```
bit := 25;  
CALL CLEAR^BITX( bit^arrayx, bit );
```

NOTE: It is up to the user to insure that the value of <bit> lies in the defined range; this PROC has no knowledge of the declared length of <array>.

SETBIT

Sets a bit of a vector.

Syntax:

CALL SETBIT(<array> , <bit>)

where:

<array>, INT:ref, is the bit vector to be modified.

<bit>, INT, is the bit to be set; the value of <bit> must
be in the range:
[0:(# words in <array> * 16) - 1].

example:

```
INT    bit^array[ 0:19 ],  
      bit;  
  
bit := 25;  
CALL SETBIT( bit^array, bit );
```

NOTE: It is up to the user to insure that the value of <bit> lies in the defined range; this PROC has no knowledge of the declared length of <array>.

SET^BITX (extended address version)

Sets a bit of a vector.

Syntax:

CALL SET^BITX(<arrayx> , <bit>)

where:

<arrayx>, INT:ref, is the bit vector to be modified.

<bit>, INT, is the bit to be set; the value of <bit> must
be in the range:
[0:(# words in <array> * 16) - 1].

example:

```
INT    .ext bit^arrayx[ 0:19 ],  
        bit;
```

```
bit := 25;  
CALL SET^BITX( bit^arrayx, bit );
```

NOTE: It is up to the user to insure that the value of <bit> lies in the defined range; this PROC has no knowledge of the declared length of <array>.

TESTBIT

Returns a TRUE/FALSE value as determined by the state of the bit in question of a bit vector.

Syntax:

<stat> := TESTBIT(<array> , <bit>)

where:

<stat>, INT,	returns TRUE (non-zero) if the bit in question is set; otherwise, it returns FALSE (0).
<array>, INT:ref,	is the bit vector to be tested.
<bit>, INT,	is the bit to be tested; the value of <bit> must be in the range: [0:(# words in <array> * 16) - 1]

example:

```

INT    bit^array[ 0:19 ],
        bit;

bit := 25;
IF TESTBIT( bit^array, bit ) THEN
    ...    !bit on
ELSE
    ...    !bit off

```

NOTE: It is up to the user to insure that the value of <bit> lies in the defined range; this PROC has no knowledge of the declared length of <array>.

TEST^BITX (extended address version)

Returns a TRUE/FALSE value as determined by the state of the bit in question of a bit vector.

Syntax:

<stat> := TEST^BITX(<arrayx> , <bit>)

where:

<stat>, INT, returns TRUE (non-zero) if the bit in question is set; otherwise, it returns FALSE (0).

<arrayx>, INT:ref, is the bit vector to be tested.

<bit>, INT, is the bit to be tested; the value of <bit> must be in the range:
[0:(# words in <array> * 16) - 1]

example:

```
INT    .ext bit^arrayx[ 0:19 ],
        bit;

bit := 25;
IF TEST^BITX( bit^arrayx, bit ) THEN
    ...    !bit on
ELSE
    ...    !bit off
```

NOTE: It is up to the user to insure that the value of <bit> lies in the defined range; this PROC has no knowledge of the declared length of <array>.

Section 4

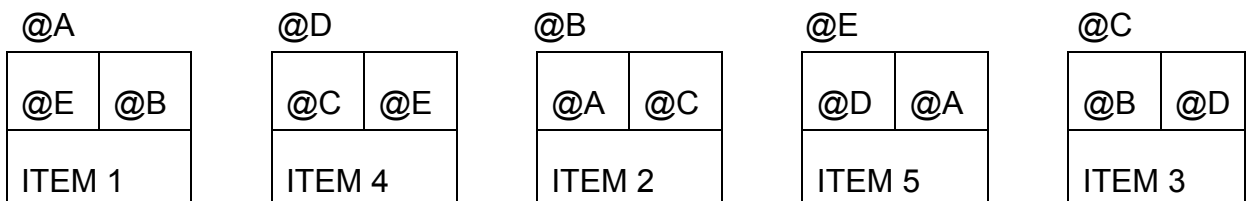
Circular Linked List Procs

Overview

An item in a circular linked list contains data coupled with pointers to the previous and next items in the list.

POINTER TO PREV	POINTER TO NEXT
DATA ELEMENT OR POINTER TO DATA	

When several of these items are combined, they may form a circular linked list similar to the following:



At the beginning of the linked list is an entry whose pointer to the previous item contains the address of the last item in the list. Its pointer to the next item contains the address of the item following it in the list. The next item pointer of the beginning entry in the linked list will point to the beginning entry itself if there are no entries in the circular linked list.

These two PROCs take care of the previous and next pointer linkages. The programmer must provide the definition of the linked list and the data to be kept in it (usually timers).

CL^DELETE

Deletes an item from a circular linked list by adjusting the appropriate linkages.

Syntax:

<stat> := CL^DELETE(<item>)

where:

<stat>, INT, returns FALSE (0) if the linkages are invalid;
otherwise, TRUE (-1) is returned.

<item>, INT, is the address of the item to be deleted. The
first two words at this address must be the
address of the previous entry, followed by
the address of the next entry.

example:

```
STRUCT                      .list^item[ 0:max ];
  BEGIN
    INT                      prev^item;
    INT                      nxt^item;
    STRING                   item^contents[ 0:contents^max ];
  END;
INT                          cur;

cur := 6;
stat := CL^DELETE( @list^item[ cur ] );
```

CL^INSERT

Adds an item to a circular linked list by adjusting the appropriate linkages. No data is entered into the list.

Syntax:

<stat> := CL^INSERT(<item> , <new^item>)

where:

<stat>, INT,	returns FALSE (0) if the linkages in <item> are invalid; otherwise, returns TRUE (-1).
<item>, INT,	is the address of an item already in the list. The first two words at this address must be the address of the previous entry, followed by the address of the next entry.
<new^item>, INT,	is the address of the item to be added after the above item. This is physically located in an empty slot in the linked list.

example:

```

STRUCT          .list^item[ 0:max ];
  BEGIN
    INT          prev^item;
    INT          nxt^item;
    STRING       item^contents [ 0:contents^max ];
  END;

INT             end^of^list,
               avail;

end^of^list := 75;
avail      := 3;
IF CL^INSERT( @list^item[ end^of^list ],
              @list^item [ avail ] ) THEN
  ... !valid linkage
ELSE
  ... !error condition

```

ACI Worldwide Inc.

Section 5

COBOL Processing

These utilities were written to aid the COBOL programmer in file management when writing COBOL programs.

COBOL^ALTERPRIORITY

Used to change the priority of a process.

Syntax:

<stat> := COBOL^ALTERPRIORITY(<pid>, <priority>)

where:

<stat>, INT,	returns TRUE (non-zero) if an error occurred.
<pid>, INT:ref:4,	is the process identification number. This is passed from a call to COBOL^GETCRTPID.
<priority>, INT,	specifies a new execution priority value in the range of 1:199 for <pid>.

NOTE: The utility library file \$<VOL>ACIUTILS.UTILB must be declared in the SPECIAL-NAMES section of the COBOL program. In the following example the SPECIAL-NAMES section would contain:

SPECIAL-NAMES. FILE \$<VOL>.ACIUTILS.UTILB IS ctalutil.

example:

```
01  STAT      PIC 99.
01  PID       PIC 9999  COMP.
01  PRIORITY  PIC 99    COMP    VALUE 100.
```

```
ENTER TAL "COBOL^ALTERPRIORITY" IN CTALUTIL
      USING PID, PRIORITY
      GIVING STAT.
```

COBOL^CREATE

Creates a file. (See the CREATE Procedure in the Guardian Operating System Programming manual for a complete description of the parameters).

Syntax:

```
<stat> := COBOL^CREATE (<fname>, <pri-extent-size>,  
                           <file-cde>, <secndry-extent-size>,  
                           <file-typ>, <rec-lgth>,  
                           <data-blk-lgth>, <keyseq-param>,  
                           <altkey-param>, <part-param> )
```

where:

<stat>, INT,	returns the file error number if an error occurred. Otherwise, returns FALSE (0).
<fname>, INT:ref:12,	is the name of the disk file to be created. See the Guardian Operating System Programming manual for file formats.
<pri-extent-size>, INT,	if present, is the size of the primary extent in 2048-byte units. Default is 1 (2048 bytes).
<file-cde>, INT,	if present, is an application defined file code. Default is 0.
<secndry-extent-size>, INT,	if present, is the size of the secondary extent. Default is the same as the primary extent size.
<file-typ>, INT,	if present, specifies the type of file to be created. Default is unstructured.
<rec-lgth>, INT,	if present, is the maximum length of the record in bytes. Default is 80.
<data-blk-lgth>, INT,	if present, is the length of each block of records in the file. Default is 1024.
<keyseq-param>, STRING:ref:6,	if present, contains parameters that define this file. Required for key-sequenced files.

<altkey-param>, STRING:ref if present, contains parameters containing any alternate keys for this file.

<part-param>, STRING:ref if present, contains parameters describing this file if the file is a multi-volume file.

NOTE: The utility library file \$<VOL>.ACIUTILS.UTILB must be declared in the SPECIAL-NAMES section of the COBOL program. In the following example the SPECIAL-NAMES section would contain:

SPECIAL-NAMES. FILE \$<VOL>.ACITUILS.UTILB IS CTALUTIL.

example:

```

01 STAT                      PIC 99          COMP.
01 COMMAND-FILE.
    02 FNAME.
        03 VOL                PIC X(8)        VALUE "$DATA1".
        03 SVOL               PIC X(8)        VALUE "PROJ  ".
        03 NAM                 PIC X(8)        VALUE "TEST  ".
    02 PRI-EXT                PIC S9(4)        COMP    VALUE 10.
    02 FILE-CDE               PIC S9(4)        COMP    VALUE 25.
    02 SECNDRY-EXT            PIC S9(4)        COMP    VALUE 6.
    02 FILE-TYP               PIC S9(4)        COMP    VALUE 0.
    02 REC-LGTH               PIC S9(4)        COMP    VALUE 130.
    02 BLK-LGTH               PIC S9(4)        COMP    VALUE 4096.

ENTER TAL "COBOL^CREATE" IN CTALUTIL
    USING FNAME, PRI-EXT, FILE-CDE, SECNDRY-EXT,
        FILE-TYP, REC-LGTH, BLK-LGTH
    GIVING STAT.

```

COBOL^DELAY

Puts the COBOL program into a delay that does not use up CPU time.

Syntax:

CALL COBOL^DELAY(<amt>)

where:

<amt>, INT(32), is the number of hundreths of seconds to
delay. An amount of 100 = 1 second of
delay.

NOTE: The utility library file \$<VOL>.ACIUTILS.UTILB must be declared in the
SPECIAL-NAMES section of the COBOL program. In the following
example the SPECIAL-NAMES section would contain:

SPECIAL-NAMES. FILE \$<VOL>.ACIUTILS.UTILB IS CTALUTIL.

example:

01 AMT PIC 9999 COMP VALUE 300.

ENTER TAL "COBOL^DELAY" IN CTALUTIL
 USING AMT.

COBOL^GETCRTPID

Returns the PID of the calling COBOL program.

Syntax:

<stat> := COBOL^GETCRTPID(<pid>)

where:

<stat>, INT, returns TRUE (non-zero) if an error occurred.

<pid>, INT:ref, is the PID of the calling COBOL program.

NOTE: The utility library file \$<VOL>.ACIUTILS.UTILB must be declared in the SPECIAL-NAMES section of the COBOL program. In the following example the SPECIAL-NAMES section would contain:

SPECIAL-NAMES. FILE \$<VOL>.ACIUTILS.UTILB IS CTALUTIL.

example:

```
01  STAT      PIC 99      COMP.  
01  PID       PIC 99.
```

```
ENTER TAL "COBOL^GETCRTPID" IN CTALUTIL  
USING PID  
GIVING STAT.
```

COBOL^GETFILENUM

Returns the file number and/or an error given the file name as input. File must have been opened once before passing it to this procedure.

Syntax:

<stat> := COBOL^GETFILENUM(<file-ptr>, <fnum>)

where:

<stat>, INT,	returns TRUE (non-zero) if an error occurred.
<file-ptr>, STRING:ref:24,	is the name of the input file.
<fnum>, INT:ref,	contains the file number associated with the input file name.

NOTE: The utility library file \$<VOL>.ACIUTILS.UTILB must be declared in the SPECIAL-NAMES section of the COBOL program. In the following example the SPECIAL-NAMES section would contain:

SPECIAL-NAMES. FILE \$<VOL>.ACIUTILS.UTILB IS CTALUTIL.

example:

```
01 MY-FNAME.
    02 VOL          PIC X(8)    VALUE "$DATA1 ".
    02 SVOL         PIC X(8)    VALUE "PROJ  ".
    02 NAME         PIC X(8)    VALUE "TEST  ".
01 FNUM            PIC 99      COMP.
01 STAT            PIC 99      COMP.

ENTER TAL "COBOL^GETFILENUM" IN CTALUTIL
USING MY-FNAME, FNUM
      GIVING STAT.
```

COBOL^LOOKUPPPD

Checks for a PPD entry associated with the input process name.

Syntax:

<stat> := COBOL^LOOKUPPPD(<input-pro-nam>)

where:

<stat>, INT, returns TRUE (non-zero) if the entry exists.

<input-pro-nam>, STRING:ref:18, contains the input process name.

NOTE: The utility library file \$<VOL>.ACIUTILS.UTILB must be declared in the SPECIAL-NAMES section of the COBOL program. In the following example the SPECIAL-NAMES section would contain:

SPECIAL-NAMES. FILE \$<VOL>.ACIUTILS.UTILB IS CTALUTIL.

example:

```
01 STAT                      PIC 99.  
01 INPUT-PRO-NAM            PIC X(18)  
                             VALUE "\SYSTEM.$PPD1   ".
```

```
ENTER TAL "COBOL^LOOKUPPPD" IN CTALUTIL  
USING INPUT-PROCESS-NAME  
      GIVING STAT.
```

COBOL^PURGE

Used to delete a disk file.

Syntax:

<stat> := COBOL^PURGE(<fname>)

where:

<stat>, INT, returns the error number if one occurred.
Otherwise, FALSE (0) is returned.

<fname>, STRING:ref contains the name of the disk file to be
purged.

NOTE: The utility library file \$<VOL>.ACIUTILS.UTILB must be declared in the
SPECIAL-NAMES section of the COBOL program. In the following
example the SPECIAL-NAMES section would contain:

SPECIAL-NAMES. FILE \$<VOL>.ACIUTILS.UTILB IS CTALUTIL.

example:

```
01 MY-FNAME.
    02 VOL          PIC X(8)    VALUE "$DATA1 ".
    02 SVOL         PIC X(8)    VALUE "PROJ  ".
    02 NAME         PIC X(8)    VALUE "TEST  ".
01 STAT            PIC 99      COMP.
```

ENTER TAL "COBOL^PURGE" IN CTALUTILB USING MY-FNAME
GIVING STAT.

COBOL^RENAME

Used to change the name of a disk file.

Syntax:

<stat> := COBOL^RENAME(<old^fname>, <new^fname>)

where:

<stat>, INT, returns the error number if one occurred.
Otherwise, FALSE (0) is returned.

<old^fname>, STRING:ref:24, contains the name of the disk file to be renamed.

<new^fname>, STRING:ref:24, contains the file name to be assigned to the disk file.

NOTE: The utility library file \$<VOL>.ACIUTILS.UTILB must be declared in the SPECIAL-NAMES section of the COBOL program. In the following example the SPECIAL-NAMES section would contain:

SPECIAL-NAMES. FILE \$<VOL>.ACIUTILS.UTILB IS CTALUTIL.

example:

```

01 OLD-FNAME.
    02 VOL          PIC X(8)    VALUE "$DATA1 ".
    02 SVOL         PIC X(8)    VALUE "PROJ  ".
    02 NAM          PIC X(8)    VALUE "TEST  ".

01 NEW-FNAME.
    02 VOL          PIC X(8)    VALUE "$DATA1 ".
    02 SVOL         PIC X(8)    VALUE "PROJ  ".
    02 NAM          PIC X(8)    VALUE "PRODUCT ".
01 STAT           PIC 99        COMP.

ENTER TAL "COBOL^RENAME" IN CTALUTIL
    USING OLD-FNAME, NEW-FNAME
    GIVING STAT.

```

ACI Worldwide Inc.

Section 6

Date and Time Conversions

The majority of the date and time conversion PROCs detailed on the following pages contain no input validation routines. Consequently, an invalid timearray or timestamp input will yield unpredictable results. These are standard Tandem timestamps and timearrays as described in the TIME and TIMESTAMP procedures in the Guardian Procedure Calls Reference Manual.

ASCII^JULIAN^TIMESTAMP

Converts a date in YYYYMMDD format and a time in HHMMSSMMMMMM format to a Julian timestamp.

Syntax:

<stat> := ASCII^JULIAN^TIMESTAMP(<ascii^dat> , <ascii^tim> , <julian^ts>)

where:

<stat>, INT,	contains TRUE (-1) if the conversion is successful, otherwise FALSE (0).
<ascii^dat>, STRING:ref:8,	is an 8 byte string containing the date in YYYYMMDD format.
<ascii^tim>, STRING:ref:12,	is a 12 byte string containing the time in HHMMSSMMMMMM format.
<julian^ts>, FIXED(0):ref,	is the converted Julian timestamp.

example:

```
FIXED          julian^ts;
STRING .ascii^dat[ 0:7 ],
        .ascii^tim[ 0:11 ];

ascii^dat := "20001015";
ascii^tim := "120000000000";    ! noon
IF NOT ascii^julian^timestamp( ascii^dat, ascii^tim, julian^ts ) THEN
  BEGIN
    ! handle error condition here
  END;
```

ASCII^TIMESTAMP

Converts a string YYMMDDHHMMSSHH to a 48 bit timestamp form.

Syntax:

<stat> := ASCII^TIMESTAMP(<yymmddhhmmsshh> , <lgth> , <ts^array>)

where:

<stat>, INT,	returns TRUE (-1) for normal execution, otherwise FALSE (0) if conversion was not possible.
<yymmddhhmmsshh>, STRING:ref,	is the string containing the time in the form of <yymmddhhmmsshh> and is up to 14 bytes long.
<lgth>, INT:val,	is the number of bytes of <yymmddhhmmsshh> to use when creating the timestamp.
<ts^array>, INT:ref:3,	a 48 bit timestamp as obtained from a call to the NSK 'TIMESTAMP' procedure.

example:

```
INT      ts[ 0:2 ];
STRING   .ascii^ts[ 0:13 ];

ascii^ts := "83062312000000";  ! high noon
IF NOT ascii^timestamp( ascii^ts, 14, ts ) THEN
  BEGIN
    ! handle error condition here
  END;
```

Note: If “YY” is less than 75, it is assumed to be 20YY otherwise it is assumed to be 19YY.

BUMP^DAY

Receives a time array (as set up by the call to "TIME") increments that date by one day and returns a binary value of the new date.

Syntax:

CALL BUMP^DAY(<timarray> , <new^dd^bin> , yyyyymmdd^bin);

where:

<timarray>, INT:ref, contains the current date and time as received from the call to "TIME".

<new^dd^bin>, INT(32):ref, contains the new date in binary form "YYMMDD."

<yyyyymmdd^bin>, INT(32):ref, contains the new date in binary form "YYYYMMDD."

example:

```
INT      .timarray[ 0:6 ];
INT(32)  new^dd^bin,
        yyyyymmdd^bin;
```

```
CALL TIME( timarray );
CALL BUMP^DAY( timarray, new^dd^bin, yyyyymmdd^bin );
```

BUMP^YYMMDD

Adds a specified number of days to an ASCII YYMMDD date string.

Syntax:

CALL BUMP^YYMMDD(<yymmdd>, <num^days>);

where:

<yymmdd>, STRING:ref, contains the original date string for input and
the modified date after the change.

<num^days>, INT, contains the number of days to add to the
date string. It is optional. The default, if this
is omitted, is 1.

example:

```
STRING      .yymmdd[ 0:5 ];  
INT         num^days;  
  
yymmdd  := "991231";  
num^days := 9;  
CALL BUMP^YYMMDD( yymmdd, num^days );
```

Note: If "YY" is less than 75, it is assumed to be 20YY otherwise it is assumed to be 19YY.

CLOCK^TIME

Returns a character string containing the time in the form HH:MM:SS AM or HH:MM:SS PM.

Syntax:

CALL CLOCK^TIME(<ptr>);

where:

<ptr>, STRING:ref:11, is an array which, upon return from the PROC, will contain the time in the form HH:MM:SS AM -or- PM.

example:

```
STRING    .hold^formatted^tim[ 0:10 ];  
CALL CLOCK^TIME( hold^formatted^tim );
```

COMPARE^ASCII^YYMMDD

This utility compares, according to the supplied operator, two 6-byte string dates in YYMMDD format. Prior to comparison, the 6-byte dates are expanded from YYMMDD to YYYYMMDD format, allowing comparisons between dates in the 20th and 21st centuries. If a supplied date's 2-digit year YY falls in the range 00-74, the date is expanded to 20YYMMDD. If a supplied date's 2-digit year YY falls in the range 75- 99, the date is expanded to 19YYMMDD. NOTE: If either supplied date is all spaces, zeroes or binary zeroes, it is expanded to eight bytes of spaces, zeroes or binary zeroes.

Syntax:

<stat> := COMPARE^ASCII^YYMMDD(<yymmdd^1> , <operator> , <yymmdd^2>)

where:

<stat>, INT, returns TRUE if the comparison evaluates to TRUE; returns FALSE if the comparison evaluates to FALSE.

<yymmdd^1>, STRING:ref:6, is the left half of the comparison - a 6-byte ASCII date in YYMMDD format.

<operator>, INT:value, is the relational operator indicating how to compare the halves. Following are the possible values.

Value	Operator	Description
1	<	signed less than
2	>	signed greater than
3	<=	signed less than or equal to
4	>=	signed greater than or equal to
5	'<'	unsigned less than
6	'>'	unsigned greater than
7	'<='	unsigned less than or equal to
8	'>='	unsigned greater than or equal to

<yymmdd^2>, STRING:ref:6, is the right half of the comparison - a 6-byte ASCII date in YYMMDD format.

example:

```
STRING      .yymmdd^1[ 0:5 ],
             .yymmdd^2[ 0:5 ];

yymmdd^1 ':=' "991231";
yymmdd^2 ':=' "000101";
IF COMPARE^ASCII^YYMMDD( yymmdd^1, 3 !<=!,
                        yymmdd^2      ) THEN
    BEGIN
        ! 991231 occurs before (is less than) 000101.
        ! Perform appropriate processing.
    END;
```

COMPARE^BINARY^YYMMDD

This utility compares, according to the supplied operator, two binary dates in YYMMDD format. Prior to comparison, the binary dates are expanded from YYMMDD to YYYYMMDD format, allowing comparisons between dates in the 20th and 21st centuries. If a supplied date's 2-digit year YY falls in the range 00-74, the date is expanded to 20YYMMDD. If a supplied date's 2-digit year YY falls in the range 75-99, the date is expanded to 19YYMMDD. NOTE: If either supplied date is zero, it remains zero for the comparison.

Syntax:

<stat> := COMPARE^BINARY^YYMMDD(<dat^1> , <operator> , <dat^2>)

where:

<stat>, INT, returns TRUE if the comparison evaluates to TRUE; returns FALSE if the comparison evaluates to FALSE.

<dat^1>, INT(32):value, is the left half of the comparison - a binary date in YYMMDD format.

<operator>, INT:value, is the relational operator indicating how to compare the halves. Following are the possible values.

Value	Operator	Description
1	<	signed less than
2	>	signed greater than
3	<=	signed less than or equal to
4	>=	signed greater than or equal to

<dat^2>, INT(32):value, is the right half of the comparison - a binary date in YYMMDD format.

example:

```
INT(32)      dat^1,  
              dat^2;  
  
dat^1 := 991231d;  
dat^2 := 000101d;  
IF COMPARE^BINARY^YYMMDD( dat^1, 3 !<=!, dat^2 ) THEN  
  BEGIN  
    ! 991231 occurs before (is less than) 000101.  
    ! Perform appropriate processing.  
  END;
```

CONTIMESTAMP

Converts the seven word version of time to 48 bit timestamp. The conversion makes it possible to compare the current time to a time and date recorded in timestamp form.

Syntax:

CALL CONTIMESTAMP(<tim^array> , <ts^array>);

where:

<tim^array>, INT:ref:7, is a Tandem standard timearray as obtained from a call to the NSK 'TIME' procedure.

NOTE: The time format is an integer array in which each word from 0-6 contains, respectively, the year, month, day, hour, minute, second and hundredths of second.

<ts^array>, INT:ref:3, is a Tandem standard 48 bit timestamp as obtained from a call to the NSK 'TIMESTAMP' procedure.

NOTE: The 48 bit timestamp is three words containing time as the number of hundredths of seconds since 12:00 AM, January 1, 1975.

example:

```
INT    tim^array[ 0:6 ] :=[ 2000, 10, 15, 12, 0, 0, 0 ],
      tim^stamp[ 0:2 ];
```

```
CALL CONTIMESTAMP( tim^array, tim^stamp );
```

CONVERT^JULIAN^TO^GREG

Constructs the month, day, and year of a date given the true Julian day number of that date. The true Julian day number is defined as the number of days since 1 January 4713 B.C.

Syntax:

CALL CONVERT^JULIAN^TO^GREG(<jdn>, <yyyy>, <mm>, <dd>);

where:

<jdn>, INT(32),	contains the Julian day number.
<yyyy>, INT:ref,	contains the value of the year (0000 - 32767).
<mm>, INT:ref,	contains the value of the month (1 - 12).
<dd>, INT:ref,	contains the value of the day (1-31).

example:

```
INT(32)    jdn;  
INT        yyyy,  
           mm,  
           dd;
```

```
jdn := 2451900d;  
CALL CONVERT^JULIAN^TO^GREG( jdn, yyyy, mm, dd );
```

CONVERT^TIME^TO^YYMMDD

Accepts a timearray and returns year, month and day in binary form.

Syntax:

```
CALL CONVERT^TIME^TO^YYMMDD( <tim^array> , <dbl^yymmdd> ,  
                                <dbl^yyyymmdd> );
```

where:

<tim^array>, INT:ref:7,	is a Tandem standard timearray
<dbl^yymmdd>, INT(32):ref,	is a double word value representing YYMMDD in binary form (i.e. 1015).
<dbl^yyyymmdd>, INT(32):ref,	is a double word value representing YYYYMMDD in binary form (i.e. 20001015).

example:

```
INT          .timarray[ 0:6 ] := [ 2000, 10, 15,  
                                12, 0, 0, 0 ];  
INT (32)     dbl^yymmdd,  
             dbl^yyyymmdd;  
  
CALL CONVERT^TIME^TO^YYMMDD( timarray, dbl^yymmdd,  
                             dbl^yyyymmdd );
```

CONVERT^TIMESTAMP^FORMAT

Will convert between Tandem's 48 bit timestamp and the new fixed Julian timestamp format in Greenwich Mean Time.

Syntax:

<stat> := CONVERT^TIMESTAMP^FORMAT(<ts> , <julian> , <direction>)

where:

<stat>, INT, contains TRUE(-1) if the conversion is successful. Otherwise, FALSE(0).

<ts>, INT, ref, contains the 48 bit timestamp.

<julian>, FIXED, ref, contains the Julian timestamp.

<direction>, INT, contains the direction to perform the conversion.

0 = convert timestamp to julian timestamp.

1 = convert julian timestamp to timestamp

example:

```
INT          .timestamp [0:2],
              direction,
              stat;
FIXED        .julian;

direction    := 1;
stat         := CONVERT^TIMESTAMP^FORMAT( timestamp,
              julian, direction );

direction    := 0;
stat         := CONVERT^TIMESTAMP^FORMAT( timestamp,
              julian, direction );
```

CONVERT^YYMMDD^TO^TIME

Accepts year, month and day in binary form, and returns a Tandem standard timearray.

Syntax:

CALL CONVERT^YYMMDD^TO^TIME(<dbl^yymmdd> , <tim^array>);

where:

<dbl^yymmdd>, INT(32), is a double word value representing
YYMMDD in binary form (i.e. 991231).

<tim^array>, INT:ref:7, is a Tandem standard timearray

example:

```
INT            .timarray[ 0:6 ];  
INT (32)       dblyymmdd;
```

```
CALL CONVERT^YYMMDD^TO^TIME( dblyymmdd, timarray );
```

CURRENT^TIME^AS^STRING

Returns the current time as YYMMDDHHMMSSSS in string form. Each field contains two digits. Will only return the bytes requested.

Syntax:

CALL CURRENT^TIME^AS^STRING(<str> , <lgth>);

where:

<str>, STRING:ref, contains the output string in the form
YYMMDDHHMMSSSS for the length of 14
digits.

<lgth>, INT, contains the maximum length to be returned.

example:

```
STRING    .str[ 0:5 ];  
INT       lgth;  
  
lgth := 6;  
CALL CURRENT^TIME^AS^STRING( str, lgth );
```

If the current date and time is "2000101512000000" then, 'str' will contain "001015" after the call.

DAY^OF^WEEK

Returns the day of the week given a Julian day number. The value returned is a number from 1 to 7. 1 is for Sunday...7 is for Saturday.

Syntax:

<dd> := DAY^OF^WEEK(<julian^dd^num>)

where:

<dd>, INT, contains the result of the conversion.

1=Sunday

2=Monday

3=Tuesday

4=Wednesday

5=Thursday

6=Friday

7=Saturday

<julian^dd^num>, INT(32):ref, contains the INT(32) true Julian day number.

example:

INT dd;

INT(32) julian^dd^num;

julian^dd^num := 2451900d;

dd := DAY^OF^WEEK(julian^dd^num);

GET^DAY^OF^WEEK

Accepts a timearray and returns the day of the week for that date. Returns 1 for Monday, 2 for Tuesday, etc.

Syntax:

CALL GET^DAY^OF^WEEK(<tim^array> , <dd>);

where:

<tim^array>, INT:ref:7, is a Tandem standard timearray

<dd>, INT:ref, contains a numeric value representing the day of the week:

- 1 = Monday
- 2 = Tuesday
- 3 = Wednesday
- 4 = Thursday
- 5 = Friday
- 6 = Saturday
- 7 = Sunday

example:

```
INT                      .timarray[ 0:6 ],  
                         dd;
```

```
CALL TIME( timarray );  
CALL GET^DAY^OF^WEEK( timarray, dd );
```

GET^NEXT^DAY

Receives a time array as set up by a call to "TIME" and returns the same time array modified to contain the next day.

Syntax:

CALL GET^NEXT^DAY(<timarray>);

where:

<timarray>, INT, ref:7,	contains the time as received from the NSK "TIME" call and the results of this proc after the conversion.
--------------------------------------	--

example:

```
INT    .timarray[ 0:6 ];  
  
CALL TIME( timarray );  
CALL GET^NEXT^DAY( timarray ) ;
```

JULIAN

Returns the true Julian day number of the date represented by the input parameter. The true Julian day number is defined as the number of days since 1 January 4713 B.C.

Syntax:

<julian^dd^num> := JULIAN(<yyyy>, <mm>, <dd>)

where:

<julian^dd^num>, INT(32), returns the Julian day number for the input date.

<yyyy>, INT, contains the value of the year (0000 - 32767).

<mm>, INT, contains the value of the month (1 - 12).

<dd>, INT, contains the value of the day (1 - 31).

example:

```
INT      yyyy,
          mm,
          dd;
INT(32)  julian^dd^num;

yyyy    := 2000;
mm      := 10;
dd      := 15;
julian^dd^num := JULIAN( yyyy, mm, dd );
```

JULIAN^DATE

Returns a 6 byte field containing the julian date at the time this proc is called in the form YYYYJJJ.

Syntax:

<stat> := JULIAN^DATE(<jul^dat>)

where:

<stat>, INT,	returns TRUE (-1) if the conversion is successful. Otherwise, FALSE (0) if the conversion from integer to ASCII should fail.
--------------	--

<jul^dat>, STRING:ref,	contains the string to be converted and the result of the conversion.
------------------------	---

example:

```
STRING    jul^dat[ 0:5 ];  
INT       stat;  
  
stat := JULIAN^DATE( jul^dat );
```

JULIAN^TIMESTAMP^ASCII

Converts a Julian timestamp to a date in YYYYMMDD format and a time in HHMMSSMMMMMM format.

Syntax:

<stat> := JULIAN^TIMESTAMP^ASCII(<ascii^dat> , <ascii^tim> , <julian^ts>)

where:

<stat>, INT,	contains TRUE (-1) if the conversion is successful, otherwise FALSE (0).
<ascii^dat>, STRING:ref:8,	is an 8 byte string containing the date in YYYYMMDD format.
<ascii^tim>, STRING:ref:12,	is a 12 byte string containing the time in HHMMSSMMMMMM format.
<julian^ts>, FIXED(0):value,	is the Julian timestamp to convert.

example:

```
FIXED    julian^ts;  
STRING   .ascii^dat[ 0:7 ],  
          .ascii^tim[ 0:11 ];
```

```
julian^ts := juliantimestamp; !get the current date/time  
IF NOT JULIAN^TIMESTAMP^ASCII( ascii^dat, ascii^tim,  
                                julian^ts ) THEN  
    BEGIN  
        ! handle error condition here  
    END;
```


LEAP^YEAR

Returns TRUE if the year supplied is a leap-year. Defaults to the current year if none is supplied. If the year supplied is 2 digits (less than 100) the following assumptions are made: If the 2 digit year is in the range 00-74, assume the century is 2000. If the 2 digit year is in the range 75-99, assume the century is 1900.

Syntax:

<stat> := LEAP^YEAR(<yy>)

where:

<stat>, INT, is TRUE (non-zero) if the year supplied is a leap year.

<yy>, INT, If not entered, the current year is used.
 <yy> is expected to be a four-digit value, such as 1999. However, if <yy> contains only two digits, the following assumptions are made: If <yy> is in the range 00-74, century 2000 is assumed. If <yy> is in the range 75-99, century 1900 is assumed.

example:

```
INT      yy;

yy      := 1999
stat    := LEAP^YEAR( yy );
yy      := 99;
stat    := LEAP^YEAR( yy );
stat    := LEAP^YEAR;
```

LONG^DATE^AND^TIME

Returns the current date and time as a readable character string in the following format:

" Sunday, October 15, 2000 - 12:00:00 PM "

The length of the string is returned in a second parameter. The maximum string length is 44 characters.

Syntax:

CALL LONG^DATE^AND^TIME(<str> , <str^lgth>);

where:

<str>, STRING:ref:44,	Upon return from the PROC it will contain the current date and time as a readable character string.
-----------------------	---

<str^lgth>, INT:ref,	is returned as the length of the character string in <str>.
----------------------	---

example:

```
STRING    .dat^char^str[ 0:43 ];  
INT       str^lgth;  
  
CALL LONG^DATE^AND^TIME( dat^char^str,  
                        str^lgth );
```

TEST^HOLIDAY

Returns TRUE in a supplied parameter if the given date is one of the following:

New Year's day	(January 1)
Independence day	(July 4)
California Admission day	(September 9)
Christmas	(December 25)
Washington's Birthday	(3rd Monday in February)
Memorial day	(Last Monday in May)
Labor day	(First Monday in September)
Thanksgiving	(4th Thursday in November)

Syntax:

CALL TEST^HOLIDAY(<tim^array> , <hol>);

where:

<tim^array>, INT:ref:7, is a Tandem standard timearray.

<hol>, INT:ref is used to hold the return code from the PROC. This parameter will contain a TRUE (non-zero) value if the date supplied is a holiday. It will contain FALSE (0) if the date is not a holiday.

example:

```
INT                    .timarray[ 0:6 ],
                      hol;

CALL TIME( timarray );
CALL TEST^HOLIDAY( timarray, hol );
```

TEST^WEEKEND

Accepts a timearray and returns TRUE in a supplied parameter if the day of the week of the array is Saturday or Sunday.

Syntax:

CALL TEST^WEEKEND(<tim^array> , <wknd>);

where:

<tim^array>, INT:ref:7, is a Tandem standard timearray.

<wknd>, INT:ref, is used to hold the return code from the PROC. This parameter will contain a TRUE (non-zero) value if the date supplied falls on a weekend. It will contain FALSE (0) if the date does not fall on a Saturday or Sunday.

example:

```
INT                      .timarray[ 0:6 ],  
                         wknd;
```

```
CALL TIME( timarray );  
CALL TEST^WEEKEND( timarray, wknd );
```

TIMESTAMP^ADJUST^NUM^MINUTES

Adds number of minutes to a given array.

Syntax:

CALL TIMESTAMP^ADJUST^NUM^MINUTES(<ts^array^1> ,
<num^minutes>, <ts^array^2>);

where:

<ts^array^1>, INT, ref:3, contains the internal form of the CPU clock
interval as received from NSK
'TIMESTAMP'.

<num^minutes>, INT, contains the number of minutes to add.

<ts^array^2>, INT, ref:3, contains the result of adding num^minutes
to the first array.

example:

```
INT    .ts^array^1[ 0:2 ],  
       .ts^array^2[ 0:2 ],  
       num^minutes;
```

```
num^minutes := 5;  
CALL TIMESTAMP( ts^array^1 );  
CALL TIMESTAMP^ADJUST^NUM^MINUTES( ts^array^1,  
                                     num^minutes,  
                                     ts^array^2 );
```

TIMESTAMP^ADJUST^NUM^MINUTES^DBL

Adds number of minutes to a given array. Similar to TIMESTAMP^ADJUST^NUM^MINUTES except that it accepts an INT(32) value for the “number of minutes” parameter.

Syntax:

CALL TIMESTAMP^ADJUST^NUM^MINUTES^DBL(<ts^array^1> ,
<num^minutes>, <ts^array^2>);

where:

<ts^array^1>, INT, .EXT:ref:3, contains the internal form of the CPU clock interval as received from NSK 'TIMESTAMP'.

<num^minutes>, INT(32), contains the number of minutes to add.

<ts^array^2>, INT, .EXT:ref:3, contains the result of adding num^minutes to the first array.

example:

```
INT      .ts^array^1[ 0:2 ],
          .ts^array^2[ 0:2 ];
INT(32)  num^minutes;

num^minutes := 70000D;
CALL TIMESTAMP( ts^array^1 );
CALL TIMESTAMP^ADJUST^NUM^MINUTES( ts^array^1,
                                     num^minutes,
                                     ts^array^2 );
```

TIMESTAMP^ASCII

Converts a timestamp array to the ASCII representation.

Syntax:

CALL TIMESTAMP^ASCII(<yymmddhhmmsshh> , <lgth> , <ts^array>);

where:

<yymmddhhmmsshh>, STRING:ref,	is the string which is to contain the ASCII representation of the timestamp.
<lgth>, INT:val,	is the byte length to use when filling <yymmddhhmmsshh> with the ASCII string representation of <ts^array>. Bits 0 through 14 of <lgth> are used to assure an even byte length, and the proc limits length to a maximum of 14 bytes.
<ts^array>, INT:ref:3,	is a Tandem standard 48 bit timestamp as obtained from a call to the NSK 'TIMESTAMP' procedure.

example:

```
INT      .ts[ 0:2 ];
STRING   .ascii^ts[ 0:11 ]; ! omit hundredth
                        ! of seconds
```

```
CALL TIMESTAMP( ts );
CALL TIMESTAMP^ASCII( ascii^ts, 12, ts );
```

TIME^ASCII

Returns the ASCII representation of either or both of date and time in the form YYMMDDHHMMSSHH or just the date in the form MMDDYY. The date/time to be converted to ASCII may be provided in the call to the proc as either a 7 word integer time array or a 3 word timestamp. If neither argument is passed, the time is acquired via a call to the NSK 'TIME' procedure. If both arguments are passed, only the 7 word time array is used.

Syntax:

```
CALL TIME^ASCII( <yymmddhhmmsshh> , <mmddyy> , <lgth>  
                  , <user^tim^array> , <ts> );
```

where:

<yymmddhhmmsshh>, STRING:ref, is the string which is to contain the ASCII representation of the date and time. The byte length of this string is passed in the <lgth> parameter; it must be even and is limited to a maximum of 14.

<mmddyy>, STRING:ref, is the string which is to contain the ASCII time in the format month, day, year. The user must ensure that the array passed is at least six bytes long.

NOTE: AT LEAST ONE of the preceding parameters MUST be passed; both MAY be passed, if desired.

<lgth>, INT:val, if specified, is the byte length of the output string <yymmddhhmmsshh>. If this parameter is not passed, the proc defaults to a length of 14 bytes; if <lgth> exceeds 14, only 14 bytes will be moved to the output string.

<user^tim^array>, INT:ref:7, if specified, is the seven word time array which is returned by a call to TIME or CONTIME.

<ts>, INT:ref:3,

if specified, is the 48 bit timestamp array which is returned by a call to 'TIMESTAMP'.

example:

```
INT      .tim^array[ 0:6 ] :=[ 2000, 10, 15, 0, 0, 0, 0 ];
STRING   .mm^dd^yy[ 0:5 ],
          .yymmddhhmmss[ 0:11 ],
          .ts[ 0:2 ];
```

```
CALL TIME^ASCII( , mm^dd^yy, , tim^array );
```

```
! CURRENT TIME
```

```
CALL TIME^ASCII( yymmddhhmmss, , 12 );
```

```
CALL TIMESTAMP( ts );
```

```
CALL TIME^ASCII( yymmddhhmmss, ,12, , ts );
```

TIME^TIMESTAMP

Converts a timearray into a Tandem 48 bit Timestamp form. This is the opposite of Tandem's NSK proc CONTIME.

Syntax:

CALL TIME^TIMESTAMP(<timarray>, <ts>);

where:

<timarray>, INT:ref:7, is a standard Tandem timearray.

<ts>, INT:ref:3, is an array to contain the <ts> in Tandem 48 bit timestamp form.

example:

```
INT    .timarray[ 0:6 ],  
        .ts[ 0:2 ];
```

```
CALL TIME( timarray );  
CALL TIME^TIMESTAMP( timarray, ts );
```

TIME^ZONE^CHANGE

Will add or subtract a number of minutes to compensate for a different time zone. Minutes are passed as INT, either positive or negative. This proc converts the string data into a timestamp and calls 'TIMESTAMP^ADJUST^NUM^MINUTES'. The timestamp returned by this proc is reconverted into ASCII. The ASCII string is picked apart to get the year, month, day, hour, and minutes as separate strings. The calling proc can then rearrange the data as needed. Midnight is 00 hours, therefore hh^s is 00 and mm^s is 00. One minute past midnight is 00 for hh^s and 01 for mm^s.

Syntax:

```
<stat> := TIME^ZONE^CHANGE( <yymmdd^s> , <hhmm^s> ,  

<minutes> , <yy^s> ,  

<mo^s> , <dd^s> , <hh^s> ,  

<mm^s> , <err> )
```

where:

<stat>, INT,	returns TRUE (-1) for normal execution. Otherwise, FALSE (0), if the data in <yymmdd^s> or <hhmm^s> is not numeric.
<yymmdd^s>, STRING:ref,	contains the data string in the form of YYMMDD.
<hhmm^s>, STRING:ref,	contains the time string in the form of HHMM.
<minutes>, INT,	contains the number of minutes to add or subtract.
<yy^s>, STRING:ref,	contains the output string value for the year.
<mo^s>, STRING:ref,	contains the output string value for the month.
<dd^s>, STRING:ref,	contains the output string value for the day.
<hh^s>, STRING:ref,	contains the output string value for the hours.

<mm^s>, STRING:ref,

contains the output string value for the minutes.

<err>, INT:ref,

if passed, will contain the value of any errors that occur. If any of the parameters are not passed, error will contain the value 3. If any other error occurs, error will contain a value of 1. If no error, then error will be zero.

example:

```
STRING      yymmdd^s[ 0:5 ],
             hhmm^s[ 0:3 ],
             yy^s[ 0:1 ],
             mo^s[ 0:1 ],
             dd^s[ 0:1 ],
             hh^s[ 0:1 ],
             mm^s[ 0:1 ];
INT          minutes,
             err,
             stat;
```

```
yymmdd^s    ':=' "991231";
hhmm^s      ':=' "1430";
minutes     := 60;
stat := TIME^ZONE^CHANGE( yymmdd^s, hhmm^s, minutes,
                          yy^s, mo^s, dd^s, hh^s,
                          mm^s, err );
```

TIME^ZONE^CHANGEX (extended address version)

Will add or subtract a number of minutes to compensate for a different time zone. Minutes are passed as INT, either positive or negative. This proc converts the string data into a timestamp and calls 'TIMESTAMP^ADJUST^NUM^MINUTES'. The timestamp returned by this proc is reconverted into ASCII. The ASCII string is picked apart to get the year, month, day, hour, and minutes as separate strings. The calling proc can then rearrange the data as needed. Midnight is 00 hours, therefore hh^s is 00 and mm^s is 00. One minute past midnight is 00 for hh^s and 01 for mm^s.

Syntax:

```
<stat> := TIME^ZONE^CHANGEX( <yymmdd^s> , <hhmm^s> ,  
                                <minutes> , <yy^s> ,  
                                <mo^s> , <dd^s> , <hh^s> ,  
                                <mm^s> , <err> )
```

where:

<stat>, INT,	returns TRUE (-1) for normal execution. Otherwise, FALSE (0), if the data in <yymmdd^s> or <hhmm^s> is not numeric.
<yymmdd^s>, STRING:ref,	contains the data string in the form of YYMMDD.
<hhmm^s>, STRING:ref,	contains the time string in the form of HHMM.
<minutes>, INT,	contains the number of minutes to add or subtract.
<yy^s>, STRING:ref,	contains the output string value for the year.
<mo^s>, STRING:ref,	contains the output string value for the month.
<dd^s>, STRING:ref,	contains the output string value for the day.
<hh^s>, STRING:ref,	contains the output string value for the hours.
<mm^s>, STRING:ref,	contains the output string value for the minutes.

<err>, INT:ref,

if passed, will contain the value of any errors that occur. If any of the parameters are not passed, error will contain the value 3. If any other error occurs, error will contain a value of 1. If no error, then error will be zero.

example:

```

STRING      .ext  yymmdd^s,[0:5],
              .ext  hhmm^s,[0:3],
              .ext  yy^s, [0:1],
              .ext  mo^s, [0:1],
              .ext  dd^s, [0:1],
              .ext  hh^s, [0:1],
              .ext  mm^s; [0:1];
INT          .ext  err,
              minutes,
              stat;

```

```

yymmdd^s    ':=' "991231";
hhmm^s      ':=' "1430";
minutes     := 60;
stat := TIME^ZONE^CHANGEX( yymmdd^s, hhmm^s,
                           minutes, yy^s, mo^s, dd^s,
                           hh^s, mm^s, err );

```

TODAYS^DATE

Returns today's date in one or both of the following forms:

- 1) 10/15/2000
- 2) October 15, 2000

A value may be passed to the PROC to indicate the length of the string to return.

Syntax:

CALL TODAYS^DATE(<short^form>, <long^form>, <lgth>);

where:

<short^form>, STRING:ref:8, is an array which, upon return from the PROC, will contain the short form of today's date, MM/DD/YY.

<long^form>, STRING:ref:18, is an array which, upon return from the PROC, will contain the long form of today's date, (e.g., October 15, 2000). The string should be long enough to hold the longest value which is 18.

<lgth>, INT:ref, is the length of <long^form> in bytes.

example:

```
INT      lgth
STRING   .short^form[ 0:7 ],
          .long^form[ 0:17 ];
```

```
CALL TODAYS^DATE( short^form, long^form, lgth );
```

VALID^DAT^TIM^STRING

Checks for a valid date and time string. Returns false for any invalid date or time component or non-string data.

Syntax:

[illegible]

where:

<stat>, INT,	contains TRUE (-1) if the input date and time string was valid. Otherwise FALSE (0).
--------------	--

<dat^tim>, STRING, ref,	contains the input date and time string in the format YYYYMMDDHHMMSSMMMMMM or YYMMDDHHMMSSMMMMMM.
-------------------------	---

<date^time^length>, INT, value,

contains the length of the date and time string. The length must be in the range 2-18 for date and time strings with 2-digit years (YY). The length must be in the range 4-20 for date and time strings with 4-digit years (YYYY). The length must be an even number when validating a date and time string down to the seconds (SS) component. When validating a date and time string down to a fraction of a second (.1 second through .000001 second), the length may be odd.

<four^digit^year>, INT, value, optional, indicates whether the date and time string to be validated contains a 4-digit year. Valid values are TRUE (-1), indicating a 4-digit year, or FALSE (0), indicating a 2-digit year. The default value is FALSE.

<error>, INT, ref, optional, if present, contains a number indicating which part of the date and time string was in error:

- 0 no error
- 1 YY was bad (first two digit os a 4 digit year)
- 2 YY was bad (two digit year or last two digits of a 4 digit year)
- 3 MM was bad (month digits)
- 4 DD was bad
- 5 HH was bad
- 6 MM was bad (minute digits)
- 7 SS was bad
- 8 1 tenth of a second was bad
- 9 1 hundredth of a second was bad
- 10 1 thousandth of a second was bad
- 11 10 thousandth of a second was bad
- 12 100 thousandth of a second was bad
- 13 1 millionth of a second was bad
- 1 if missing a required parameter or if a parameter was invalid

example:

```
LITERAL    true = -1,
            false = 0;
STRING     .dat^tim[ 0:19 ];
INT        error,
            stat;
```

```
dat^tim ':=' "20001015123000000001";
stat      := VALID^DAT^TIM^STRING( dat^tim, 20, true, error );
```

VALID^DATE^YYMMDD

Checks for a valid date. Returns false for any non-valid date or nonstring data.

Syntax:

<stat> := VALID^DATE^YYMMDD(<yymmdd>, <err>)

where:

<stat>, INT,	contains TRUE (-1) if the input date was valid. Otherwise FALSE(0).
<yymmdd>, STRING, ref,	contains the string input acceptable in the form "yymmdd."
<err>, INT, ref, optional,	if present, contains a number indicating which part of the date was in error: 0 no error 1 YY was bad 2 MM was bad 3 DD was bad 4 if no YYMMDD parameter

example:

```
STRING .yymmdd[ 0:5 ];  
INT    err,  
       stat;
```

```
yymmdd := "001015";  
stat   := VALID^DATE^YYMMDD( yymmdd, err );
```

Note: If "YY" is less than 75 it is assumed to be 20YY otherwise it is assumed to be 19YY.

VALID^TIM^STRING

Checks for a valid time string. Returns false for any invalid time component or non-string data.

Syntax:

<stat> := VALID^TIM^STRING(<tim> , <tim^lgth> , <error>)

where:

<stat>, INT,	contains TRUE (-1) if the input time string was valid. Otherwise FALSE (0).
<tim>, STRING, ref,	contains the input time string in the format HHMMSSMMMMMM.
<tim^lgth>, INT, value,	contains the length of the time string. The length must be in the range 2-12. The length must be an even number when validating a time string down to the seconds (SS) component. When validating a time string down to a fraction of a second (.1 second through .000001 second), the length may be odd.
<error>, INT, ref, optional,	if present, contains a number indicating which part of the time string was in error:
0	no error
1	HH was bad
2	MM was bad
3	SS was bad
4	1 tenth of a second was bad
5	1 hundredth of a second was bad
6	1 thousandth of a second was bad
7	10 thousandth of a second was bad
8	100 thousandth of a second was bad
9	1 millionth of a second was bad
-1	if missing a required parameter or if a parameter was invalid

example:

```
STRING      .tim[ 0:11 ];  
INT         error,  
           stat;  
tim := "123000000001";  
stat := VALID^TIM^STRING( tim, 12, error );
```

YYMMDD^TO^JULIAN

Converts a date in YYMMDD format to a true Julian day. The true Julian day number is defined as the number of days since 1 January 4713 B.C.

Syntax:

julian^dd := YYMMDD^TO^JULIAN(<yymmdd>)

where:

<julian^dd>, INT(32), is the number of elapsed days since January 1, 4713 B.C. represented by <yymmdd>.

<yymmdd>, STRING:ref, is the string array of date to be converted to a julian day.

example:

```
STRING      dat[ 0:5 ];  
INT(32)     julian^dd;  
  
dat         ':= "001015";  
julian^dd   := yymmdd^to^julian( dat );
```

Note: If “YY” is less than 75 it is assumed to be 20YY otherwise it is assumed to be 19YY.

YY^DDD^TO^TIME

Converts a 2 digit year and a 3 digit day count from Jan. 1 of the year to a timearray.

Syntax:

CALL YY^DDD^TO^TIME(<yy> , <ddd> , <tim^array>);

where:

<yy>, INT:ref, is an integer value containing the year in 2 or 4 digits in the form: "YY" (i.e., "00" or "2000")

<ddd>, INT:ref, is an integer value containing the date in 3 digits between 1 and 366.

<tim^array>, INT:ref:7, is a Tandem standard time array.

example:

```
INT    yy,  
        ddd;  
INT    .timarray[ 0:6 ];  
  
yy      := 1990;  
ddd     := 86;  
CALL YY^DDD^TO^TIME( yy, ddd, timarray );
```

Note: If "YY" is less than 75 it is assumed to be 20YY otherwise it is assumed to be 19YY.

Section 7

Debugging Procs

These utilities are additional debugging tools that can be used along with Tandem's debugging facility DEBUG. Information on how to use DEBUG can be found in Tandem's "DEBUG Reference Manual".

HEX^DUMP

Converts to hexadecimal representation any block of data storage beginning with an address specified as a variable name. The number of an open file is passed for the hexadecimal data to be written to.

Syntax:

CALL HEX^DUMP(<dump^file>, <tag> , <where> , <lgth>)

where:

<dump^file>, INT,	contains the number of an open file to which the hexadecimal data is written.
<tag>, INT(32),	if present, is written to the <dump^file> file to identify the source of the dump (in case the data is being dumped from several places).
<where>, STRING:ref,	points to the buffer containing the dumped data.
<lgth>, INT,	is the length of the dumped data buffer.

example:

```
INT      dump^file,
          buf^lgth;
INT(32)  src^tag;
STRING   .data^buf[ 0:1999 ];

buf^lgth := 25;
src^tag  := 909d;
CALL HEX^DUMP( dump^file, src^tag, data^buf, buf^lgth );
```

PEP^OFFSET

Accepts an input address and returns in a character string the procedure name that contains that input address. If present, the object file name entered in <obj^nam> is searched; otherwise, it searches the calling program. If an error occurs during the search, the file error number is returned; otherwise, it returns FALSE (0).

Syntax:

<stat> := PEP^OFFSET(<procedure^addr>, <str>, <lgth>, <obj^nam>)

where:

<stat>, INT,	returns the file error number if an error occurs; otherwise, it returns FALSE (0);
<procedure^addr>, INT,	an address in some procedure.
<str>, STRING:ref:86	contains the procedure name of the input address, if found. It is always formatted as shown in this example:
<pre>.....+.....1.....+.....2.....+.....3.....+.....4.....+.....5.....+ 'ADDR %012345 = INIT^MSG + %000236 for \$SYSTEM.AN.OBJECT'</pre>	
	If an error occurred or the procedure is not found, <str> will contain an error message.
<lgth>, INT:ref,	contains the length of the character string being returned.
<obj^nam>, INT:ref:12,	if present, contains the name of the object file to be searched.

example:

```
INT      procedure^addr,  
         .obj^nam[ 0:11 ] := ["$system an    object "],  
         lgth,  
         file^err;  
STRING  .return^str[ 0:85 ];  
  
procedure^addr := %012345;  
file^err      := PEP^OFFSET( procedure^addr, return^str, lgth,  
                             obj^nam );
```

TRACER

May be used to dump to an appropriate dump file global or local data in formats to be determined by the user. All formats include an ASCII dump printed to the right of the hexadecimal or octal dump (see examples on the following page). The volume of data to be written in any given call to the proc is limited only by NSK.

Syntax:

CALL TRACER(<dump^file>, <label> , <where> , <lgth> , <typ>, <xlate>)

where:

<dump^file>, INT,	contains the number of an open file to which the data is written.
<label>, INT(32),	if present, is written to the <dump^file> file to identify the source of the dump (the four bytes of text passed are printed as a label).
<where>, STRING:ref,	points to the buffer containing the dumped data.
<lgth>, INT,	is the number of bytes of data to be dumped, even if dump-by-word is chosen.
<typ>, INT,	if present, is used to determine the dump format. 0 = Hexadecimal by full word (default) 1 = Octal by full word; if an odd byte length is passed, an extra byte is displayed in the octal dump 2 = Octal by byte
<xlate>, INT,	if present, the data are assumed to be EBCDIC, so EBCDIC to ASCII translation is applied to the text buffer; the value associated with this parameter is irrelevant as only its presence is checked by the procedure.

NOTE: If the <dump^file> file is not open at the time of the call, TRACER does an immediate return.

example:

```
INT      dump^file^num,  
         buf^lgth;  
INT(32)  src^tag;  
STRING   .data^buf[ 0:1999 ];  
  
buf^lgth := 25;  
src^tag  := "LABL";  
CALL TRACER( dump^file^num, src^tag, data^buf, buf^lgth );  
          ! DEFAULTS TO HEXADECIMAL
```

TRACER printout example:

```
TST0  01 SEP 1983  14:15:51.32
00036 * 00 01    02 03    04 05 ... 0E 0F * * ..... * 000000
00056 * 10 11    12 13    14 15 ... 1E 1F * * ..... * 000020
00076 * 20 21    22 23    24 25 ... 2E 2F * * !"#$%&'()*+,-./ * 000040
00116 * 30 31    32 33    34 35 ... 3E 3F * * 0123456789:;<=>? * 000060
00136 * 40 41    42 43    44 45 ... 4E 4F * * @ABCDEFGHIJKLMNO * 000100
00156 * 50 51    52 53    54 55 ... 5E 5F * * PQRSTUVWXYZ[\]^_ * 000120
00176 * 60 61    62 63    64 65 ... 6E 6F * * @abcdefghijklmnopqrstuvwxyz * 000140
00216 * 70 51    72 53    74 75 ... 7E   * * pqrstuvwxyz[\]^ * 000160

TST1  01 SEP 1983  14:15:52.55
00036 * 000001  001003  002005...007017 * * ..... * 000000
00056 * 010021  011023  012025...017037 * * ..... * 000020
00076 * 020041  021043  022045...027057 * * !"#$%&'()*+,-./ * 000040
00116 * 030061  031063  032065...037077 * * 0123456789:;<=>? * 000060
00136 * 040101  041103  042105...047117 * * @ABCDEFGHIJKLMNO * 000100
00156 * 050121  051123  052125...057137 * * PQRSTUVWXYZ[\]^_ * 000120
00176 * 060141  061143  062145...067157 * * @abcdefghijklmnopqrstuvwxyz * 000140
00216 * 070161  071163  072165...077177 * * pqrstuvwxyz[\]^ * 000160

TST2  01 SEP 1983  14:15:53.12
44416 * 120 061 101 136 111 124 123 ... * * P1A^ITS * 000000
44436 * 040 040 000 000 000 001 070 ... * * ....830831S1AI * 000020
44456 * 124 123 040 040 040 040 040 ... * * TS S1AI * 000040
44476 * 124 123 040 040 040 040 040 ... * * TS .... * 000060
44516 * 000 010 000 037 000 013 000 ... * * .....).<..... * 000100
44536 * 000 000 000 000 000 000 000 ... * * ..... * 000120
44556 * 000 000 000 000 000 000 000 ... * * ..... * 000140
44576 * 111 124 123 040 120 061 101 ... * * ITS P1A^ITS * 000160
```

The following example shows the effect of applying ebcDic^to^ascii translation:

```
EBCD  01 SEP 1983  11:18:09.21
00156 * E4 D7    D7 C5    D9 40 ... E3 40 * * UPPER CASE TEXT * 000000
00176 * 93 96    A6 85    99 40 ... A4 40 * * lower case text * 000020
```

TRACK

Enables a programmer to follow program flow as a procedure executes and to view the contents of one or two variables at the same time. It also allows a program to abend or go into DEBUG following its execution.

Syntax:

CALL TRACK(<dump^file>, <tag> , <var1> , <var2> , <base> , <disp>)

where:

<dump^file>, INT,	contains the number of an open file to which the hexadecimal data is written.
<tag>, INT(32),	if present, is an alphabetic tag that distinguishes this call to TRACK from others. If omitted, the PROC uses "....".
<var1> & <var2>, INT,	if present, displays the values of string or integer variables at the time of the call to TRACK.
<base>, INT,	if present, indicates the base that you want to display <var1> and/or <var2>. Valid values are 2 thru 10, and 16. If absent, the PROC defaults to base 10.
<disp>, INT,	if present, indicates the action TRACK will take upon termination. If absent, execution continues with the statement following the call to TRACK. 0 - execution continues with the statement following the call to TRACK 1 - TRACK calls DEBUG Any other value - TRACK calls ABEND

example:

```
INT      dump^file,  
         dspy^var1,  
         dspy^var2,  
         base,  
         call^debug;  
INT(32)  tag := [ "xfl" ];  
  
base     := 16;  
call^debug := 1;  
CALL TRACK( dump^file, tag, dspy^var1, dspy^var2, base,  
            call^debug );
```

ACI Worldwide Inc.

Section 8

Encryption/Decryption Procs

These PROCs are based on the federal Data Encryption Standard (DES). DES specifies an algorithm to be used for the cryptographic protection of computer data. Encrypting data converts it to an unintelligible form called cipher. Decrypting cipher converts the data back to its original form. The algorithm specifies both enciphering and deciphering operations which are based on a binary number called a key. The key consists of 64 bits of which 56 bits are used directly by the algorithm and 8 bits are used for error detection. For a more detailed description of this algorithm, you may refer to the Federal Information Data Encryption Standard.

DECODE

Provides a software implementation of the DES. DECODE accepts text and a key, and returns decoded text.

Syntax:

CALL DECODE(<txt> , <key> , <wrk>);

where:

<txt>, INT:ref:4,	contains the text to be decoded upon entry to the PROC. Upon return from the PROC, it contains the decoded text.
<key>, INT:ref:4,	is the key to be used to decode the text.
<wrk>, INT:ref:52,	is a work area used to avoid recalculating the subkeys for each 64 bits of input.

example:

```
INT    .txt[ 0:3 ],
        .key[ 0:3 ],
        .wrk[ 0:51 ];

TXT    '[:]'[%hd5d4,%h4ff7,%h2068,%h3d0d];
KEY    '[:]'[%H0123,%H4567,%H89AB,%HCDEF];

CALL DECODE( txt, key, wrk );
```

The DES encryption routines build an intermediate transposition table in the allocated "work" space. The table is created from the key by extensive bit transposition and substitution routines. Each time ENCODE or DECODE is called, the procs test to determine if the "work" table has been created. If the table isn't present, one is created. The table building step takes the majority of the CPU time required for this routine. For the sake of efficiency, the "work" area avoids the need to rebuild the tables when using the same key over. Rebuilding the table should be avoided if at all possible.

Therefore, you should try to associate the "work" area with the key so that the first time the key is used, the table is built. Each subsequent time the key is used, the same table should be used. Notice that, in general, this requires that work areas need to be in global space or within a storage area associated with a record. An example would be to include the work space within a TDF record. If the TDF was usually maintained in extended memory, extra space could be allocated in extended memory associated with the record. Similarly, since authorization by IBM DES verification requires a DES encryption step, the work space allocated should be associated with the IDF that has the PIN verification DES key.

DECODEX (extended address version)

Provides a software implementation of the DES. DECODEX accepts text and a key, and returns decoded text.

Syntax:

CALL DECODEX(<txtx> , <keyx> , <wrkx>);

where:

<txtx>, INT .EXT:ref:4, contains the text to be decoded upon entry to the PROC. Upon return from the PROC, it contains the decoded text.

<keyx>, INT .EXT:ref:4, is the key to be used to decode the text.

<wrkx>, INT .EXT:ref:52, is a work area used to avoid recalculating the subkeys for each 64 bits of input.

example:

```
INT    .ext txtx[ 0:3 ],
        .ext keyx[ 0:3 ],
        .ext wrkx[ 0:51 ];

txt     ':[%hd5d4,%h4ff7,%h2068,%h3d0d];
key     ':[%H0123,%H4567,%H89AB,%HCDEF];
CALL DECODEX( txtx, keyx, wrkx );
```

The DES encryption routines build an intermediate transposition table in the allocated "work" space. The table is created from the key by extensive bit transposition and substitution routines. Each time ENCODE or DECODE is called, the procs test to determine if the "work" table has been created. If the table isn't present, one is created. The table building step takes the majority of the CPU time required for this routine. For the sake of efficiency, the "work" area avoids the need to rebuild the tables when using the same key over. Rebuilding the table should be avoided if at all possible.

Therefore, you should try to associate the "work" area with the key so that the first time the key is used, the table is built. Each subsequent time the key is used, the same table should be used. Notice that, in general, this requires that work areas need to be in global space or within a storage area associated with a record. An example would be to include the work space within a TDF record. If the TDF was usually maintained in extended memory, extra space could be allocated in extended

memory associated with the record. Similarly, since authorization by IBM DES verification requires a DES encryption step, the work space allocated should be associated with the IDF that has the PIN verification DES key.

DECRYPT

Receives a string and "unscrambles" it by means of bit-shifting and complementing. It is compatible with the ENCRYPT procedure.

Syntax:

<stat> := DECRYPT(<str> , <lgth>)

where:

<stat>, INT,	contains TRUE (-1) if the string was successfully decrypted. Otherwise, FALSE (0).
<str>, STRING, ref,	contains the string to be decrypted and the results of the decryption.
<lgth>, INT,	contains the byte-length of the data to be decrypted. The number is in multiples of 8. The maximum number of words this proc will handle is 256.

example:

```
INT      stat;
INT      lgth;
STRING   .str[0:7];

lgth     :=8;
str      := "ABCD1234";
stat := DECRYPT( str, lgth );
```

DECRYPT^PIN

Used to decode encrypted PIN. The encrypted PIN is represented in graphic characters.

Syntax:

<stat> := DECRYPT^PIN(<e^PIN>, <key>, <d^PIN>)

where:

<stat>, INT,	returns TRUE (non-zero) if the PIN was decoded successfully.
<e^PIN>, INT:ref:8,	contains the DIEBOLD 910 encrypted PIN to be decoded.
<key>, INT:ref:4,	is the key used to decode the PIN.
<d^PIN>, INT:ref:8,	contains the decoded PIN in ASCII display form.

example:

```
INT    .encrypted^PIN[ 0:7 ],
        .key[ 0:3 ],
        .decrypted^PIN[ 0:7 ];

encrypted^pin  ':=' "2580=0=6;489==1;";
key           ':=' [%h0123,%h4567,%h89AB,%hCDEF];
IF DECRYPT^PIN( encrypted^PIN, key,
               decrypted^PIN ) THEN
    ...      !successful
ELSE
    ...      !error
```

Note: In 'graphic' characters the hexdigits "ABCDEF" are represented by ASCII characters ":", "<=>?" (colon, semi-colon, less-than, equal-sign, greater-than, and question mark.)

DECRYPT^PIN^1

Used to decode an encrypted PIN that has been represented by ASCII characters.

Syntax:

<stat> := DECRYPT^PIN^1(<e^PIN>, <key>, <d^PIN>)

where:

<stat>, INT,	returns TRUE (non-zero) if the PIN was decoded successfully.
<e^PIN>, INT:ref:8,	contains the encrypted PIN to be decoded.
<key>, INT:ref:4,	is the key used to decode the PIN.
<d^PIN>, INT:ref:8,	contains the decoded PIN in ASCII display form.

Example:

```
INT    .encrypted^PIN[ 0:7 ],
        .key[ 0:3 ],
        .decrypted^PIN[ 0:7 ];

encrypted^pin  ':= "2580D0D6B489DD1B";
key           ':= [%h0123,%h4567,%h89AB,%hCDEF];
IF DECRYPT^PIN^1( encrypted^PIN, key,
                  decrypted^PIN ) THEN
    ... !successful
ELSE
    ... !error
```

ENCODE

Provides a software implementation of the DES. ENCODE accepts text and a key, and returns encoded text.

Syntax:

CALL ENCODE(<txt> , <key> , <wrk>);

where:

<txt>, INT:ref:4, contains the text to be encoded upon entry to the PROC. Upon return from the PROC, it contains the encoded text.

<key>, INT:ref:4, is the key to be used to encode the text.

<wrk>, INT:ref:52, is a work area used to avoid recalculating the subkeys for each 64 bits of input.

example:

```
INT    .txt[ 0:3 ],  
        .key[ 0:3 ],  
        .wrk[ 0:51 ];  
  
txt  ':=' [%h0000,%h0000,%h0000,%h0000];  
key  ':=' [%h0123,%h4567,%h89AB,%hCDEF];  
CALL ENCODE( txt, key, wrk );
```

The DES encryption routines build an intermediate transposition table in the allocated "work" space. The table is created from the key by extensive bit transposition and substitution routines. Each time ENCODE or DECODE is called, the procs test to determine if the "work" table has been created. If the table isn't present, one is created. The table building step takes the majority of the CPU time required for this routine. For the sake of efficiency, the "work" area avoids the need to rebuild the tables when using the same key over. Rebuilding the table should be avoided if at all possible.

Therefore, you should try to associate the "work" area with the key so that the first time the key is used, the table is built. Each subsequent time the key is used, the same table should be used. Notice that, in general, this requires that work areas need to be in global space or within a storage area associated with a record. An example would be to include the work space within a TDF record. If the TDF was usually maintained in extended memory, extra space could be allocated in extended

memory associated with the record. Similarly, since authorization by IBM DES verification requires a DES encryption step, the work space allocated should be associated with the IDF that has the PIN verification DES key.

ENCODEx (extended address version)

Provides a software implementation of the DES. ENCODEx accepts text and a key, and returns encoded text.

Syntax:

CALL ENCODEx(<txtx> , <keyx> , <wrkx>);

where:

<txtx>, INT .EXT:ref:4, contains the text to be encoded upon entry to the PROC. Upon return from the PROC, it contains the encoded text.

<keyx>, INT .EXT:ref:4, is the key to be used to encode the text.

<wrkx>, INT .EXT:ref:52, is a work area used to avoid recalculating the subkeys for each 64 bits of input.

example:

```
INT    .ext txt[ 0:3 ],
        .ext key[ 0:3 ],
        .ext wrk[ 0:51 ];

txt    ':= ' [%h0000,%h0000,%h0000,%h0000];
key    ':= ' [%h0123,%h4567,%h89AB,%hCDEF];
CALL ENCODEx( txtx, keyx, wrkx );
```

The DES encryption routines build an intermediate transposition table in the allocated "work" space. The table is created from the key by extensive bit transposition and substitution routines. Each time ENCODE or DECODE is called, the procs test to determine if the "work" table has been created. If the table isn't present, one is created. The table building step takes the majority of the CPU time required for this routine. For the sake of efficiency, the "work" area avoids the need to rebuild the tables when using the same key over. Rebuilding the table should be avoided if at all possible.

Therefore, you should try to associate the "work" area with the key so that the first time the key is used, the table is built. Each subsequent time the key is used, the same table should be used. Notice that, in general, this requires that work areas need to be in global space or within a storage area associated with a record. An example would be to include the work space within a TDF record. If the TDF was usually maintained in extended memory, extra space could be allocated in extended

memory associated with the record. Similarly, since authorization by IBM DES verification requires a DES encryption step, the work space allocated should be associated with the IDF that has the PIN verification DES key.

ENCRYPT

Receives a string and "scrambles" it by means of bit-shifting and complementing. It is compatible with the DECRYPT procedure.

Syntax:

<stat> := ENCRYPT(<str> , <lgth>)

where:

<stat>, INT,	contains TRUE (-1) if the string was successfully encrypted. Otherwise, FALSE (0).
<str>, STRING:ref,	contains the string to be encrypted and the results of the encryption.
<lgth>, INT,	contains the byte-length of the data to be encrypted. The number is in multiples of 8. The maximum number of words this proc will handle is 256.

example:

```
INT      stat;
INT      lgth;
STRING   .str[0:7];

lgth     :=8;
str      := "ABCD1234";
stat := ENCRYPT( str, lgth );
```

PROCESS^DES^PIN

An implementation of the DES-based PIN verification algorithm. It can handle variable-length PINs, ranging from 1 to 16 digits.

Syntax:

```
<stat> := PROCESS^DES^PIN( <pin> , <pin^lgth> , <val^data> , <val^ofst> ,  
                           <val^lgth> , <val^pad> , <encr^key> , <dec^tab> , <pin^ofst> ,  
                           <ofst^lgth> , <min^pin^lgth> , <max^pin^lgth> , <process^type> ,  
                           <status> , <des^ofst^ind> );
```

where:

<stat>, INT,	returns TRUE (non^zero) if <pin> was valid, else returns FALSE.
<pin>, STRING:ref,	is the customer-entered <pin>, and ranges from 1 to 16 digits.
<pin^lgth>, INT:val,	is the number of digits entered for the PIN; range is 1 to 16.
<val^data>, STRING:ref,	is usually the customer's PAN, but can be any hexadecimal data.
<val^ofst>, INT:val,	is the number of bytes to offset into <val^data> to find the beginning of the validation data. The default is 0. The valid range is 0 to 19.
<val^lgth>, INT:val,	is the number of bytes, starting at <val^data>[val^ofst], to use. The default value is 16. The valid range is 1 to \$min(16, (19 - <val^ofst>)).
<val^pad>, STRING:ref,	if specified, is a one-byte character to pad <val^data> with to make it 16 bytes. If not passed, "F" is used.
<encr^key>, INT:ref,	is a 4-word hex encryption key to use for DES encoding.

<dec^tab>, STRING:ref,	if specified, is a 16-byte decimal string to use for decimalizing the encrypted <val^data>. If not passed, "0123456789012345" is used.
<pin^ofst>, STRING:ref,	if specified, is the offset value mod10-added to the intermediate PIN; default is all zeroes ([16 * ["0"]]); i.e., no offset is used. <ofst^lgth> must be specified if <pin^ofst> is specified.
<ofst^lgth>, INT:val,	if specified, is the number of bytes in <pin^ofst>; default is 16. NOTE that this value must be greater than or equal to <min^pin^lgth>. <pin^ofst > must be specified if <ofst^lgth> is specified.
<min^pin^lgth>, INT:val,	if specified, is the minimum number of digits the customer must enter; this parameter indicates the number of digits this utility will check. Do not make the mistake of passing 1 for this parameter. The default is 4.
<max^pin^lgth>, INT:val,	if specified, is the maximum number of digits the customer may enter; default is 16.
<process^type>, INT:val,	if specified, is the type of processing to perform (default is 1)
<des^ofst^ind>, STRING:val,	if specified, and if the pin^lgth and the ofst^lgth are not equal, this denotes whether the PIN data for comparison is right justified or left justified. If omitted, or passed as "R", denotes right justification, otherwise it's tested as left justification. This only impacts processing when process^type is 1.

<u>value</u>	<u>type of processing</u>
1	Given <val^data>, <offset>, and <encr^key>, verify that the PIN is valid.
2	Given <pin>, <val^data>, and <encr^key>, generate the appropriate offset value of length <ofst^lgth>.

NOTE: <pin^ofst> and <ofst^lgth> become required parameters for this <process^type>. Also, the description of steps 6 and 7 in the following validation narrative are changed (basically, a modulo-10 subtract is performed instead of a modulo-10 addition).

3 Given <val^data>, <encr^key>, and <offset>, generate the appropriate PIN value. <Pin^lgth> determines how many digits are in the generated PIN.

<status>, INT:ref, is optional; if passed, on return this proc will contain the following:

- 0 - no error, PIN was valid
- 1 - error, PIN was invalid (for any reason)
- 2 - error, required parameter missing from call
- 3 - error, invalid parameter passed

examples:

```

INT      .encr^key[0:3];
INT      err;
INT      max^pin^lgth;
INT      min^pin^lgth;
INT      ofst^lgth;
INT      pin^lgth;
INT      stat;
INT      val^lgth;
INT      val^ofst;
INT      verify^pin;

STRING   .dec^tab[0:15];
STRING   .pin[0:15];
STRING   .pin^ofst[0:15];
STRING   .val^data[0:15];
STRING   val^pad;

pin      := "00000000000001234";
pin^lgth := 4;
val^data := "4321555566667777";
val^ofst := 1;
val^lgth := 10;
val^pad  := "F";
encr^key := [%h0123, %h4567, %h89ab, %hcd ef];
dec^tab  := "0123456789012345";
pin^ofst := "9240000000000000";

```

```
ofst^lgth      := 4;  
min^pin^lgth   := 4;  
max^pin^lgth   := 16;
```

TYPE 1

```
verify^pin      := 1;  
stat := process^des^pin( pin, pin^lgth, val^data, val^ofst, val^lgth,  
                          val^pad, encr^key, dec^tab, pin^ofst,  
                          ofst^lgth, min^pin^lgth, max^pin^lgth,  
                          verify^pin, err, "R" !des^ofst^ind!);
```

TYPE 2

```
pin^ofst        := "*****";  
verify^pin      := 2;  
stat := process^des^pin( pin, pin^lgth, val^data, val^ofst, val^lgth,  
                          val^pad, encr^key, dec^tab, pin^ofst,  
                          ofst^lgth, min^pin^lgth, max^pin^lgth,  
                          verify^pin, err );
```

NOTE: On return, pin^ofst will equal "9240*****".

TYPE 3

```
pin             := "*****";  
verify^pin      := 3;  
stat := process^des^pin( pin, pin^lgth, val^data, val^ofst, val^lgth,  
                          val^pad, encr^key, dec^tab, pin^ofst,  
                          ofst^lgth, min^pin^lgth, max^pin^lgth,  
                          verify^pin, err );
```

NOTE: On return, pin will equal "0000000000001234".

The basic steps in validation of a PIN using DES are

1. Determine the validation data. This is usually some portion of the PAN, determined by an offset into the PAN and the length (number of bytes) to use.
2. If necessary, pad this validation data on the right with a pad character to make it 16 bytes long.
3. Encode this data using the specified encryption key, which is usually the institution's PIN key.
4. Decimalize the result by mapping each of the 16 bytes into a decimal digit.
5. Take the leftmost <n> digits of this result, where <n> is the number of customer-entered digits in the PIN. This value is called the intermediate PIN.
6. Right-align the intermediate PIN with the offset data, and perform a modulo-10 addition.
7. Compare the rightmost <n> digits of the result with the customer-entered PIN, where <n> is the value passed for <min^PIN^lgth> (or 4 if <min^PIN^lgth> is not passed). If equal, the PIN is valid.

ACI Worldwide Inc.

Section 9

Format Conversion Procs

ALL^ASCII^DOUBLE

Converts an ASCII string to an INT(32) value skipping non-numerics. Values can be negative by placing a "-" sign before the number.

Syntax:

CALL ALL^ASCII^DOUBLE(<txt> , <lgth> , <val>)

where:

<txt>, STRING:ref, is the string to be converted.

<lgth>, INT, is the length of the string to be converted. If the length is 0, then a value of 0d will be returned.

<val>, INT(32):ref, contains the converted string.

example:

```
STRING      .txt[ 0:4 ];  
INT         lgth;  
INT(32)     val;
```

```
txt      ':= "12345";  
lgth     := 5;  
CALL ALL^ASCII^DOUBLE( txt, lgth, val );
```

ALL^ASCII^FIXED

Converts an ASCII string to a FIXED value skipping non-numerics. Values can be negative by placing a "-" sign before the number.

Syntax:

CALL ALL^ASCII^FIXED(<txt> , <lgth> , <val>)

where:

<txt>, STRING:ref, is the string to be converted.

<lgth>, INT, is the length of the string to be converted. If
the length is 0, then a value of 0F will be
returned.

<val>, FIXED:ref, contains the converted string.

example:

```
STRING      .txt[ 0:4 ];
INT          lgth;
FIXED       val;

txt      ':= "-1234";
lgth     := 5;
CALL ALL^ASCII^FIXED( txt, lgth, val );
```

ALL^ASCII^INTEGER

Converts a string to an INT value. It first removes all non-numeric characters and returns a negative value if negative.

Syntax:

CALL ALL^ASCII^INTEGER(<val> , <txt> , <lgth>)

where:

<val>, INT:ref,	contains the converted value. It will contain a negative value if the input was negative.
<txt>, STRING:ref,	contains the text to be converted.
<lgth>, INT,	is the length of the text to be edited. If the length is 0, then a value of 0 is returned.

example:

```
STRING    .txt[0:4];
INT        lgth;
INT        val;

txt      := "12345";
lgth     := 5
CALL ALL^ASCII^INTEGER( val, txt, lgth )
```

ALL^HEX

Used to check if all characters of the given input string are valid hexadecimal.

Syntax:

<stat> := ALL^HEX(<hex^str> , <lgth>)

where:

<stat>, INT, return TRUE(-1) if the string contains all hex characters. Otherwise, it returns FALSE(0).

<hex^str>, STRING:ref, is the string to check if all hexadecimal.

<lgth>, INT, length of the input string.

example:

```
STRING    .txt[ 0:7 ],
INT        lgth;

txt    ':= ' "01abcdef";
lgth    := 8;
IF ALL^HEX( txt , lgth ) THEN
    ! success
ELSE
    ! error
```

ALL^NUMERIC

Checks an ASCII field for all numeric characters. Returns TRUE (-1) if all characters are numeric.

Syntax:

<stat> := ALL^NUMERIC(<txt> , <lgth>)

where:

<stat>, INT, returns TRUE (-1) if the field supplied is numeric; otherwise, it returns FALSE (0).

<txt>, STRING:ref, is the text to be verified.

<lgth>, INT, is the number of characters to verify.

example:

```
STRING      .txt[ 0:7 ],
INT         txt^lgth;

txt         ':=' "01234567";
txt^lgth    := 8;
IF ALL^NUMERIC( txt, txt^lgth ) THEN
    ! all numeric field
ELSE
    ! error
```

ALL^NUMERICX (extended address version)

Checks an ASCII field for all numeric characters. Returns TRUE (-1) if all characters are numeric.

Syntax:

<stat> := ALL^NUMERICX(<txt> , <lgth>)

where:

<stat>, INT, returns TRUE (-1) if the field supplied is numeric; otherwise, it returns FALSE (0).

<txt>, STRING.EXT:ref, is the text to be verified.

<lgth>, INT, is the number of characters to verify.

example:

```
STRING      .ext txt[ 0:7 ],
INT          txt^lgth;

txt          := "01234567";
txt^lgth     := 8;
IF ALL^NUMERICX( txt, txt^lgth ) THEN
    ! all numeric field
ELSE
    ! error
```

ASCII^DOUBLE

Converts ASCII to a double-word value. The number to be converted must be numeric and greater than or equal to zero. The PROC will return TRUE or FALSE to indicate whether or not the conversion was successful. An overflow condition or a non-numeric character will result in a return of FALSE.

Syntax:

<stat> := ASCII^DOUBLE(<txt> , <txt^lgth> , <val>)

where:

<stat>, INT,	returns TRUE (-1) if the conversion was successful; otherwise, FALSE (0) is returned.
<txt>, STRING:ref,	contains the variable length numeric text to be converted.
<txt^lgth>, INT,	contains the length of the string array.
<val>, INT(32):ref,	contains the converted double-word number.

example:

```

STRING      .txt[ 0:3 ];
INT         txt^lgth;
INT (32)    val;

txt         := "1982"
txt^lgth    := 4;
IF ASCII^DOUBLE( txt, txt^lgth, val ) THEN
    ! successful
ELSE
    ! error

```

ASCII^DOUBLEX (extended address version)

Converts ASCII to a double-word value. The number to be converted must be numeric and greater than or equal to zero. The PROC will return TRUE or FALSE to indicate whether or not the conversion was successful. An overflow condition or a non-numeric character will result in a return of FALSE.

Syntax:

<stat> := ASCII^DOUBLEX(<txtx> , <txt^lgth> , <valx>)

where:

<stat>, INT,	returns TRUE (-1) if the conversion was successful; otherwise, FALSE (0) is returned.
<txtx>, STRING.EXT:ref,	contains the variable length numeric text to be converted.
<txt^lgth>, INT,	contains the length of the string array.
<valx>, INT(32).EXT:ref,	contains the converted double-word number.

example:

```
STRING      .ext txtx[ 0:3 ];
INT          txt^lgth;
INT (32)     .ext valx;

txtx        ':= "1982"
txt^lgth     := 4;
IF ASCII^DOUBLEX( txtx, txt^lgth, valx ) THEN
    ! successful
ELSE
    ! error
```

ASCII^FIXED

Converts ASCII to a FIXED (64 bit) value. The number to be converted must be numeric and greater than or equal to zero. The PROC will return TRUE or FALSE to indicate whether or not the conversion was successful. An overflow condition or a non-numeric character will result in a return of FALSE.

Syntax:

<stat> := ASCII^FIXED(<txt> , <txt^lgth> , <val>)

where:

<stat>, INT,	returns TRUE (-1) if the conversion was successful; otherwise, FALSE (0) is returned.
<txt>, STRING:ref,	contains the variable length numeric text to be converted.
<txt^lgth>, INT,	contains the length of the string array.
<val>, FIXED(*):ref,	contains the converted FIXED number.

example:

```

STRING    .txt[ 0:9 ];
INT        txt^lgth;
FIXED      val;

txt        := "0123456789";
txt^lgth   := 10;
IF ASCII^FIXED( txt, txt^lgth, val ) THEN
    ! successful
ELSE
    ! error

```

ASCII^FIXEDX (extended address version)

Converts ASCII to a FIXED (64 bit) value. The number to be converted must be numeric and greater than or equal to zero. The PROC will return TRUE or FALSE to indicate whether or not the conversion was successful. An overflow condition or a non-numeric character will result in a return of FALSE.

Syntax:

<stat> := ASCII^FIXEDX(<txtx> , <txt^lgth> , <valx>)

where:

<stat>, INT,	returns TRUE (-1) if the conversion was successful; otherwise, FALSE (0) is returned.
<txtx>, STRING.EXT:ref,	contains the variable length numeric text to be converted.
<txt^lgth>, INT,	contains the length of the string array.
<valx>, FIXED(*).EXT:ref,	contains the converted FIXED number.

example:

```
STRING      .ext txtx[ 0:9 ];
INT          txt^lgth;
FIXED(*)     .ext valx;

txtx         ':= "0123456789";
txt^lgth     := 10;
IF ASCII^FIXEDX( txtx, txt^lgth, valx ) THEN
    ! successful
ELSE
    ! error
```

ASCII^INTEGER

Converts ASCII to an INT value. The number to be converted must be numeric and greater than or equal to zero. The PROC will return TRUE or FALSE to indicate whether or not the conversion was successful. An overflow condition or a non-numeric character will result in a return of FALSE.

Syntax:

<stat> := ASCII^INTEGER(<txt> , <txt^lgth> , <val>)

where:

<stat>, INT,	returns TRUE (-1) if the conversion was successful; otherwise, FALSE (0) is returned.
<txt>, STRING:ref,	contains the variable length numeric text to be converted.
<txt^lgth>, INT,	contains the length of the string array.
<val>, INT:ref,	contains the converted INT number.

example:

```

STRING      .txt[ 0:1 ];
INT          txt^lgth,
              val;

txt          := "12";
txt^lgth     := 2;
IF ASCII^INTEGER( txt, txt^lgth, val ) THEN
    ! successful
ELSE
    ! error

```

ASCII^INTEGERX (extended address version)

Converts ASCII to an INT value. The number to be converted must be numeric and greater than or equal to zero. The PROC will return TRUE or FALSE to indicate whether or not the conversion was successful. An overflow condition or a non-numeric character will result in a return of FALSE.

Syntax:

<stat> := ASCII^INTEGERX(<txtx> , <txt^lgth> , <valx>)

where:

<stat>, INT,	returns TRUE (-1) if the conversion was successful; otherwise, FALSE (0) is returned.
<txtx>, STRING.EXT:ref,	contains the variable length numeric text to be converted.
<txt^lgth>, INT,	contains the length of the string array.
<valx>, INT.EXT:ref,	contains the converted INT number.

example:

```
STRING      .ext txtx[ 0:1 ];
INT          txt^lgth,
             .ext valx;

txtx        ':=' "12";
txt^lgth     := 2;
IF ASCII^INTEGERX( txtx, txt^lgth, valx ) THEN
    ! successful
ELSE
    ! error
```

ASCII^TO^D2230

Used to translate a string from ASCII character to Dataproducts 2230 line printer subset.

Syntax:

CALL ASCII^TO^D2230(<src^str> , <lgth>)

where:

<src^str>, STRING:ref, initially contains the variable length string to be translated. After execution of the PROC, contains the translated string.

NOTE: It is up to the user to declare <src^str> long enough to receive the converted string.

<lgth>, INT, contains the length of the string array.

example:

```
STRING    .src^str[ 0:19 ];  
INT       lgth;  
  
src^str    := "translate";  
lgth       := 9;  
CALL ASCII^TO^D2230( src^str, lgth );
```

ASCII^TO^EBCDIC

Used to translate a string from ASCII character to EBCDIC.

Syntax:

CALL ASCII^TO^EBCDIC(<src^str> , <lgth>);

where:

<src^str>, STRING:ref, initially contains the variable length string to
be translated. After execution of the PROC,
contains the translated string.

<lgth>, INT, contains the length of the string array.

example:

```
STRING      .src^str[ 0:4 ];  
INT         lgth;  
  
src^str      ':=' "ASCII";  
lgth         := 5;  
CALL ASCII^TO^EBCDIC( src^str, lgth );
```

ASCII^TO^EBCDICX (extended address version)

Used to translate a string from ASCII character to EBCDIC.

Syntax:

CALL ASCII^TO^EBCDICX(<src^strx> , <lgth>);

where:

<src^strx>, STRING .EXT:ref, initially contains the variable length string to be translated. After execution of the PROC, contains the translated string.

<lgth>, INT, contains the length of the string array.

example:

```
STRING      .ext src^strx[ 0:4 ];  
INT         lgth;  
  
src^strx    := "ASCII";  
lgth        := 5;  
CALL ASCII^TO^EBCDICX( src^strx, lgth );
```

ASTERISK^FILL

Replaces leading zeroes, if any, with asterisks. For example:

input string contains	"000189"
output will be	"***189"

Syntax:

CALL ASTERISK^FILL(<input^str> , <str^lgth>)

where:

<input^str>, STRING:ref	initially contains the string to be changed. Will contain the edited output after suppression.
-------------------------	---

<str^lgth>, INT,	contains the length of the input string.
------------------	--

example:

```
INT      lgth
STRING   input^str[0:5];

input^str ':=' "000189";
lgth      :=6;
CALL ASTERISK^FILL( input^str , lgth )
```

BASE64^DECODE

This procedure converts base64-encoded data to binary format according to the standard set forth in RFC 1521, section 5.2 – Base64 Content-Transfer-Encoding (N. Borenstein, 1993). Every four bytes of base64-encoded data become three bytes of decoded binary data. All non-base64 characters encountered in the input data (e.g. carriage return/line feed) are ignored. However, the total number of valid base64 characters in the input data must be a multiplier of four.

For example:

```
Input      "ASNfZ4mrze8="
Yields     [%h0123456789abcdef]
(The '=' in the input string is a pad character.)
```

Syntax:

```
<stat> := base64^decode( <input^data>, <input^lgth>,  
                        <output^data>, <output^lgth> );
```

where: (all parameters are required)

<stat>, INT,	returns FALSE (0) if an error occurs; otherwise, TRUE (-1) is returned.
<input^data>, STRING .EXT:ref,	the base64-encoded data to be decoded.
<input^lgth>, INT,	the byte length of <input^data>.
<output^data>, STRING .EXT:ref,	the decoded binary data.
<output^lgth>, INT .EXT:ref,	the byte length of the <output^data> contents.

Example:

```
int    input^lgth;  
int    output^lgth;  
string .input^data[ 0:11 ] := [ "ASNfZ4mrze8=" ];  
string .output^data[ 0:7 ];  
  
input^lgth := 12;  
err := base64^decode( input^data, input^lgth, output^data, output^lgth );
```

BASE64^ENCODE

This procedure converts binary data to base64-encoded data according to the standard set forth in RFC 1521, section 5.2 – Base64 Content-Transfer-Encoding (N. Borenstein, 1993). Every three bytes of binary data become four bytes of base64-encoded data. Pad characters are appended if the input data length is not a multiple of three. NOTE: This procedure deviates slightly from the standard in that it does not insert a carriage return/line feed after every 76th output character.

For example:

```
Input      [%h0123456789abcdef]
Yields     "ASNfZ4mrze8="
(The '=' in the output string is a pad character.)
```

Syntax:

```
<stat> := base64^encode( <input^data>, <input^lgth>,  
                        <output^data>, <output^lgth> );
```

where: (all parameters are required)

<stat>, INT, returns FALSE (0) if an error occurs;
 otherwise, TRUE (-1) is returned.

<input^data>, STRING .EXT:ref, the binary data to encode.

<input^lgth>, INT, the byte length of <input^data>.

<output^data>, STRING .EXT:ref, the base64-encoded data.

<output^lgth>, INT .EXT:ref, the byte length of the <output^data>
 contents.

Example:

```
int      input^lgth;
int      output^lgth;
string   .input^data[ 0:7 ] := [ %h01, %h23, %h45, %h67,
                                %h89, %hab, %hcd, %hef ];
string   .output^data[ 0:11 ];

input^lgth := 8;
err := base64^encode( input^data, input^lgth, output^data, output^lgth );
```


BASE94^ENCODE

This procedure converts binary data to base94-encoded format. Every four bytes of binary data become five bytes of base94-encoded data. The index for base94 includes the first 94 printable ASCII characters starting with the space character (ASCII 32) through the closing brace (ASCII 125).

For example:

```
Input      [%h0022446688aaccee]
Yields     "j5b{5TfV=S"
(The '=' in the output string is a pad character.)
```

Syntax:

<stat> := base94^encode(<input^data>, <input^lgth>,
 <output^data>, <output^lgth>);

where: (all parameters are required)

<stat>, INT, returns FALSE (0) if an error occurs;
otherwise, TRUE (-1) is returned.

<input^data>, STRING .EXT:ref, the binary data to encode.

<input^lgth>, INT, the byte length of <input^data>.

<output^data>, STRING .EXT:ref, the base94-encoded data.

<output^lgth>, INT .EXT:ref, the byte length of the <output^data>
contents.

Example:

```
int      input^lgth;
int      output^lgth;
string   .input^data[ 0:7 ] := [ %h00, %h22, %h44, %h66,
                                %h88, %haa, %hcc, %hee ];
string   .output^data[ 0:9 ];

input^lgth := 8;
err := base94^encode( input^data, input^lgth, output^data, output^lgth );
```

BINARY^HEXCHAR

Converts binary data to hexadecimal display form. Each 4 bits is unpacked into an 8-bit character representing the 4-bit binary value. For example:

<binary> containing %B00011110 (i.e., a binary 1 in 4 bits followed by a binary 14 in the next 4 bits)

converts to

<hex> containing %B0011000101000101 = "1E"

Syntax:

<stat> := BINARY^HEXCHAR(<hex> , <lgth> , <binary>)

where:

<stat>, INT, returns FALSE (0) if <lgth> is not an even number; otherwise, TRUE (-1) is returned.

<hex>, STRING:ref, contains the result of the binary to hex conversion. The length of <hex> must be twice the length of <binary>.

<lgth>, INT, contains the length of the output string. <lgth> must be an even number.

<binary>, STRING:ref, contains the binary array to be converted.

example:

```

STRING      .hex^str[ 0:13 ],
              .binary^str[ 0:6 ];
INT          hex^lgth;

binary^str := [%h12,%h34,%h56,%h78,%h9A,%hBC];
hex^lgth   := 14;
IF BINARY^HEXCHAR( hex^str, hex^lgth, binary^str ) THEN
    ! successful
ELSE
    ! error

```

BINARY^HEXCHARX (extended address version)

Converts binary data to hexadecimal display form. Each 4 bits is unpacked into an 8-bit character representing the 4-bit binary value. For example:

<binary> containing %B00011110 (i.e., a binary 1 in 4 bits followed by a binary 14 in the next 4 bits)

converts to

<hex> containing %B0011000101000101 = "1E"

Syntax:

<stat> := BINARY^HEXCHARX(<hexx> , <lgth> , <binaryx>)

where:

<stat>, INT, returns FALSE (0) if <lgth> is not an even number; otherwise, TRUE (-1) is returned.

<hexx>, STRING.EXT:ref, contains the result of the binary to hex conversion. The length of <hex> must be twice the length of <binary>.

<lgth>, INT, contains the length of the output string. <lgth> must be an even number.

<binaryx>, STRING.EXT:ref, contains the binary array to be converted.

example:

```
STRING      .ext hex^strx[ 0:13 ],
              .ext binary^strx[ 0:6 ];
INT          hex^lgth;

binary^str := [%h12,%h34,%h56,%h78,%h9A,%hBC];
hex^lgth   := 14;
IF BINARY^HEXCHARX( hex^strx, hex^lgth, binary^strx ) THEN
    ! successful
ELSE
    ! error
```

CRNCY^DECIMAL^PLACES

Accepts the ASCII representation of a currency code and returns the number of decimal places for the currency code as an integer. The data for the countries was taken from the ISO manual.

Syntax:

<stat> := CRNCY^DECIMAL^PLACES(<crncy^cde>)

where:

<stat>, INT, contains the number of decimal places or, -1
if the input currency code was not found

<crncy^cde>, STRING:ref, contains the currency code for which the
number of decimal places is to be returned.

example:

```
STRING      .ext crncy^cde[ 0:2 ];
INT          stat;

crncy^cde    ':=' "840";
stat         := CRNCY^DECIMAL^PLACES( crncy^cde );
```

Alternative Entry Point #1

GET^CNTRY^DATA

Accepts the ASCII representation of the country code and returns the currency code used by the country, the standard abbreviation of the name of the country, and the number of decimal places used by the currency. The data for the countries was taken from the ISO manual.

Syntax:

<stat> := GET^CNTRY^DATA(<crncy^cde> , <cntry^cde> , <cntry^abrv> ,
 <dec^place>)

where:

<stat>, INT,	contains TRUE(-1) if the country code was found. Otherwise, FALSE (0).
<crncy^cde>, STRING:ref,	contains the currency code used by the country.
<cntry^cde>, STRING:ref,	contains the country code for which the data is desired.
<cntry^abrv>, STRING:ref,	contains the standard abbreviation of the name of the country (eg "US" , "UK").
<dec^place>, INT:ref,	contains the number of decimal places used by the currency of the country.

example:

```
STRING      .ext crncy^cde[ 0:2 ],
              .ext cntry^cde[ 0:2 ],
              .ext cntry^abrv[ 0:1 ];
INT          .ext dec^place,
              stat;

cntry^cde    ' := ' "086";
stat         := GET^CNTRY^DATA( crncy^cde, cntry^cde,
                                cntry^abrv, dec^place );
```

Alternative Entry Point #2

GET^CRNCY^CDE^ALPHA

Accepts the ASCII representation of the currency code and returns the standard abbreviation of the name of the country, the number of decimal places used by the currency, and the alpha representation of the numeric currency code. The data for the countries was taken from the ISO manual.

Syntax:

<stat> := GET^CRNCY^CDE^ALPHA(<crncy^cde> , <cntry^abrv> ,
<dec^place> , <crncy^cde^alpha>)

where:

<stat>, INT,	contains the number of decimal places or, -1 if the input currency code was not found.
<crncy^cde>, STRING:ref,	contains the currency code.
<cntry^abrv>, STRING:ref,	contains the standard abbreviation of the name of the country (eg "US" , "UK").
<dec^place>, INT:ref,	contains the number of decimal places used by the currency.
<crncy^cde^alpha>, STRING:ref,	contains the alpha representation of the numeric currency cde (i.e. "840" = "USD")

example:

```

STRING      .ext crncy^cde[ 0:2 ],
              .ext cntry^abrv[ 0:1 ],
              .ext crncy^cde^alpha[ 0:2 ];
INT          .ext dec^place,
              stat;

cntry^cde    ':= "840";
stat         := GET^CNTRY^DATA( crncy^cde, ,cntry^abrv,
                                dec^place, crncy^cde^alpha );
  
```

Alternative Entry Point #3

CRNCY^DECIMAL^PLACES^ALPHA

Accepts the ALPHA representation of the numeric currency code and returns the numeric currency code used by the country, and the number of decimal places used by the currency. The data for the countries was taken from the ISO manual.

Syntax:

<stat> := CRNCY^DECIMAL^PLACES^ALPHA(<crncy^cde>, , , <dec^place>
, <crncy^cde^alpha>)

where:

<stat>, INT, contains the number of decimal places or, -1
if the input currency code was not found.

<crncy^cde>, STRING:ref, contains the currency code.

<dec^place>, INT:ref, contains the number of decimal places used
by the currency.

<crncy^cde^alpha>, STRING:ref, contains the alpha representation of the
numeric currency code (i.e. "840" = "USD")

example:

```
STRING      .ext crncy^cde[ 0:2 ],  
              .ext crncy^cde^alpha[ 0:2 ];  
INT          .ext dec^place,  
              stat;  
  
crncy^cde^alpha  ':= ' "USD";  
stat      := CRNCY^DECIMAL^PLACES^ALPHA( crncy^cde, , ,  
              dec^place, crncy^cde^alpha );
```

DOLLAR^SUPPRESS

Floats a dollar sign.

Syntax:

CALL DOLLAR^SUPPRESS(<amt> , <max^lgth>)

where:

<amt>, STRING:ref, contains the string to modify. Leading positions must be zero filled instead of space filled.

<max^lgth>, INT, contains the length of the string.

example:

```
STRING    .amt[ 0:6 ];  
INT       max^lgth;
```

```
amt        := "0000012";  
max^lgth   := 7;  
CALL DOLLAR^SUPPRESS( amt, max^lgth );
```

```
result:    amt    = "  $12"
```

```
amt        := "0000.12";  
max^lgth   := 7;  
CALL DOLLAR^SUPPRESS( amt, max^lgth );
```

```
result:    amt    = " $0.12"
```

DOUBLE^ASCII

Converts an double-word value to an ASCII number. The number to be converted must be numeric and greater than or equal to zero. The PROC will return TRUE or FALSE to indicate whether or not the conversion was successful. An overflow condition will result in a return of FALSE.

Syntax:

<stat> := DOUBLE^ASCII(<txt> , <txt^lgth> , <val>)

where:

<stat>, INT, returns TRUE (-1) if the conversion was successful; otherwise, FALSE (0) is returned.

<txt>, STRING:ref, contains the converted, variable length numeric text.

NOTE: It is up to the user to declare <txt> long enough to receive the converted string.

<txt^lgth>, INT, contains the length of the string array.

<val>, INT(32):ref, contains the double-word number to be converted.

example:

```
STRING    .txt^str[ 0:7 ];
INT        txt^lgth;
INT (32)   val;

txt^lgth   := 8;
val        := 82164d;
IF DOUBLE^ASCII( txt^str, txt^lgth, val ) THEN
    ! successful
ELSE
    ! error
```

DOUBLE^ASCII (extended address version)

Converts an double-word value to an ASCII number. The number to be converted must be numeric and greater than or equal to zero. The PROC will return TRUE or FALSE to indicate whether or not the conversion was successful. An overflow condition will result in a return of FALSE.

Syntax:

<stat> := DOUBLE^ASCII(<txt> , <txt^lgth> , <val>)

where:

<stat>, INT,	returns TRUE (-1) if the conversion was successful; otherwise, FALSE (0) is returned.
<txt>, STRING.EXT:ref,	contains the converted, variable length numeric text.

NOTE: It is up to the user to declare <txt> long enough to receive the converted string.

<txt^lgth>, INT,	contains the length of the string array.
<val>, INT(32):ref,	contains the double-word number to be converted.

example:

```
STRING      .ext txt^strx[ 0:7 ];
INT          txt^lgth;
INT (32)     val;

txt^lgth     := 8;
val          := 82164d;
IF DOUBLE^ASCII( txt^strx, txt^lgth, val ) THEN
    ! successful
ELSE
    ! error
```

DOUBLE^DOLLAR

Converts a double-word value to an ASCII string formatted with a \$ and two decimal places. Leading zeros are suppressed.

Syntax:

CALL DOUBLE^DOLLAR(<amt> , <txt^lgth> , <txt>)

where:

<amt>, INT(32):ref	contains the double word amt to be converted and formatted.
<txt^lgth>, INT,	contains the number of bytes the output string will contain.
<txt>, STRING:ref,	contains the output string. If negative the negative sign will be the right most character (i.e. -99999 will be formatted as \$999.99-).

NOTE: It is up to the user to declare <txt> large enough to receive the converted string. The largest number that an INT(32) can represent is 10 digits long. The maximum number of characters that the string can contain is thus 13, including the "\$", the ".", and the "-" signs.

If <txt^lgth> > 12 (positive number), or
13 (negative number) then
the output will be left-justified.
if <txt^lgth> < 12 (positive number), or
13 (negative number) then
the output will be truncated.

examples:

! POSITIVE DOUBLE NUMBER

STRING .txt^str[0:12];

INT txt^lgth;

INT(32) amt;

txt^lgth := 13;

amt := 82164d;

CALL DOUBLE^DOLLAR(amt, txt^lgth, txt^str);

result: txt^str = "\$ 821.64 "

!NEGATIVE DOUBLE NUMBER

txt^lgth := 13;

amt := -82164d;

CALL DOUBLE^DOLLAR(amt, txt^lgth, txt^str);

result: txt^str = "\$ 821.64-"

DOUBLE^DOLLAR^ASTERISK

Converts an INT(32) value to an ASCII string formatted with a \$ and two decimal places. Leading zeroes are replaced with asterisks.

Syntax:

CALL DOUBLE^DOLLAR^ASTERISK(<amt> , <txt^lgth> , <txt>)

where:

<amt>, INT(32):ref	contains the value to be converted.
<txt^lgth>, INT,	contains the maximum number of bytes you want the display string to contain.
<txt>, STRING:ref,	contains the output string. It will contain a \$ sign, two decimal places and a negative sign, if negative. Format \$*99999.99-.

NOTE: It is up to the user to declare <txt> large enough to receive the converted string. The largest number that an INT(32) can represent is 10 digits long. The maximum number of characters that the string can contain is thus 13, including the "\$", the ".", and the "-" signs.

If <txt^lgth> > 12 (positive number), or
13 (negative number) then
the output will be left-justified.
if <txt^lgth> < 12 (positive number), or
13 (negative number) then
the output will be truncated.

examples:

```
! POSITIVE DOUBLE NUMBER
STRING      .output^text[ 0:12 ];
INT         txt^lgth;
INT(32)     .amt;
```

```
txt^lgth    := 10;
amt         := 84164d;
CALL DOUBLE^DOLLAR^ASTERISK( amt, txt^lgth, txt );
```

```
result:     txt      = "$*****841.64 "
```

```
! NEGATIVE DOUBLE NUMBER
```

```
txt^lgth    := 13;
amt         := -84164d;
CALL DOUBLE^DOLLAR^ASTERISK( amt, txt^lgth, txt );
```

```
result:     txt      = "$*****841.64-"
```

EBCDIC^TO^ASCII

Used to translate a string from EBCDIC characters to ASCII.

Syntax:

CALL EBCDIC^TO^ASCII(<src^str> , <lgth>);

where:

<src^str>, STRING:ref, initially contains the variable length string to
be translated. After execution of the PROC,
contains the translated string.

NOTE: It is up to the user to declare <src^str> long enough to receive the
converted string.

<lgth>, INT, contains the length of the string array.

example:

```
STRING      .src^str[ 0:4 ];  
INT         lgth;  
  
src^str      ':= ' [%hC1,%hE2,%hC3,%hC9,%hC9];  
lgth         := 5;  
CALL EBCDIC^TO^ASCII( src^str, lgth )
```

EBCDIC^TO^ASCII (extended address version)

Used to translate a string from EBCDIC characters to ASCII.

Syntax:

CALL EBCDIC^TO^ASCII(<src^strx> , <lgth>);

where:

<src^strx>, STRING .EXT:ref, initially contains the variable length string to be translated. After execution of the PROC, contains the translated string.

NOTE: It is up to the user to declare <src^strx> long enough to receive the converted string.

<lgth>, INT, contains the length of the string array.

example:

```
STRING .ext src^strx[ 0:4 ];  
INT          lgth;  
  
src^strx      :=' [%hC1,%hE2,%hC3,%hC9,%hC9];  
lgth          := 5;  
CALL EBCDIC^TO^ASCII( src^strx, lgth )
```

FIXED^ASCII

Converts a FIXED (64 bit) value to an ASCII number. The number to be converted must be numeric and greater than or equal to zero. The PROC will return TRUE or FALSE to indicate whether or not the conversion was successful. An overflow condition will result in a return of FALSE.

Syntax:

<stat> := FIXED^ASCII(<txt> , <txt^lgth> , <val>)

where:

<stat>, INT, returns TRUE (-1) if the conversion was successful; otherwise, FALSE (0) is returned.

<txt>, STRING:ref, contains the converted, variable length numeric text.

NOTE: It is up to the user to declare <txt> long enough to receive the converted string.

<txt^lgth>, INT, contains the length of the string array.

<val>, FIXED(*), contains the FIXED number to be converted.

example:

```
STRING    .txt^str[ 0:17 ];
INT        txt^lgth;
FIXED      val;

txt^lgth   := 18;
val        := 23984098493983494f;
IF FIXED^ASCII( txt^str, txt^lgth, val ) THEN
...
ELSE
...

```

FIXED^ASCIIIX (extended address version)

Converts a FIXED (64 bit) value to an ASCII number. The number to be converted must be numeric and greater than or equal to zero. The PROC will return TRUE or FALSE to indicate whether or not the conversion was successful. An overflow condition will result in a return of FALSE.

Syntax:

<stat> := FIXED^ASCIIIX(<txtx> , <txt^lgth> , <val>)

where:

<stat>, INT, returns TRUE (-1) if the conversion was successful; otherwise, FALSE (0) is returned.

<txtx>, STRING.EXT:ref, contains the converted, variable length numeric text.

NOTE: It is up to the user to declare <txtx> long enough to receive the converted string.

<txt^lgth>, INT, contains the length of the string array.

<val>, FIXED(*), contains the FIXED number to be converted.

example:

```
STRING .ext txt^strx[ 0:17 ];
INT      txt^lgth;
FIXED    val;

txt^lgth  := 18;
val      := 23984098493983494f;
IF FIXED^ASCIIIX( txt^strx, txt^lgth, val ) THEN
    ...
ELSE
    ...
```

FIXED^DOLLAR

Converts a FIXED value to an ASCII string formatted with a "\$" sign, commas, and two decimal places. Leading zeros are suppressed.

Syntax:

CALL FIXED^DOLLAR(<amt> , <txt^lgth> , <txt>)

where:

<amt>, FIXED:ref,	contains the FIXED amt to be converted and formatted.
<txt^lgth>, INT,	contains the maximum number of bytes you want the display string to contain.
<txt>, STRING:ref,	contains the output string. If negative, the negative sign will be the right most character (i.e., -9999999999f becomes \$99,999,999.99-).

NOTE: It is up to the user to declare <txt> long enough to receive the converted string. If the <amt> contains more digits than <txt^lgth>, the most significant digits of the output might get truncated. The largest number that the type 'fixed' can represent is 19 digits long. The maximum number of characters that the string can contain is thus 27, including the "\$", the ",", the ".", and the "-" signs.

example:

```
STRING      .output^txt[ 0:26 ];
INT         txt^lgth;
FIXED       amt;

txt^lgth     := 27;
amt         := 23984098493983494f;
CALL FIXED^DOLLAR( amt, txt^lgth, output^txt )

result: output^txt    = "  $239,840,984,939,834.94 "
```

FIXED^DOLLAR^ASTERISK

Converts a FIXED value to an ASCII string formatted with a "\$", commas, and two decimal places. Leading zeroes are replaced with asterisks.

Syntax:

CALL FIXED^DOLLAR^ASTERISK(<amt> , <txt^lgth> , <txt>)

where:

<amt>, FIXED: ref	contains the value to be converted.
<txt^lgth>, INT: ref,	contains the maximum number of bytes you want the display string to contain.
<txt>, STRING: ref,	contains the output string. It will contain a \$ sign and 2 decimal places and a negative sign, if negative. Format \$*99,999.99-.

NOTE: It is up to the user to declare <txt> long enough to receive the converted string. If the <amt> contains more digits than <txt^lgth>, the most significant digits of the output might get truncated. The largest number that the type 'fixed' can represent is 19 digits long. The maximum number of characters that the string can contain is thus 27, including the "\$", the ",", the ".", and the "-" signs.

example:

```
STRING      .output^txt[ 0:26 ];
INT         edit^lgth;
FIXED       amt;

edit^lgth := 27;
amt      := 84164375678998927f;
CALL FIXED^DOLLAR^ASTERISK( amt, edit^lgth, output^txt)

result: output^txt = "$***841,643,756,789,989.27 "
```

FNUMIN

Converts an ASCII number into a fixed number.

Syntax:

<addr> := FNUMIN(<txt> , <fnum> , <fpoint> , <result>)

where:

<addr>, INT,	contains the address of the first byte not used in the conversion.
<txt>, STRING:ref,	contains the ASCII input string.
<fnum>, FIXED:ref,	contains the converted fixed number.
<fpoint>, INT,	contains the fixed point or scale value of the fixed number.
<result>, INT:ref,	contains the returned result flag: >= 0 means scale change mod 19 < 0 means conversion error

example:

STRING	.txt[0:5];
INT	.addr,
	.result,
	fpoint;
FIXED	fnum;
fpoint	:= 0;
txt	:= "123456";
@addr	:= FNUMIN(txt, fnum, fpoint, result);

FNUMOUT

Converts a fixed positive or negative number from binary to ASCII. The ASCII number will be zero suppressed, and either right or left justified, depending upon the flag parameter.

Syntax:

<addr> := FNUMOUT(<txt> , <fnum> , <scale> , <width> , <stat> , <flags>)

where:

<addr>, INT,	contains the address of the next unused byte.
<txt>, STRING:ref,	contains the ASCII converted number. Must be large enough for a decimal point and a negative sign.
<fnum>, FIXED(*),	contains the fixed number to convert.
<scale>, INT,	if present, contains the scale factor of fnum. Default is 0. If scale > 0, then a decimal point will be inserted according to the scale.
<width>, INT,	contains the width in bytes of the string buffer. Must be large enough for a decimal point and a negative sign.
<stat>, INT:ref,	if present, contains the overflow indication.
<flags>, INT,	if present, contains the justification. Default is zero. .<15> 1 = left justify, 0 = right justify .<14> 1 = delete trailing zeroes after the decimal point

example:

```
STRING      .txt [0:7 ];
INT          .addr,
            scale,
            width,
            stat,
            flags;
FIXED       fnum;

fnum        := -123456F;
width       := 8;
flag        := 0;
scale       := 2;
@addr       := FNUMOUT( txt, fnum, scale, width,stat, flags );

result      :      text    =      "-1234.56"
```

FORMATTED^FNUMOUT

Converts a fixed number into its ASCII representation, using a template similar to a COBOL PICTURE.

Syntax:

<stat> := FORMATTED^FNUMOUT(<outbuf> , <fnum> ,<template>)

where:

<stat>, INT,	contains TRUE (-1) if the conversion succeeds. Otherwise, FALSE (0).
<outbuf>, STRING:ref,	contains the converted number.
<fnum>, FIXED,	contains the fixed point number to be formatted.
<template>, STRING:ref,	contains a PICTURE type template (e.g. "\$999,999.99"). Must be delimited by a trailing null. Valid template symbols are: 9 - insert a digit, including zeroes Z - insert a digit, zero-space-fill * - insert a digit, zero-asterisk-fill . - insert a decimal point , - insert a comma \$ - insert a dollar sign \$\$\$ - floating dollar sign, zero-suppress S - insert sign (+ or -) - - insert minus sign if applicable

NOTE: This utility doesn't do strenuous error checking on the template. For example, "99,99,9,999.99" would be accepted, with the results just as you would expect.

example:

```
STRING      .ext  outbuf[ 0:9 ],
              .ext  template[ 0:9 ];
INT          stat;
FIXED        fnum;

fnum         := 123456F;
template     ':= ' "$99,999.99";
stat         := FORMATTED^FNUMOUT( outbuf,fnum,template );
```

result: outbuf = "\$01,234.56"

GRAPHIC^HEX

Converts ASCII "graphics" to an 8 byte packed binary field. The characters 0-9 and ":", ";", "<", "=", ">" and "?" are used to represent the hex- characters 0-9, A-F. Each 8 bits representing a character from "0" through "9" in <graphic^str> has its corresponding binary value packed into the next subsequent 4 bits in <binary^str>. The ":", ";", "<", "=", ">", and "?" in graphic string are converted to binary 10 through 15 respectively and are packed into the subsequent 4 bits in <binary^str>. For example,

<graphic^str> containing "1>" = %B0011000100111110

converts to

<binary^str> which contains %B00011110

Syntax:

<stat> := GRAPHIC^HEX(<graphic^str>, <binary^str>)

where:

<stat>, INT,	returns TRUE (non-zero) if the conversion was successful.
<graphic^str>, INT:ref,	contains the graphic string to be converted.
<binary^str>, INT:ref,	contains the converted code in packed binary format.

example:

```

INT          .graphic^str[ 0:7 ],
              .binary^str[ 0:3 ];

graphic^str  ':= "123456:=";

IF GRAPHIC^HEX( graphic^str, binary^str ) THEN
    ! successful
ELSE
    ! error

```

HEXCHAR^BINARY

Converts data representing hex digits in ASCII display form to internal binary form. Each 8 bits representing a character in <hex> has its corresponding binary value packed into the next subsequent 4 bits in <binary>. For example,

<hex> containing %B0011000101000101 = "1E"

converts to

<binary> containing %B00011110 (i.e., a binary 1 in 4 bits followed by a binary 14 in the next 4 bits)

Syntax:

<stat> := HEXCHAR^BINARY(<hex> , <lgth> , <binary>)

where:

<stat>, INT,	returns FALSE (0) if <lgth> is not an even number or if <hex> contains characters other than 1-9 or A-F; otherwise, TRUE (-1) is returned.
<hex>, STRING:ref,	contains the hexadecimal string to be converted.
<lgth>, INT,	contains the length, in bytes, of the array to be converted. <lgth> must be an even number.
<binary>, STRING:ref,	contains the result of the hex to binary conversion.

example:

```
STRING    .hex^str[ 0:15 ];
INT        hex^lgth;
STRING     .binary^str[ 0:7 ];

hex^str     := "0123456789ABCDEF";
hex^lgth    := 16;
IF HEXCHAR^BINARY( hex^str, hex^lgth, binary^str ) THEN
    ! successful
ELSE
    ! error
```

HEXCHAR^BINARYX (extended address version)

Converts data representing hex digits in ASCII display form to internal binary form. Each 8 bits representing a character in <hex> has its corresponding binary value packed into the next subsequent 4 bits in <binary>. For example,

<hex> containing %B0011000101000101 = "1E"

converts to

<binary> containing %B00011110 (i.e., a binary 1 in 4 bits followed by a binary 14 in the next 4 bits)

Syntax:

<stat> := HEXCHAR^BINARYX(<hexx> , <lgth> , <binaryx>)

where:

<stat>, INT,	returns FALSE (0) if <lgth> is not an even number or if <hex> contains characters other than 1-9 or A-F; otherwise, TRUE (-1) is returned.
<hexx>, STRING.EXT:ref,	contains the hexadecimal string to be converted.
<lgth>, INT,	contains the length, in bytes, of the array to be converted. <lgth> must be an even number.
<binaryx>, STRING.EXT:ref,	contains the result of the hex to binary conversion.

example:

```
STRING .ext hex^strx[ 0:15 ];
INT      hex^lgth;
STRING .ext binary^strx[ 0:7 ];

hex^strx    := "0123456789ABCDEF";
hex^lgth    := 16;
IF HEXCHAR^BINARYX( hex^strx, hex^len, binary^strx ) THEN
    ! successful
ELSE
    ! error
```

HEXCHAR^INTEGER

Converts a single hex character value ("0"-"F") into its integer equivalent (0-15). If the hex character to be converted is invalid, the PROC returns FALSE.

Syntax:

<stat> := HEXCHAR^INTEGER(<hexval> , <intval>)

where:

<stat>, INT,	returns FALSE (0) if the hex character is invalid and returns TRUE (-1) if the conversion is successful.
<hexval>, STRING:ref,	is the single hex character to be converted to its integer equivalent.
<intval>, INT:ref,	contains the result of the conversion from the hex character to its integer equivalent.

example:

INT	stat,
	intval;
STRING	hexval;
hexval	:= "A";
stat	:= HEXCHAR^INTEGER(hexval, intval);

INTEGER^ASCII

Converts an INT value to an ASCII number. The number to be converted must be numeric and greater than or equal to zero. The PROC will return TRUE or FALSE to indicate whether or not the conversion was successful. An overflow condition results in a return of FALSE.

Syntax:

<stat> := INTEGER^ASCII(<txt> , <txt^lgth> , <val>)

where:

<stat>, INT, returns TRUE (-1) if the conversion was successful; otherwise, FALSE (0) is returned.

<txt>, STRING:ref, contains the converted, variable length numeric text.

NOTE: It is up to the user to declare <txt> long enough to receive the converted string.

<txt^lgth>, INT, contains the length of the string array.

<val>, INT, contains the INT number to be converted.

example:

```

STRING  .txt^str[ 0:2 ];
INT      txt^lgth,
          val;

txt^lgth  := 3;
val       := 12;
IF INTEGER^ASCII( txt^str, txt^lgth, val ) THEN
    ! successful
ELSE
    ! error
  
```

INTEGER^ASCIIIX (extended address version)

Converts an INT value to an ASCII number. The number to be converted must be numeric and greater than or equal to zero. The PROC will return TRUE or FALSE to indicate whether or not the conversion was successful. An overflow condition results in a return of FALSE.

Syntax:

<stat> := INTEGER^ASCIIIX(<txt> , <txt^lgth> , <val>)

where:

<stat>, INT, returns TRUE (-1) if the conversion was successful; otherwise, FALSE (0) is returned.

<txt>, STRING.EXT:ref, contains the converted, variable length numeric text.

NOTE: It is up to the user to declare <txt> long enough to receive the converted string.

<txt^lgth>, INT, contains the length of the string array.

<val>, INT, contains the INT number to be converted.

example:

```
STRING      .ext txt^strx[ 0:2 ];
INT          txt^lgth,
             val;

txt^lgth     := 3;
val          := 12;
IF INTEGER^ASCIIIX( txt^strx, txt^lgth, val ) THEN
    ! successful
ELSE
    ! error
```

INTEGER^HEXCHAR

Converts an integer value (0-15) into its hex character equivalent ("0"- "F"). If the integer to be converted is invalid, the PROC returns FALSE.

Syntax:

<stat> := INTEGER^HEXCHAR(<hexval> , <intval>)

where:

<stat>, INT,	returns FALSE (0) if the integer value is invalid and returns TRUE (-1) if the conversion is successful.
<hexval>, STRING:ref,	contains the result of the conversion from an integer value to its hex character equivalent.
<intval>, INT:val,	is the integer value to be converted to its hex character equivalent.

example:

```

INT      stat,
         intval;
STRING   hexval;

intval   := 15;
stat     := INTEGER^HEXCHAR( hexval, intval );
  
```

SET^PARITY

Sets the parity (leftmost) bit on each character in the supplied buffer so that overall character parity becomes even or odd as requested. Note that none of the parameters has its value changed; only the supplied buffer is modified.

Syntax:

CALL SET^PARITY(eptr, lgth, even);

where:

<eptr>, STRING.EXT;ref, is an extended string address pointer to the caller's buffer

<lgth>, INT:val, is the effective length of the caller's buffer in bytes

<even>, INT:val, contains the type of parity requested:
-1 = TRUE, for even parity
0 = FALSE, for odd parity

example:

```
INT          lgth          := 120,
              even          := -1,      ! true
              not^even      := 0;
STRING       .buf^s[ 0:lgth -1 ],
              .ext eptr;
```

```
@eptr        := $XADR( buf^s );
```

```
! sets even parity
CALL SET^PARITY( eptr, lgth, even );
```

```
! sets odd parity
CALL SET^PARITY( eptr, lgth, not^even );
```

TRANSLATE

Used to translate a string from one character set to another:

- 1) ASCII to EBCDIC,
- 2) EBCDIC to ASCII,
- 3) ASCII to DATAPRODUCTS 2230 line printer subset.

Syntax:

CALL TRANSLATE(<src^str> , <lgth> , <translation>)

where:

<src^str>, STRING:ref, initially contains the variable length string to be translated. After execution of the PROC, contains the translated string.

<lgth>, INT, contains the length of the string array.

<translation>, INT, contains the type of translation requested:

0 = ASCII to EBCDIC

1 = EBCDIC to ASCII

2 = ASCII to Dataproducts 2230 line printer subset

example:

```
LITERAL  ascii^to^ebcdic^l      = 0;
STRING   .txt^str[ 0:15 ];
INT       lgth,
          translate^typ;
```

```
txt^str   := "translate text";
lgth      := 14;
translate^typ := ascii^to^ebcdic^l;
CALL TRANSLATE( txt^str, lgth, translate^typ );
```

TRANSLATEX (extended address version)

Used to translate a string from one character set to another:

- 1) ASCII to EBCDIC,
- 2) EBCDIC to ASCII,
- 3) ASCII to DATAPRODUCTS 2230 line printer subset.

Syntax:

CALL TRANSLATEX(<src^strx> , <lgth> , <translation>)

where:

<src^strx>, STRING.EXT:ref, initially contains the variable length string to be translated. After execution of the PROC, contains the translated string.

<lgth>, INT, contains the length of the string array.

<translation>, INT, contains the type of translation requested:

0 = ASCII to EBCDIC

1 = EBCDIC to ASCII

2 = ASCII to Dataproducts 2230 line printer subset

example:

```
LITERAL      ascii^to^ebcdic^l      = 0;
STRING .ext txt^strx[ 0:15 ];
INT          lgth,
            translate^typ;

txt^strx      := "translate text";
lgth          := 14;
translate^typ := ascii^to^ebcdic^l;
CALL TRANSLATEX( txt^strx, lgth, translate^typ );
```

ZDNUMIN

Converts an ASCII string to a signed double word. It performs the same function as Tandem's NUMIN procedure except that the ASCII number is converted to an INT(32) instead of an integer.

Syntax:

<addr> := ZDNUMIN(<str> , <dnum> , <base> , <stat>)

where:

<addr>, INT,	contains the byte address of the first character in the string that wasn't useful in the conversion.
<str>, STRING:ref,	contains the ASCII number to be converted.
<dnum>, INT(32):ref,	contains the converted number in binary.
<base>, INT,	is the number base for the resulting conversion. Valid numbers are 2 through 10.
<stat>, INT:ref,	returns the status of the conversion: -1 string does not start with "+", "-", or "%" 0 good number 1 overflow or illegal value

example:

STRING	.ascii^num[0:2];
INT	.addr, base, stat;
INT(32)	dnum;
ascii^num	:= "123";
base	:= 10;
@addr	:= ZDNUMIN(ascii^num, dnum, base, stat);

ZDNUMOUT

Converts a signed double word to an ASCII string. It performs the same function as Tandem's NUMOUT procedure except that an INT(32) number is converted to an ASCII string.

Syntax:

<addr> := ZDNUMOUT(<strng> , <dnum> , <base> , <width>)

where:

<addr>, INT,	contains the byte address of the first character in the string that wasn't useful in the conversion.
<strng>, STRING:ref,	contains the converted ASCII number.
<dnum>, INT(32),	contains the number in binary to be converted.
<base>, INT,	is the number base for the resulting conversion. Valid numbers are 2 through 10.
<width>, INT,	if present, will put the result in the buffer right justified. If not present, the result will be put in the buffer left justified.

example:

```
STRING      .ascii^num[ 0:3 ];
INT          .addr,
             base,
             width;
INT(32)      dnum;

dnum         := 123d;
base         := 10;
width        := 4;
@addr        := ZDNUMOUT( ascii^num, dnum, base, width );

result: ascii^num = " 123"
```

ZNUMOUT

Converts an unsigned integer value to its ASCII equivalent. If the width is passed then the result is right justified. Otherwise it returned left justified.

Syntax:

<addr> := ZNUMOUT(<str> , <num> , <base> , <width>)

where:

<addr>, INT,	contains the address of the next available byte in the buffer.
<str>, STRING:ref,	contains the converted number.
<num>, INT,	contains the integer to be converted.
<base>, INT,	is the number base for the resulting conversion. Valid numbers are 2 through 10.
<width>, INT,	if present, will put the result in the buffer right justified. If not present, the result will be put in the buffer left justified.

example:

```

STRING  .ascii^num[ 0:3 ];
INT      .addr,
          num,
          base,
          width;

num      := 123
base     := 10;
width    := 4;
@addr    := ZNUMOUT( ascii^num, num, base, width );

```

result: ascii^num = " 123"

ACI Worldwide Inc.

Section 10

Manipulation

String Manipulation

CONVERT^FIELD^JUSTIFICATION

Converts a field that is right justified, zero filled on the left to a field that is left justified, blank filled on the right and vice versa.

Syntax:

CALL CONVERT^FIELD^JUSTIFICATION(<rt^field> , <rt^field^lgth> ,
<lf^field> , <lf^field^lgth> ,
<conv^dir>);

where:

<rt^field>, STRING:ref,	contains the right justified, zero filled text.
<rt^field^lgth>, INT:value,	contains the length of <rt^field>.
<lf^field>, STRING:ref,	contains the left justified, blank filled text.
<lf^field^lgth>, INT:value,	contains the length of <lf^field>.
<conv^dir>, INT:value,	indicates the type of conversion to perform: 0 = convert a right justified, zero filled field to a left justified, blank filled field. 1 = convert a left justified, blank filled field to a right justified, zero filled field.

example:

LITERAL	right^to^left^l	= 0,
	left^to^right^l	= 1;

STRING	.rt^field[0:10],
	.lf^field[0:10];

rt^field	':=' "0000justify";
CALL convert^field^justification(	rt^field, 11, lf^field, 11,
	right^to^left^l);

CONVERT^STATE^CODE

Converts a two digit state code to a two letter state code and back. The state table built is based on fips processing standards publication 6-3.

Syntax:

<stat> := CONVERT^STATE^CODE(<scde> , <alpha> , <direction>)

where:

<stat>, INT,	contains TRUE(-1) if the conversion was successful. Otherwise, FALSE (0).
<scde>, STRING:ref,	contains a two digit representation of a state code (i.e. "01" for "AL", "02" for "AK" etc.).
<alpha>, STRING:ref,	contains a two letter representation for the state.
<direction>, INT,	contains the direction for the conversion to work. 1 convert from number to letter code 0 convert from letter code to number

example:

```
LITERAL    num^to^letter^cde^l    = 1,  
           letter^to^num^cde^l    = 0;  
  
STRING .ext scde[ 0:1 ],  
       .ext alpha[ 0:1 ];  
  
scde      ' := "02";  
CALL CONVERT^STATE^CODE( scde, alpha,  
                        num^to^letter^cde^l );
```

DELETE^EXTRA^BLANKS

Removes extra blanks from a string buffer. 'Extra blanks' are defined as leading blanks, trailing blanks, or any occurrence of two or more consecutive blanks (which are reduced to one blank). Blanks which occur within quotes are not affected (quotes within a string are denoted by two consecutive quotation marks).

Syntax:

CALL DELETE^EXTRA^BLANKS(<str> , <lgth>)

where:

<str>, STRING:ref, contains the string to be modified and the results of the conversion.

<lgth>, INT:ref, contains the number of bytes in the old string buffer and the number of bytes in the new buffer.

example:

```
STRING      .str[ 0:21 ];
INT          lgth;

lgth         := 22;
str          := " 123 " ab "hello";
CALL DELETE^EXTRA^BLANKS( str, lgth );

result: str   = "123 " ab "hello"
        lgth  = 17
```

FIELD^FINDER

Given a string of data and an offset from the data string where the search is to begin, returns the pointer to a data string containing either alpha or numeric data. The search starting position is the first byte at or after the offset position which contains alpha or numeric data. The search is discontinued when a data type other than that specified in the search starting position is encountered. The string array to be searched must be terminated by a null value.

Syntax:

<addr> := FIELD^FINDER(<in> , <typ> , <ofst^or^lgth>)

where:

<addr>, STRING,	contains the address of the search starting position. If <in> does not contain either alpha or numeric data at or after the offset position, then <addr> contains a zero.
<in>, STRING:ref,	pointer to the data string to be searched. This string must be delimited (ended) by a null value.
<typ>, INT:ref,	contains the data type of the returned string. The data type is determined by the data type in the search starting position. A "0" is returned when <in> does not contain either alpha or numeric data at or after the offset position. 0 - non-alpha/non-numeric data 1 - numeric data (0-9) 2 - alpha data (a-z/A-Z)
<ofst^or^lgth>, INT:ref,	on entry, contains the offset from the <in> data string where search is to begin. Offset 0 is the beginning of the string. Upon return, it contains the length of the "found" data string. A -1 is returned when <in> does not contain either alpha or numeric data at or after the offset position.

example:

```

STRING    .str^addr,
           .txt[ 0:10 ];
INT        typ,
           lgth;

txt        :='[ "hello 123",0 ];
lgth       := 1;
@str^addr  := FIELD^FINDER( txt, typ, lgth );

result:    @str^addr = 1 (points to the "e" of "hello")
           txt      = "hello 123"
           lgth     = 4
           typ      = 2

```

FIND^STRING

Scans a string array for the occurrence of a specified string. FIND^STRING returns zero if the search string was not found. If found, FIND^STRING returns the address of the occurrence.

Syntax:

<addr> := FIND^STRING(<array> , <search^str> , <str^lgth^or^ofst>)

where:

<addr>, INT, contains the address of the "found" string. If the search string was not found, <addr> contains 0.

NOTE: DO NOT USE INDIRECT ADDRESSING. UNPREDICTABLE RESULTS MIGHT OCCUR...

<array>, STRING:ref, is the array which is to be scanned for <search^str>. It must be terminated by a byte containing binary zeros.

<search^str>, STRING:ref, is the array containing the subject of the search.

<str^lgth^or^ofst>, INT:ref, on entry to the PROC, must contain the length of <search^str>. Upon return, this field will contain -1 if the search string was not found. Otherwise, it contains the offset from the beginning of <array> at which the string was found.

example:

```
STRING    .scan^array[ 0:20 ],
           .search^str[ 0:1 ];
INT        found,
           ofst^flg;

scan^array :=["this is my day ",0];
search^str := "my";
ofst^flg   := 2;
found      := FIND^STRING( scan^array, search^str, ofst^flg );

result:    found points to the "m" of "my"
           ofst^flg = 8
```

FORMAT^DECIMAL

Places a decimal point in the input string given the number of decimal places. If a decimal point already exists, the string is padded or truncated to that many decimal places. Returns FALSE (0) if non-numeric data was entered. For example, if the input string contains "12335" and the number of decimal places requested is 4, the result would be "1.2335", but if the string was "123.35" the result becomes "123.3500".

Syntax:

<stat> := FORMAT^DECIMAL(<input^str>, <input^lgth>, <output^str>, <output^lgth>, <num^dec^places>)

where:

<stat>, INT, return FALSE (0) if non-numeric input was entered.

<input^str>, STRING:ref, input string to be edited.

<input^lgth>, INT, length in bytes of input string. Length can be up to 131 bytes.

<output^str>, STRING:ref: edited string containing decimal point with number of input decimal places.

NOTE: It is up to the user to declare <output^str> long enough to receive the converted string.

<output^lgth>, INT, maximum length in bytes the output string can contain.

<num^dec^places>, INT, number of decimal places the output string is to contain.

example:

```
STRING      .input^str[ 0:7 ],
             .output^str[ 0:9 ];
INT         input^lgth,
            output^lgth,
            num^dec^places;

input^str    ':=' "12345678";
input^lgth   := 8;
output^lgth  := 10;
num^dec^places := 4;
IF FORMAT^DECIMAL( input^str, input^lgth, output^str,
                  output^lgth, num^dec^places ) THEN
    ! valid string
ELSE
    ! error occurred
```

LEFT^JUSTIFY

Moves a string array to the left. Blanks are suppressed and inserted at the right. For example:

the string array: " contents"
becomes: "contents "

Syntax:

CALL LEFT^JUSTIFY(<str^array>, <str^lgth>)

where:

<str^array>, STRING:ref, contains the input string array.

<str^lgth>, INT, length of input string in bytes. Allows for
string length to 131 characters.

example:

```
STRING     .str^array[ 0:9 ];  
INT        str^lgth;  
  
str^lgth    := 10;  
str^array   := "  12354";  
CALL LEFT^JUSTIFY( str^array, str^lgth );
```

MOD10^ADD

Used to perform mod-10 addition on two buffers which contain ASCII- numeric data. The two buffers are right-aligned and added mod-10, digit by digit.

Syntax:

<stat> := MOD10^ADD(<num1> , <num2> , <num1^lgth> , <num2^lgth> ,
<result>);

where:

<stat>, INT,	returns TRUE if input data was valid ASCII-numeric data and is at most 16 bytes long; otherwise, zero-fills result and returns FALSE.
<num1>, STRING:ref,	is a buffer containing ASCII-numerics.
<num2>, STRING:ref,	is a buffer containing ASCII-numerics.
<num1^lgth>, INT:val,	is the number of bytes in <num1> to add.
<num2^lgth>, INT:val,	is the number of bytes in <num2> to add.
<result>, STRING:ref,	is the buffer to store ASCII-numeric result of <num1> plus <num2>.

NOTE: Neither buffer may exceed 16 bytes. Also, note that the result will be <n> digits, where <n> equals \$max(<num1^lgth>, <num2^lgth>).

example:

```

STRING      .num1[ 0:15 ] := "56928",
              .num2[ 0:15 ] := "01234",
              .result[ 0:15 ];
INT         num1^lgth  := 5,
              num2^lgth := 5;
```

```
CALL MOD10^ADD( num1, num2, num1^lgth, num2^lgth, result);
```

```
result: result = "57152"
```

MOD10^ADDX (extended address version)

Used to perform mod-10 addition on two buffers which contain ASCII- numeric data. The two buffers are right-aligned and added mod-10, digit by digit.

Syntax:

<stat> := MOD10^ADDX(<num1x> , <num2x> , <num1^lgth> , <num2^lgth> ,
<resultx>);

where:

<stat>, INT, returns TRUE if input data was valid ASCII-
 numeric data and is at most 16 bytes long;
 otherwise, zero-fills result and returns
 FALSE.

<num1x>, STRING:ref, is a buffer containing ASCII-numerics.

<num2x>, STRING:ref, is a buffer containing ASCII-numerics.

<num1^lgth>, INT:val, is the number of bytes in <num1> to add.

<num2^lgth>, INT:val, is the number of bytes in <num2> to add.

<resultx>, STRING:ref, is the buffer to store ASCII-numeric result of
 <num1> plus <num2>.

NOTE: Neither buffer may exceed 16 bytes. Also, note that the result will be
<n> digits, where <n> equals \$max(<num1^lgth>, <num2^lgth>).

example:

```
STRING .ext num1x[ 0:15 ]      := "56928",  
      .ext num2x[ 0:15 ]      := "1234",  
      .ext resultx[ 0:15 ];  
INT     num1^lgth              := 5,  
      num2^lgth                := 4;
```

```
CALL MOD10^ADDX(num1x, num2x, num1^lgth, num2^lgth,resultx);
```

```
result: resultx = "57152"
```

MOD10^SUBTRACT

Used to perform mod-10 subtraction on two buffers which contain ASCII numeric data. The two buffers are right-aligned and subtracted mod-10, digit by digit.

Syntax:

<stat> := MOD10^SUBTRACT(<num1> , <num2> , <num1^lgth> ,
<num2^lgth> , <result>);

where:

<stat>, INT,	returns TRUE if input data was valid ASCII-numeric data and is at most 16 bytes long; otherwise, zero-fills result and returns FALSE.
<num1>, STRING:ref,	is a buffer containing ASCII-numerics.
<num2>, STRING:ref,	is a buffer containing ASCII-numerics.
<num1^lgth>, INT:val,	is the number of bytes in <num1> to subtract.
<num2^lgth>, INT:val,	is the number of bytes in <num2> to subtract.
<result>, STRING:ref,	is the buffer to store ASCII-numeric result of <num1> minus <num2>.

NOTE: Neither buffer may exceed 16 bytes. Also note that the result will be <n> digits, where <n> equals \$max(num1^lgth, num2^lgth).

example:

```

STRING      .num1[ 0:15 ]      := "56928",
              .num2[ 0:15 ]      := "1234",
              .result[ 0:15 ];
INT          num1^lgth          := 5,
              num2^lgth          := 4;
```

```

CALL MOD10^SUBTRACT( num1, num2, num1^lgth, num2^lgth,
                    result );
```

```

result: result = "55794"
```

MOD10^SUBTRACTX (extended address version)

Used to perform mod-10 subtraction on two buffers which contain ASCII- numeric data. The two buffers are right-aligned and subtracted mod-10, digit by digit.

Syntax:

<stat> := MOD10^SUBTRACTX(<num1x> , <num2x> , <num1^lgth> ,
<num2^lgth> , <resultx>);

where:

<stat>, INT, returns TRUE if input data was valid ASCII-
 numeric data and is at most 16 bytes long;
 otherwise, zero-fills result and returns
 FALSE.

<num1x>, STRING.EXT:ref, is a buffer containing ASCII-numerics.

<num2x>, STRING.EXT:ref, is a buffer containing ASCII-numerics.

<num1^lgth>, INT:val, is the number of bytes in <num1> to
 subtract.

<num2^lgth>, INT:val, is the number of bytes in <num2> to
 subtract.

<resultx>, STRING.EXT:ref, is the buffer to store ASCII-numeric result of
 <num1> minus <num2>.

NOTE: Neither buffer may exceed 16 bytes. Also note that the result will be
 <n> digits, where <n> equals \$max(num1^lgth, num2^lgth).

example:

```
STRING      .ext num1x[ 0:15 ] := "56928",
              .ext num2x[ 0:15 ] := "1234",
              .ext resultx[ 0:15 ];
INT          num1^lgth := 5,
              num2^lgth := 4;
```

```
CALL MOD10^SUBTRACTX( num1x, num2x, num1^lgth,
                      num2^lgth, resultx );
```

```
result: result = "55794"
```

PACK

Packs one string into another using the following packing algorithm:

<sub> <char> <count code>
 <sub> - is the ASCII sub character (%h1A)
 <char> - the character packed
 <count code> - specifies the number of consecutive <char>'s replaced
 by the pack sequence. It is related to the actual character
 count by: <count code> := <actual count> - 3 + "@" ("@"
 = %h40).

Syntax:

<stat> := PACK(<in^buf>, <in^lgth>, <out^buf>, <out^lgth>, <out^size>)

where:

<stat>, INT,	returns TRUE (non^zero) if successful, otherwise FALSE (0) is returned.
<in^buf>, STRING:ref,	contains the text to be packed.
<in^lgth>, INT,	is the length to be packed in bytes. Length must be at least the length of <in^buf> + 1.
<out^buf>, STRING:ref:<out^size>,	contains the packed string. Length must be as long as <out^size>.
<out^lgth>, INT:ref,	is the length of the packed string.
<out^size>, INT,	is <out^buf>'s maximum size in bytes.

example:

```
STRING      .in^buf[ 0:9 ],
             .out^buf[ 0:19 ];
INT         in^lgth,
            out^lgth,
            max^lgth;

in^lgth      := 11;
max^lgth     := 20;
IF PACK( in^buf, in^lgth, out^buf,out^lgth, max^lgth ) THEN
    ! successful pack
ELSE
    ! error condition
```

REMOVE^BLANKS

Removes blanks from a string array and left justifies the characters in the process.
For example:

" my data " becomes "mydata "

Syntax:

CALL REMOVE^BLANKS(<str^array>, <str^lgth>)

where:

<str^array>, STRING:ref, contains the input string array. Will contain
the edited string at return.

<str^lgth>, INT, length of input string in bytes. Allows for
string length up to 131 characters.

example:

```
STRING    .str^array[ 0:9 ];
INT       str^lgth;

str^array  ':=' " my data";
str^lgth   := 10;
CALL REMOVE^BLANKS( str^array, str^lgth );
```

REMOVE^NON^NUMERIC

Removes non-numerics from a string array. For example:

the character string "abc12cd34" becomes "1234 "

Syntax:

CALL REMOVE^NON^NUMERIC(<str^array>, <str^lgth>)

where:

<str^array>, STRING:ref,	contains the input string array.
<str^lgth>, INT:ref,	length of input string in bytes. Allows for string length to 131 characters. On return, <str^lgth> contains the number of numeric characters in <str^array>.

example:

```
STRING      .str^array[ 0:9 ];  
INT         str^lgth;  
  
str^array    := "123 my 890";  
str^lgth     := 10;  
CALL REMOVE^NON^NUMERIC( str^array, str^lgth );
```

RIGHT^JUSTIFY

Moves a string array to the right. Only rightmost blanks are suppressed and inserted at the left. For example:

the character string "123 abc " becomes " 123 abc"

Syntax:

CALL RIGHT^JUSTIFY(<str^array>, <str^lgth>)

where:

<str^array>, STRING:ref, contains the input string array.

<str^lgth>, INT, length of input string in bytes. Allows for
string length to 131 characters.

example:

```
STRING      .str^array[ 0:9 ];
INT          str^lgth;

str^array    ':=' "123abc ";
str^lgth     := 10;
CALL RIGHT^JUSTIFY( str^array, str^lgth );
```

STRLEN

Computes and returns the length of an input string. The string must be terminated by a null delimiter at the end, and any trailing blanks (preceding the null) will be ignored.

Syntax:

<lgth> := STRLEN(<str> , <max^lgth>)

where:

<lgth>, INT:ref,	contains the length of the string.
<str>, INT:ref,	contains the string whose length is desired.
<max^lgth>, INT, optional,	if passed, contains the maximum length of the string.

example:

STRING	.str^blk[0:9];
INT	lgth,
	max^lgth;
str^blk	:= "hello " & [0];
max^lgth	:= 10;
lgth	:= STRLEN(string, max^lgth);

STRLENX (extended address version)

Computes and returns the length of an input string. The string must be terminated by a null delimiter at the end, and any trailing blanks (preceding the null) will be ignored.

Syntax:

<lgth> := STRLENX(<strx> , <max^lgth>)

where:

<lgth>, INT:ref, contains the length of the string.

<strx>, INT:ref, contains the string whose length is desired.

<max^lgth>, INT, optional, if passed, contains the maximum length of the string.

example:

```
STRING .ext strx[ 0:9 ];  
INT      max^lgth,  
          lgth;  
  
strx      ':= "hello  " & [ 0 ];  
max^lgth  := 10;  
lgth      := STRLENX( strx, max^lgth );
```

SYM^NAME^LEN

Determines the length of a symbolic name, without the trailing blanks. A symbolic name can contain Alpha Numeric Characters plus '^' and '-'

Syntax:

<lgth> := SYM^NAME^LEN(<sym^name>)

where:

<lgth>, INT, contains the length of the symbolic name
(0-16).

<sym^name>, STRING.EXT:ref, contains the symbolic name up to 16 bytes

example:

```
INT      lgth;  
STRING   .sym^name[0:15]  
  
sym^name  ':= "PROC-NAME^LEN.*";  
lgth      :=SYM^NAME^LEN(sym^name);  
  
results:   lgth will be 13 after this call.
```

UNPACK

Unpacks one string into another using the following packing algorithm:

<sub> <char> <count code>
 <sub> - is the ASCII sub character (%h1A)
 <char> - the character packed
 <count code> - specifies the number of consecutive <char>'s replaced by the pack sequence. It is related to the actual character count by:
 <count code> := <actual count> - 3 + "@". ("@" = %h40)

Syntax:

<stat> := UNPACK(<in^buf>, <in^lgth>, <out^buf>, <out^lgth>, <out^size>)

where:

<stat>, INT,	returns TRUE (non^zero) if successful.
<in^buf>, STRING:ref,	contains the text to be unpacked.
<in^lgth>, INT,	is the length to be unpacked in bytes. Length must be at least 1 less than the length of <in^buf>.
<out^buf>, STRING:ref,	contains the unpacked string.
<out^lgth>, INT:ref,	is the length of the unpacked string.
<out^size>, INT,	is <out^buf>'s maximum size in bytes.

example:

```
STRING      .in^buf[ 0:9 ],
             .out^buf[ 0:19 ];
INT          in^lgth,
             out^lgth,
             max^lgth;

in^lgth      := 11;
max^lgth     := 20;
IF UNPACK( in^buf, in^lgth, out^buf,out^lgth, max^lgth ) THEN
    ! successful
ELSE
    ! error condition
```

VALID^DECIMAL^NUMBER

Checks for a valid decimal number and returns an index to the decimal point if it exists (e.g., if the input string contains "1234.789", the index would be returned as 4). Returns TRUE (-1) if the input string is a valid decimal number.

Syntax:

<stat> := VALID^DECIMAL^NUMBER(<str^array>, <str^lgth>, <decimal^idx>
)

where:

<stat>, INT,	returns TRUE (non-zero) if this is a valid decimal number.
<str^array>, STRING:ref,	contains the input string array.
<str^lgth>, INT,	length of input string in bytes.
<decimal^idx>, INT,	if present, contains an index to the decimal point if there is one. -1 is returned if this is a valid decimal number but contains no decimal points.

example:

```

STRING      .str^array[ 0:9 ];
INT         str^lgth,
            decimal^index;

str^array    := "1234567.00";
str^lgth     := 10;
IF VALID^DECIMAL^NUMBER( str^array, str^lgth,
                        decimal^index )THEN
    ! valid decimal number
ELSE
    ! invalid decimal number

```

ZERO^SUPPRESS

Replaces leading zeros with blanks. For example:

the character string "00001234.69" becomes " 1234.69"

Syntax:

CALL ZERO^SUPPRESS(<str^array>, <str^lgth>)

where:

<str^array>, STRING:ref, contains the input string array.

<str^lgth>, INT, length of input string in bytes.

example:

```
STRING    .str^array[ 0:9 ];  
INT       str^lgth;  
  
str^array  ':= "0000123.49";  
str^lgth   := 10;  
CALL ZERO^SUPPRESS( str^array, str^lgth );
```

Data Manipulation

These procedures allow you to work with memory above 32K. They have two major functions, to compare one area of memory to another, or to move one area of memory to another.

To simplify the use of these PROCs when using structs, source in the file \$<VOL>.ACIUTILS.UTILDEFS(DCF^DEFINES). This file contains a series of DEFINES as described on the following page.

These require copystruct and comparestruct sections from \$<VOL>.ACIUTILS.UTILEXTS

Examples are given using the following struct definitions:

```
STRUCT          my^crd;
  BEGIN
    STRUCT          mynam;
      BEGIN
        STRING          bytes[ 0:29 ];
      END;
    INT          crd^id;
    STRING          ssn[ 0:8 ];
  END;
```

```
STRUCT          emp^crd;
  BEGIN
    STRUCT          emp^nam;
      BEGIN
        STRING          bytes[ 0:29 ];
      END;
    INT          comp^id;
    INT(32)          emp^date;
    STRING          ssn[ 0:8 ];
    STRUCT          address;
      BEGIN
        STRING          bytes[ 0:20 ];
      END;
  END;
```

DEFINEs

NOTE: The parameters <from> and <to>, represent the base address of the struct, while <fromfield> and <tofield> are the data items byte offset.

CFIELDLITNU(lit, litlen, to, field) - moves a literal to a field within a struct. No upshifting is done. Example:

```
CFIELDLITNU( "JOHN DOE", 8, mycrd, mynam )
```

CFIELDLIT(lit, litlen, to, field) - upshifts the literal and moves it to a field in a struct.

CFIELDLIT1(lit, litlen, to, field, fill) - moves a literal to a field and pads with fill character. Example:

```
CFIELDLIT1( "JOHN DOE", 8, mycrd, mynam, " " )
```

CFIELDNU(from, to, field) - moves one like field to another. No upshifting is done. Example:

```
CFIELDNU( mycrd, emp^crd, ssn )
```

CFIELD(from, to, field) - upshifts the field to be moved and moves it to a like field.

COMPFIELDNU(from, to, field) - compares two like fields and returns TRUE if equal. Does not upshift to do the compare.

Example:

```
IF COMPFIELDNU( mycrd, emp^crd, ssn ) THEN
    ...
ELSE
    ...
```

COMPFIELD(from, to, field) - compares two like fields in uppercase.

CSTRUCTNU(from, fromfield, to, tofield) - copies one data area to another. Does not upshift. Example:

```
CSTRUCTNU( mycrd, mynam, emp^crd, emp^nam )
```

CSTRUCT(from, fromfield, to, tofield) - upshifts the field to be moved and then moves it to another data area.

COMPSTRUCTNU(from, fromfield, to, tofield) - compares one data area to another.
Returns TRUE if they are equal. Does not upshift to do the
compare.

Example:

```
IF COMPSTRUCTNU( mycrd, mynam, emp^crd, emp^nam ) THEN
... !equal condition
ELSE
...
```

COMPSTRUCT(from, fromfield, to, tofield) - compares one data area to another in
uppercase.

UPSHIFT^FIELD

Used to upshift an area in memory.

Syntax:

CALL UPSHIFT^FIELD(<base>, <ofst>, <lgth>)

where:

<base>, INT:ref,	is the word address of the area to be upshifted.
<ofst>, INT,	contains the number of bytes from <base> to the data item being upshifted.
<lgth>, INT,	number of bytes the data item contains. Length must be > 0 and less than 4096.

example:

```
INT          .fld[ 0:9 ],
              ofst,
              fld^lgth;

ofst          := 0;
fld^lgth      := 20;
CALL UPSHIFT^FIELD( @fld, ofst, fld^lgth );
```

NOTE: This procedure does not upshift for <lgth> if a field terminator is contained in the area to be upshifted. Commas and possibly other such characters will stop this procedure.

Example: abcd,ef becomes ABCD

It is recommended that the NSK call to the SHIFTSTRING procedure be used INSTEAD of upshift^field.

Section 11

Timer Routines

Overview

There are two sets of timer routines that provide an easy method of working with interval timers.

The User Data Segment Timer Routines work on timers allocated in the User Data Segment (the lower and upper 64K bytes of global memory).

The Extended Memory Segment Timer Routines take advantage of extended memory segments. Due to the memory space available in Extended Memory, these routines support substantially more timers. Over six million timers could be supported in one extended memory segment, however using large numbers of timers may produce performance burdens on the system.

There is a significant difference in the timestamp used by each set of timer routines. The NSK system supports a proprietary 48-bit timestamp, and a 64-bit Julian Timestamp. The User Data Segment Timer Routines utilize the 48-bit timestamp, while the Extended Memory Segment Timer Routines use the Julian Timestamp.

The NSK system changes the 48-bit timestamp value during Daylight Savings Time. The Julian Timestamp is not altered for Daylight Savings Time. Consequently, the User Data Segment Timers are affected by the Daylight Savings Time change, whereas the Extended Memory Segment Timer Routines are not.

These two sets are not interchangeable.

User Data Segment Timer Routines

The procedures DELETE^THIS^TIMER, FIND^SPECIFIC^TIMER, NEXT^EXPIRE^TIME, TIMER^DELETE, TIMER^DELETE^1, TIMER^EXPIRED, TIMER^INIT, and TIMER^INSERT are used when the timer data structures reside within the data area of the User Data Segment. This includes the global memory in the 64K byte Data Stack, and the upper 64K bytes of global memory.

The structures for the User Data Segment Timer Routines are defined as follows:

```
STRUCT          timer^def( * );
  BEGIN
    INT          next;
    FIXED        exptime;
    INT          et[ 0:3 ] = exptime;
    INT          typ;
    INT          subtyp;
    INT          userbuf;
    STRING       s^userbuf = userbuf;
  END;

STRUCT          timer^control^block^def( * );
  BEGIN
    INT          first^timer^addr,
                free^timer^addr,
                num^of^timers,
                timer^depth,
                max^timer^depth;
  END;
```

The userbuf element of the timer^def structure locates a word address at the end of each timer where user data can be placed. When timers are initialized, the word size of the largest user data area is specified so that the timers are allocated in memory with space at the end of each timer for a data structure of the developer's choice. The address of a particular timer is returned by each of the following procs: FIND^SPECIFIC^TIMER, TIMER^EXPIRED, and TIMER^INSERT. This address can be used to locate the beginning of the userdata areas for application specific purposes.

The user should modify no other area of the timer structure.

To use these procs do the following:

- 1) Source in =ACIUTILS_UTILDEFS(timer^structs) in your program where struct templates are defined.
- 2) Declare timer control block (using TIMER^CONTROL^BLOCK^DEF).

- 3) Declare the timer table large enough to contain the maximum number of timers, plus any user data associated with timer elements.
- 4) CALL TIMER^INIT before calling any other TIMER procedures.

Example:

```
LITERAL    timer^user^buf^lgth^l  = 4, ! size in words of userbuf
           num^of^timers^l         = 100,
           timer^entry^size^l      = $len( timer^def ) / 2 +
                                   timer^user^buf^lgth^l - 1,
           timer^tbl^limit^l       = num^of^timers^l * timer^entry^size^l - 1;
```

```
?source = aciutils_utildefs( timer^structs )
```

```
STRUCT     tcb( timer^control^block^def );
INT        . timer^tbl[ 0:timer^tbl^limit ];
```

```
?source = aciutils_utiltexts( timer^init, timer^delete, ... )
```

```
PROC main^proc MAIN;
BEGIN
```

```
LITERAL    req^timer^l             = 0,
           saf^timer^l             = 1,
           wft^timer^l             = 2,
           fetimedout              = 40;
```

```
INT        error,
           file,
           .timer( timer^def );
```

```
INT(32)    timer^wait;
```

```
CALL timer^init( tcb, timer^tbl, num^of^timers^l, timer^user^buf^lgth^l );
CALL timer^insert( tcb, 100d, saf^timer^l );
CALL timer^insert( tcb, 55d, wft^timer^l );
CALL timer^insert( tcb, 300d, req^timer^l );
```

```
file       := -1;
timer^wait := next^expire^time( tcb );
```

```
CALL awaitio( file , , , , timer^wait );
```

```
IF <> THEN
  BEGIN
    CALL fileinfo( file, error );
    IF error = fetimedout THEN
      BEGIN
        @timer := timer^expired( tcb, true );
        WHILE @timer <> 0 DO
```

Timer Routines

User Data Segment Timer Routines

```
        BEGIN
        CALL process^timeout( timer );
        @timer := timer^expired( tcb, true );
        END;
    END
ELSE
    ! handle file error, other than timeout ...
    END
ELSE
    ! handle normal i/o completion ...
```

DELETE^THIS^TIMER

Designed to be used after a call to timer^expired or find^specific^timer, when the location of the timer that is to be deleted is known.

Syntax:

<status> := DELETE^THIS^TIMER(<tcb>, <timer>);

where:

<status>, INT,	returns true if the timer was deleted, and false otherwise.
<tcb>,INT:ref,	contains the timer control block.
<timer>, INT:ref,	a pointer returned by one of the following: - find^specific^timer - timer^insert - timer^expired

example:

```
@timer := find^specific^timer( tcb, type^1 );
IF not DELETE^THIS^TIMER( tcb, timer ) THEN
  BEGIN
    CALL log^message( timer^not^found );
  END;
```

FIND^SPECIFIC^TIMER

Used to find the first timer (i.e., the one with the earliest expiration time), which matches the calling parameters.

Syntax:

<timer^addr> := FIND^SPECIFIC^TIMER(<tcb>, <typ>, <subtyp>, <userbuf>, <userbufsize>)

where:

<timer^addr>, INT,	contains the address of the found timer. Returns FALSE(0) if no timer was found.
<tcb>,INT:ref,	contains the timer control block.
<typ>, INT,	is the type of timer to be returned. Type is an integer value specified when the timer is inserted.
<subtyp>, INT,	is the subtype of timer to be returned. Subtype is an integer value specified when the timer is inserted.
<userbuf>, INT:ref,	only timers with <timer.userbuf> equal to userbuf for the length of <userbufsize> will be returned.
<userbufsize>, INT,	word length of the comparison between <userbuf> parameter and <timer.userbuf> data area that must match.

This is a variable type procedure. Only <tcb> is required. The rest of the parameters are optional. If no additional parameters are specified, no timer will be located.

If <userbuf> or <userbufsize> are specified, both are required.

example:

@timer := FIND^SPECIFIC^TIMER(tcb, typ, subtyp, userbuf, userbufsize);

NEXT^EXPIRE^TIME

Returns the time interval, in hundredths of a second, which must pass before the next timer expires.

Syntax:

<wait^time> := NEXT^EXPIRE^TIME(<tcb>)

where:

<wait^time>, INT(32), contains 0d if a timer has already expired. If there are no timers in the timer list, -1d is returned.

The value that is returned may be passed directly as the <wait time> parameter in the AWAITIO[X] procedure.

<tcb>, INT:ref, contains the timer control block.

example:

```
file := -1;  
wait^time := NEXT^EXPIRE^TIME( tcb );  
CALL awaitio( file, , , timer^wait );
```

TIMER^DELETE

Used to delete one or more timers from the timer list.

Syntax:

<error> := TIMER^DELETE(<tcb>, <typ>, <subtyp>, <userbuf>, <userbufsize>)

where:

<error>, INT	contains false (0) if any of the required parameters are missing, otherwise returns true (-1).
<tcb>, INT:ref,	contains the timer control block.
<typ>, INT,	is the type of timer to be deleted. Type is an integer value specified when the timer is inserted.
<subtyp>, INT,	is the subtype of timer to be deleted. Subtype is an integer value specified when the timer is inserted.
<userbuf>, INT:ref,	only timers with <timer.userbuf> equal to userbuf for the length of <userbufsize> will be deleted.
<userbufsize>, INT,	word length of the comparison between <userbuf> parameter and <timer.userbuf> data area that must match.

This is a variable type procedure. Only <tcb> is required. The rest of the parameters are optional. If no additional parameters are specified, no timer will be deleted.

If <userbuf> or <userbufsize> are specified, both are required.

example:

```
CALL TIMER^DELETE( tcb, typ, subtyp, userbuf, userbufsize );
```

TIMER^DELETE^1

Used to delete one or more timers from the timer list and also gives the user two additional options concerning the deleted timers. The last parameter is for a user proc that allows the user to process the data in the timer and/or the user proc can specify whether to continue searching the timer table.

Syntax:

<error> := TIMER^DELETE^1(<tcb>, <typ>, <subtyp>, <userbuf>, <userbufsize>, <user proc>);

where:

<error>, INT,	contains true (-1) only if all the required parameters are there and if all parameters are valid.
<tcb>, INT:ref,	contains the timer control block.
<typ>, INT,	is the type of timer to be deleted. Type is an integer value specified when the timer is inserted.
<subtyp>, INT,	is the subtype of timer to be deleted. Subtype is an integer value specified when the timer is inserted.
<userbuf>, INT:ref,	only timers with <timer.userbuf> equal to userbuf for the length of <userbufsize> will be deleted.
<userbufsize>, INT,	word length of the comparison between <userbuf> parameter and <timer.userbuf> data area that must match.
<user proc>, INT,	proc to be called with each matching timer found. Must be declared INT proc userproc(timer); INT .timer(timer^def);

This is a variable type procedure. Only <tcb> is required. The rest of the parameters are optional. If no additional parameters are specified, no timer will be deleted.

If <userbuf> or <userbufsize> are specified, both are required.

The <user proc> should return true (-1) if the search for more timers is to continue. It can be used to perform cleanup tasks when a timer is deleted. Such tasks might include deallocating memory that was reserved by the given timer.

example:

```
CALL TIMER^DELETE^1( tcb, typ, subtyp, userbuf, userbufsize, userproc );
```

TIMER^EXPIRED

Checks if any timer has expired. Only the first timer is checked. If more than one timer has expired, TIMER^EXPIRED must be called once for each timer that has expired, until false(0) is returned.

Syntax:

<timer^addr> := TIMER^EXPIRED(<tcb>, <delete>)

where:

<timer^addr>, INT,	will contain the address of the timer if it expired. Otherwise, FALSE(0) is returned.
<tcb>, INT:ref,	contains the timer control block.
<delete>, INT,	is used to delete a timer that has expired. If omitted, the timer will not be deleted.

example:

```
DO
  BEGIN
    @timer := TIMER^EXPIRED( tcb, true );
    IF @timer <> 0 THEN
      BEGIN
        ! process expired timer
      END;
    END
  UNTIL @timer = 0;
```

TIMER^INIT

Used to initialize the timer table. This proc must be called before using any of the other timer procedures.

Syntax:

CALL TIMER^INIT(<tcb>, <timer^tbl> , <num^timers> , <user^buf^size>)

where:

<tcb>, INT:ref,	contains the timer control block.
<timer^tbl>, INT:ref,	is the timer table declared in the globals.
<num^timers>, INT,	is the maximum number of timers that timer^tbl can hold.
<user^buf^size>, INT,	is the word length of the user buffer area of the timer entry. If no user buffer is needed, the value of the parameter may be 0.

The user data area of the timers does not have to match for each timer type and subtype. If multiple layouts are to be used, specify the word size of the largest user data structure. The size specified here will be allocated at the end of every timer structure to provide this user data area to each timer.

example:

```
CALL TIMER^INIT( tcb, timer^tbl, $occurs( timer^tbl ),  
                user^buf^size );
```

TIMER^INSERT

Used to insert an interval timer in the active timer list.

Syntax:

<timer^addr> := TIMER^INSERT(<tcb>, <delay^time>, <typ>, <subtyp>)

where:

<timer^addr>, INT,	contains the address of the new timer. Returns FALSE (0) if no unused timers were available.
<tcb>, INT:ref,	contains the timer control block.
<delay^time>, INT (32),	is the time interval, in hundredths of a second, for the added timer.
<typ>, INT,	is the type of timer to be inserted.
<subtyp>, INT,	is the subtype of timer to be inserted.

example:

```
@timer := TIMER^INSERT( tcb, delay^time, typ, subtyp );
timer.userbuf := some^value;
OR
@user^area := @timer.userbuf;
! manipulate user^area structure
```

Extended Memory Segment Timer Routines

The procedures DELETE^THIS^TIMERX, FIND^SPECIFIC^TIMERX, NEXT^EXPIRE^TIMEX, TIMER^ALLOCATEX, TIMER^DELETEx, TIMER^EXPIREDX, TIMER^INITX, and TIMER^INSERTX are used when the timer data structures reside with an extended memory segment. It doesn't matter whether it is a selectable segment or a flat segment.

The structures for the Extended Memory Segment Timer Routines are defined as follows:

```
STRUCT          timerx^def( * );
  BEGIN
    INT(32)      next;                ! address of next timer in the list
    FIXED        exptime;            ! expiration timestamp
    INT          type;                ! user defined timer type
    INT          subtype;            ! user defined timer subtype
    INT          userbuf;            ! first word of variable length
                                ! user buffer
    STRING       s^userbuf = userbuf; ! string userbuf symbol
  END;
```

```
STRUCT          timer^control^blockx^def ( * );
  BEGIN
    INT(32)      first^timer^addr;
    INT(32)      free^timer^addr;
    INT(32)      number^of^timers;
    INT(32)      timer^depth;
    INT(32)      max^timer^depth;
    INT(32)      number^allocated;
    INT(32)      alloc^block^size;
    INT          timer^size;
  END;
```

The userbuf element of the timerx^def structure locates a word address at the end of each timer where user data can be placed. When timers are initialized, the word size of the largest user data area is specified so that the timers are allocated in memory with space at the end of each timer for a data structure of the developer's choice.

The address of a particular timer is returned by each of the following procs: FIND^SPECIFIC^TIMERX, TIMER^EXPIREDX, and TIMER^INSERTX. This address can be used to locate the beginning of the userdata areas for application specific purposes.

The user should modify no other area of the timer structure.

To use these procs do the following:

- 1) Source in =ACIUTILS_UTILDEFS(timer^structs) in your program where struct templates are defined.
- 2) Declare timer control block (using TIMER^CONTROL^BLOCKX^DEF).
- 3) Declare the timer table large enough to contain the maximum number of timers, plus any user data associated with timer elements. This timer table should be allocated in an extended memory segment.
- 4) CALL TIMER^INITX before calling any other TIMER procedures.

Example:

```
LITERAL    timer^user^buf^lgth^l  = 4, ! size in words of userbuf
           num^of^timers^l         = 100,
           timer^entry^size^l      = $len( timerx^def ) / 2 +
                                   timer^user^buf^lgth^l - 1,
           timer^tbl^limit^l       = num^of^timers^l * timer^entry^size^l - 1;
```

```
?source = aciutils_utildefs( timer^structs )
```

```
STRUCT    .ext tcb( timer^control^blockx^def );
INT       .ext timer^tbl;
```

```
?source = aciutils_utilexts( timer^deletex, timer^insertx, timer^initx, ... )
```

```
PROC main^proc MAIN;
BEGIN
LITERAL    req^timer^l             = 0,
           saf^timer^l             = 1,
           wft^timer^l             = 2,
           fetimedout              = 40;
INT        error,
           file,
           .ext timer( timerx^def );
INT(32)    timer^wait;
```

```
! allocate extended memory segment and
! assign tcb & timer^tbl appropriate addresses
@tcb := base_addr;
@timer^tbl := base_addr + $len( tcb );
```

```
CALL timer^initx( tcb, timer^tbl, num^of^timers^l, timer^user^buf^lgth^l );
CALL timer^insertx( tcb, 100d, saf^timer^l );
CALL timer^insertx( tcb, 55d, wft^timer^l );
CALL timer^insertx( tcb, 300d, req^timer^l );
```

```
file      := -1;
```

Timer Routines

Extended Memory Segment Timer Routines

timer^wait := next^expire^timex(tcb);

CALL awaitio(file , , , timer^wait);

IF <> THEN

BEGIN

CALL fileinfo(file, error);

IF error = fetimedout THEN

BEGIN

@timer := timer^expiredx(tcb, true);

WHILE @timer <> 0 DO

BEGIN

CALL process^timeout(timer);

@timer := timer^expiredx(tcb, true);

END;

END

ELSE

! handle file error, other than timeout ...

END

ELSE

! handle normal i/o completion ...

DELETE^THIS^TIMERX

Designed to be used after a call to timer^expiredx or find^specific^timerx, when the location of the timer that is to be deleted is known.

Syntax:

<status> := DELETE^THIS^TIMERX(<tcb>, <timer>);

where:

<status>, INT,	returns true if the timer was deleted, and false otherwise.
<tcb>,INT .EXT:ref,	contains the timer control block.
<timer>, INT .EXT:ref,	a pointer returned by one of the following: - find^specific^timerx - timer^insertx - timer^expiredx

example:

```
@timer := find^specific^timerx( tcb, type^1 );  
IF not DELETE^THIS^TIMERX( tcb, timer ) THEN  
  BEGIN  
    CALL log^message( timer^not^found );  
  END;
```

FIND^SPECIFIC^TIMERX

Used to find the first timer (i.e., the one with the earliest expiration time), which matches the calling parameters.

Syntax:

<timer^addr> := FIND^SPECIFIC^TIMERX(<tcb>, <typ>, <subtyp>, <userbuf>, <userbufsize>)

where:

<timer^addr>, INT(32),	contains the address of the found timer. Returns FALSE(0d) if no timer was found.
<tcb>,INT .EXT:ref,	contains the timer control block.
<typ>, INT,	is the type of timer to be returned. Type is an integer value specified when the timer is inserted.
<subtyp>, INT,	is the subtype of timer to be returned. Subtype is an integer value specified when the timer is inserted.
<userbuf>, INT .EXT:ref,	only timers with <timer.userbuf> equal to userbuf for the length of <userbufsize> will be returned.
<userbufsize>, INT,	word length of the comparison between <userbuf> parameter and <timer.userbuf> data area that must match.

This is an extensible type procedure. Only <tcb> is required. The rest of the parameters are optional. If no additional parameters are specified, no timer will be located.

If <userbuf> or <userbufsize> are specified, both are required.

example:

```
@timer := FIND^SPECIFIC^TIMERX( tcb, typ, subtype, userbuf, userbufsize );
```

NEXT^EXPIRE^TIMEX

Returns the time interval, in hundredths of a second, which must pass before the next timer expires.

Syntax:

<wait^time> := NEXT^EXPIRE^TIMEX(<tcb>)

where:

<wait^time>, INT(32), contains 0d if a timer has already expired. If there are no timers in the timer list, -1d is returned.

The value that is returned may be passed directly as the <wait time> parameter in the AWAITIO[X] procedure.

<tcb>, INT .EXT:ref, contains the timer control block.

example:

```
file := -1;  
wait^time := NEXT^EXPIRE^TIMEX( tcb );  
CALL awaitio( file, , , timer^wait );
```

TIMER^ALLOCATEX

Due to the larger number of timers possible using extended memory it is not required that all the timers be initialized by TIMER^INITX. If only a subset of timers are initialized by TIMER^INITX, TIMER^ALLOCATEX can be called to allocate an additional block of timers. If there are unallocated timers in the extended memory table, and there are no available allocated timers for TIMER^INSERTX to use, TIMER^INSERTX will automatically call TIMER^ALLOCATEX to build additional timers in the available timer list.

Syntax:

CALL TIMER^ALLOCATEX(<tcb>, <num^to^allocate>)

where:

<tcb>, INT .EXT:ref,	contains the timer control block.
<num^to^allocate>, INT(32),	is the number of timers to allocate. If omitted, the number is determined by the value specified by the <alloc^block^size> param in the call to TIMER^INITX.

example:

CALL TIMER^ALLOCATEX(tcb);

TIMER^DELETEx

Used to delete one or more timers from the timer list. Provides the user the ability to pass a user defined procedure to perform additional cleanup related to the deleted timers.

This extended memory timer routine combines functionality similar to TIMER^DELETE and TIMER^DELETE^1. This extended memory version treats the userproc in TIMER^DELETE^1 as an optional parameter. The userproc is only called if it is passed as an argument. Otherwise, the functionality is similar to TIMER^DELETE.

Syntax:

<error> := TIMER^DELETEx(<tcb>, <typ>, <subtyp>, <userbuf>, <userbufsize>, <user proc>);

where:

<error>, INT,	contains true (-1) only if all the required parameters are there and if all parameters are valid.
<tcb>, INT .EXT:ref,	contains the timer control block.
<typ>, INT,	is the type of timer to be deleted. Type is an integer value specified when the timer is inserted.
<subtyp>, INT,	is the subtype of timer to be deleted. Subtype is an integer value specified when the timer is inserted.
<userbuf>, INT .EXT:ref,	only timers with <timer.userbuf> equal to userbuf for the length of <userbufsize> will be deleted.
<userbufsize>, INT,	word length of the comparison between <userbuf> parameter and <timer.userbuf> data area that must match.

<user proc>, INT, proc to be called with each matching timer found. Must be declared
INT proc userproc(timer);
INT .ext timer(timerx^def);

This is an extensible type procedure. Only <tcb> is required. The rest of the parameters are optional. If no additional parameters are specified, no timer will be deleted.

If <userbuf> or <userbufsize> are specified, both are required.

The <user proc> should return true (-1) if the search for more timers is to continue. It can be used to perform cleanup tasks when a timer is deleted. Such tasks might include deallocating memory that was reserved for the given timer.

example:

CALL TIMER^DELETEx(tcb, typ, subtyp, userbuf, userbufsize, userproc);

TIMER^EXPIREDX

Checks if any timer has expired. Only the first timer is checked. If more than one timer has expired, TIMER^EXPIREDX must be called once for each timer that has expired, until false (0) is returned.

Syntax:

<timer^addr> := TIMER^EXPIREDX(<tcb>, <delete>)

where:

<timer^addr>, INT(32),	will contain the address of the timer if it expired. Otherwise, FALSE (0) is returned.
<tcb>, INT .EXT:ref,	contains the timer control block.
<delete>, INT,	is used to delete a timer that has expired. If omitted, the timer will not be deleted.

example:

```
DO
  BEGIN
    @timer := TIMER^EXPIREDX( tcb, true );
    IF @timer <> 0d THEN
      BEGIN
        ! process expired timer
      END;
    END
  UNTIL @timer = 0d;
```

TIMER^INITX

Used to initialize the timer table. This proc must be called before using any of the other timer procedures.

Syntax:

CALL TIMER^INITX(< tcb>, < timer^tbl> , < num^timers> , < user^buf^size> ,
< alloc^block^size>)

where:

< tcb>, INT .EXT:ref,	contains the timer control block.
< timer^tbl>, INT .EXT:ref,	is the timer table declared in extended memory.
< num^timers>, INT(32),	is the maximum number of timers that timer^tbl can hold.
< user^buf^size>, INT,	is the word length of the user buffer area of the timer entry. If no user buffer is needed, the value of the parameter may be 0.
< alloc^block^size>, INT(32),	indicates how many timers allocate in the available timer pool initially, and with each subsequent call to TIMER^ALLOCATEX. -1d – allocate all timers 0d – allocate 1000 timers any other positive number – indicates specific number to allocate.

This call initializes the timer control block, and calls TIMER^ALLOCATEX to allocate all or a subset of the timers to create a pool of available allocated timers. This allocation doesn't allocate memory; rather it adds timer elements to a linked list of available timers. When there are no timer elements in the linked list, TIMER^INSERTX will call TIMER^ALLOCATEX if the total number of timers allocated is less than < num^timers>.

The < alloc^block^size> is intended to provide some flexibility when the number of timers in use creates a performance issue by initializing them all up front. For the use of fewer timers, it is most likely best to specify -1d and allocate them all initially.

The user data area of the timers does not have to match for each timer type and subtype. If multiple layouts are to be used, specify the word size of the largest user data structure. The size specified here will be allocated at the end of every timer structure to provide this user data area to each timer.

example:

```
CALL TIMER^INITX( tcb, timer^tbl, max^timers, user^buf^size, -1d );
```

TIMER^INSERTX

Used to insert an interval timer in the timer list.

Syntax:

<timer^addr> := TIMER^INSERTX(<tcb>, <delay^time> , <typ> , <subtyp>)

where:

<timer^addr>, INT(32),	contains the address of the new timer. Returns FALSE (0d) if no unused timers were available.
------------------------	---

<tcb>, INT .EXT:ref,	contains the timer control block.
----------------------	-----------------------------------

<delay^time>, INT (32),	is the time interval, in hundredths of a second, for the added timer.
-------------------------	--

<typ>, INT,	is the type of timer to be inserted.
-------------	--------------------------------------

<subtyp>, INT,	is the subtype of timer to be inserted.
----------------	---

example:

```
@timer := TIMER^INSERTX( tcb, delay^time, typ, subtyp );  
timer.userbuf := some^value;  
OR  
@user^area := @timer.userbuf;  
! manipulate user^area structure
```

Miscellaneous Procs

example:

```
STRING .ext tbl,  
      .ext key;  
INT      num^entries,  
        entry^lgth,  
        keyoff,  
        keylgth;  
INT(32)  addr;  
  
addr      := BINARY^SEARCH( tbl, num^entries,  
                           entry^lgth, key, keyoff, keylgth );
```

NOTE: The BASE utility UT^BINARY^SEARCH should be used instead of BINARY^SEARCH. UT^BINARY^SEARCH is faster and more efficient.

CIRC^LEFT^SHIFT

Performs a circular left shift of the 64 bit value starting at a given parameter for a specified length. As the bits are shifted out at the left of the field, they are shifted into the right most bits of the field.

Syntax:

CALL CIRC^LEFT^SHIFT(<acc> , <num^bits>)

where:

<acc>, INT:ref:4, is the array to be shifted.

<num^bits>, INT, is the number of bits to shift. This value is assumed to be a multiple of 4.

example:

```
INT      .array[ 0:3 ],
          num^bits;

num^bits  := 16;
CALL CIRC^LEFT^SHIFT( array, num^bits );
```

CIRC^RIGHT^SHIFT

Performs a circular right shift of the 64 bit value starting at "ACC". The procedure performs the shifting in multiples of 4 bits. It receives as input the number of bits to be shifted and the bit string to be shifted. The result of the operation is returned in "ACC". The shift operation is "logical" rather than "arithmetic".

Syntax:

CALL CIRC^RIGHT^SHIFT(<acc> , <num^bits>)

where:

<acc>, INT:ref, contains the bit string to shift. It is assumed to be 64 bits.

<num^bits>, INT, contains the number of bits to be shifted in multiples of 4.

example:

CALL CIRC^RIGHT^SHIFT(acc, num^bits);

COMPLEMENT

Takes an array and complements it.

Syntax:

CALL COMPLEMENT(<buf> , <cnt>)

where:

<buf>, INT:ref, contains the array to complement.

<cnt>, INT, contains the number of words to
complement.

example:

CALL COMPLEMENT(buf , cnt);

CONVERT^REC^ADDR^TO^REF^NO

Converts the record's address to the record's reference number. The reference number of a record is its relative position in an entry sequenced file where all records are of equal (fixed lengths).

Syntax:

```
CALL CONVERT^REC^ADDR^TO^REF^NO( <rec^addr>, <ref^no> ,  
<blk^lgth>, <blk^hdr^lgth> ,  
<rec^lgth> );
```

where:

<rec^addr>, INT(32): val	is the record's disk address.
<ref^no>, INT(32): ref	is the reference number of the record.
<blk^lgth>, INT: val	length of blocks in bytes.
<blk^hdr lgth>, INT: val	length of the block header in bytes.
<rec^lgth>, INT: val	length of the record in bytes.

example:

```
LITERAL    blk^lgth           = 4096,  
            blk^hdr^lgth       = 20,  
            rec^size           = $len( rec^def );  
INT(32)    ref^no,  
            rec^addr;  
  
CALL CONVERT^REC^ADDR^TO^REF^NO( rec^addr,ref^no,  
                                   blk^lgth, blk^hdr^lgth,  
                                   rec^size );
```

CONVERT^REF^NO^TO^REC^ADDR

Converts the reference number of a record to a record address that can be used to position into the file and read the record. The reference number of a record is its relative position in an entry sequenced file where all records are of equal (fixed lengths).

Syntax:

```
CALL CONVERT^REF^NO^TO^REC^ADDR( <ref^num>, <rec^addr> ,  
<blk^lgth>, <blk^hdr^lgth> ,  
<rec^lgth> );
```

where:

<ref^num>, INT(32): val	is the reference number of the record.
<rec^addr>, INT(32): ref	is the record's disk address.
<blk^lgth>, INT: val	length of blocks in bytes.
<blk^hdr^lgth>, INT: val	length of the block header in bytes.
<rec^lgth>, INT: val	length of the record in bytes.

example:

LITERAL	blk^lgth	= 4096,
	blk^hdr^lgth	= 20,
	rec^size	= \$len(rec^def);
INT(32)	ref^no,	
	rec^addr;	

```
CALL CONVERT^REF^NO^TO^REC^ADDR( ref^no, rec^addr,  
                                   blk^lgth,  
                                   blk^hdr^lgth,  
                                   rec^size );
```

CURRENT^ADDR

Returns the 'P' relative address of the caller, and is used mainly for logging errors detected.

Syntax:

<stat> := CURRENT^ADDR(<e^reg> , <p^reg>)

where:

<stat>, INT, contains the caller's 'P' register value.
Should only be used by programs which "know" that they are a single code segment with no library; the p^reg alone contains insufficient data for multi-segment object files. Single segment programs could do this: `wlform(convfail, "ASCII-INTEGGER failed, 'P'= %@@@@@")`

<e^reg>, INT:ref, contains the contents of the E register.

<p^reg>, INT:ref, contains the contents of the P register.

example:

If a call to ASCII^INTEGER failed, there could be one standard wlform:

`wlform(conv^fail, "ASCII^INTEGER failed, location\C")`

that can be used throughout the program, with the same message number for logger:

```
INT    .ext e^reg,  
        .ext p^reg;
```

```
CALL CURRENT^ADDR( e^reg, p^reg );  
CALL LOGMESSAGE^( 9101, @conv^fail,,, e^reg, p^reg  
                  );
```

Note: Consider using the HIST_procedures as described in the Guardian Proc Calls Manual instead.

DBL^OCTAL^WIDTH

Returns the number of character positions necessary to print the octal representation of an INT(32) number, thus giving an alternate to zero suppression, using only those bytes needed to print a number.

Syntax:

<width> := DBL^OCTAL^WIDTH(<val>)

where:

<width>, INT, is the number of character positions necessary to print <value>.

<val>, INT(32), is the octal value to be measured by DBL^OCTAL^WIDTH.

example:

INT	width;
INT(32)	octal^val;
octal^val	:= 7d;
width	:= DBL^OCTAL^WIDTH(octal^val);

DBL^WIDTH

Returns the number of character positions necessary to print an INT(32) number, thus giving an alternative to zero suppression, using only those bytes needed to print a number.

Syntax:

<width> := DBL^WIDTH(<val>)

where:

<width>, INT, is the number of character positions
 necessary to print <value>.

<val>, INT(32), is the value to be measured by
 DBL^WIDTH.

example:

INT	width;
INT(32)	val;
val	:= 226d;
width	:= DBL^WIDTH(val);

EDITLINENO

EDITLINENO places into a string array the character version of an EDIT file sequence number sent as an argument. Since EDIT file sequence numbers are placed into INT(32) variables any similar value can of course be converted.

Syntax:

CALL EDITLINENO(<outstr>, <seq^num>)

where:

<outstr>, STRING:ref:9, is an array into which the converted sequence number is to be placed. Leading and trailing zeroes are suppressed. The decimal point is always in position <outstring>[5], whether the value returned is zero, less than one or a whole number.

<seq^num>, INT(32):val, is the value that is to be converted. Edit file sequence numbers are in the form "n.nnn", so the number is divided by 1000 to obtain the whole number and the remainder is the fraction.

example:

```
STRING      .out^string[ 0:8 ];  
INT(32)     seq^num;  
  
seq^num     := 12345d;  
CALL EDITLINENO( out^string, seq^num );
```

FUNCTION^KEY

Returns a value from -16 to +16 for the values of the function keys or an error value. Value 0 indicates that break was pressed. If an error occurs, it is added to 1000 before returning. This prevents errors between 1 and 16 being confused with valid function keys. The parameter error contains the actual error number. Negative numbers are returned for shifted function keys.

Syntax:

```
<key^num> := FUNCTION^KEY( <file^num>, <err> )
```

where:

<key^num>, INT,	contains the number of the function key. Negative numbers are returned for shifted keys. If an error occurs, the error number is added to 1000 before returning.
-----------------	--

<file^num>, INT,	contains the file number of the file to be read.
------------------	--

<err>, INT:ref,	contains a value for any error. Contains 0 if no error occurs.
-----------------	--

example:

```
INT      function^key^num,  
         file^num,  
         err;
```

```
function^key^num := FUNCTION^KEY( file^num, error );
```

GET^NEXT^ARG

Makes it possible to obtain the next of a series of elements from an array, with any specified field separator.

Syntax:

<addr> := GET^NEXT^ARG(<ptr>, <outbuf>, <lgth>, <delimiter>)

where:

<addr>, INT,	contains the address of the next element in the list if there is one, or FALSE if there is not.
<ptr>, STRING:ref,	is the pointer to an array containing the (remaining) elements in the list. This array must be delimited by a null byte at the end. The PROC will count anything contained in parentheses, single or double quotes, as a single element.
<outbuf>, STRING:ref,	is the array into which the next element is to be placed. This array remains unchanged if no elements remain.
<lgth>, INT:ref,	on entry, contains the maximum number of bytes to be fetched -- normally, the length of <outbuf>. On return to the calling program, <lgth> contains the length of the element fetched or -1 if no element was fetched.
<delimiter>, STRING:value,	is the character used as field separator in the array pointed at by <ptr>. If omitted, a comma is used as the default delimiter.

examples:

```
STRING      .str^array[ 0:10 ],
             .output^array[ 0:3 ];
INT         .addr,
             lgth,
             delimiter;

str^array[ 10 ]  ':[ 0 ];
lgth            := 11;
delimiter       := "#";
str^array       := "hi # bye #";
@addr          := GET^NEXT^ARG( str^array,
                                output^array,
                                lgth, delimiter );
```

MOTHER

Provides the process name or identification number of its creator and/or the creator's program file name.

Syntax:

CALL MOTHER(<mom^name>, <mom^prog>)

where:

<mom^name>, INT:ref:4,	if present, is an array where this process returns the process ID or the process name of the caller's creator. If <mom^name> exists it will be in the format:
<mom^name>[0:2]	= \$<process name>
<mom^name>[3].<0:7>,<8:15>	= <cpu,pin>
<mom^prog>, INT:ref:12,	if present, contains the program file name of the caller's creator in format:
<mom^prog>[0:3]	= \$<vol name>
<mom^prog>[4:7]	= <subvol name>
<mom^prog>[8:11]	= <file name>

example:

```
INT          .mom^name[ 0:11 ],  
             .mom^prog[ 0:11 ];
```

```
CALL MOTHER( mom^name, mom^prog );
```

Note: Not Highpin enabled.

MSG^AGE

Returns the number of seconds that a message has been in our queue until we read it.

Syntax:

<elapsed> := MSG^AGE(<msg^time>)

where:

<elapsed>, INT(32), contains the number of seconds elapsed
between the time net delivered a message
to us, and the time we actually read it from
our queue.

<msg^time>, INT:ref, contains the 3 word timestamp from the
message header.

example:

```
INT(32)    lgth;  
INT        .msg^time;
```

```
IF MSG^AGE( msg^time ) > 'an INT(32) value' THEN  
    ...
```

OCTAL^WIDTH

Returns the number of character positions necessary to print the octal representation of an octal number, thus giving an alternative to zero suppression, using only the bytes needed to print a number.

Syntax:

<width> := OCTAL^WIDTH(<val>)

where:

<width>, INT, is the number of character positions
necessary to print <value>.

<val>, INT, is the octal value to be measured by
OCTAL^WIDTH.

example:

INT	width, octal^val;
octal^val	:= 226;
width	:= OCTAL^WIDTH(octal^val);

RANDOM

Accepts a double word "seed" and returns a double word random number. When the seed for this call is the random number from the last call, the number generated does not fall into a repeating pattern for several million calls.

Syntax:

CALL RANDOM(<num> , <seed>)

where:

<num>, INT(32):ref,	contains the generated "random" number upon return from the PROC.
<seed>, INT(32):ref,	is the "seed" used to generate the "random" number.

example:

```
INT(32)    random^num,  
           seed;  
  
seed      := 2389d;  
CALL RANDOM( random^num, seed );
```

SHA1^HASH

This procedure generates a 20-byte hash value from an input string using the SHA-1 (Secure Hash Algorithm 1) function.

Syntax:

<stat> := SHA1^HASH (<input^data>, <input^lgth>, <sha1^hash>)

where:

<stat>, INT,	returns TRUE (non-zero) if an error occurred.
<input-data>, STRING:ref	data to be hashed.
<input^lgth>, INT	the byte length of <input^data>.
<sha1^hash>, STRING:ref	the 20-byte hash value.

example:

stat := SHA1^HASH (input^data, input^lgth, sha1^hash);

UT^FNAMECOLLAPSE

Works exactly like Guardian's FNAMECOLLAPSE except that if there is not an expand system name on the returned external^name then it tries to put one there, even if the volume name is 8 characters long. This means the length of the external file name has to be 35 characters long instead of 34.

Syntax:

<retn^val> := UT^FNAMECOLLAPSE(internal^name , external^name)

where:

<retn^val>, INT,	contains the number of bytes in the external name.
<internal^name>, INT:ref,	contains the name to be converted.
<external^name>, STRING:ref,	contains the external form of the internal name after the procedure.

example:

```
retn^val := UT^FNAMECOLLAPSE( internal^name,  
                               external^name );
```

WIDTH

Returns the number of character positions necessary to print a given integer value, thus giving an alternative to zero suppression, using only the bytes needed to print a number.

Syntax:

<width> := WIDTH(<val>)

where:

<width>, INT, is the number of character positions
 necessary to print <value>.

<val>, INT, is the value to be measured by WIDTH.

example:

INT	width,
	val;
val	:= 1000;
width	:= WIDTH(val);